

config.py

```
import os
from datetime import timedelta

# Application Configuration
APP_NAME = "VulnEdu - Educational CVE Analysis Tool"
SECRET_KEY = os.environ.get('SECRET_KEY', 'dev-key-change-in-production')

# NVD API Configuration
NVD_API_URL = "https://services.nvd.nist.gov/rest/json/cves/2.0"
NVD_API_KEY = os.environ.get('NVD_API_KEY', '9d289859-60a3-4f9f-af3c-30fdcddf3918')

# Data Storage Paths
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, 'data')
CACHE_DIR = os.path.join(DATA_DIR, 'cache')
NVD_DIR = os.path.join(DATA_DIR, 'nvd')
NVD_HISTORICAL_DIR = os.path.join(NVD_DIR, 'historical')
NVD_PROCESSED_DIR = os.path.join(NVD_DIR, 'processed')
CWE_XML_PATH = os.path.join(DATA_DIR, 'CWE_Catalog.xml')

# NVD Data Feeds
NVD_FEEDS_BASE_URL = "https://nvd.nist.gov/feeds/json/cve/1.1"

# Historical years configuration - USE LOCAL FILES
# REDUCED to 3 years for memory efficiency
current_year = 2025
HISTORICAL_YEARS = [current_year - 2, current_year - 1, current_year] # 2023, 2024, 2025

# Cache Configuration
CACHE_DURATION = timedelta(minutes=30)
MAX_CACHE_ENTRIES = 100

# Data Loading Strategy
RECENT_DAYS_THRESHOLD = 30
USE_LOCAL_FEEDS = True # CHANGED: Use local files
AUTO_UPDATE_FEEDS = False
USE_GITHUB_DATA = False # CHANGED: Disable GitHub

# API Rate Limiting
API_REQUESTS_PER_30_SECONDS = 50
API_TIMEOUT = 30

# Pagination
DEFAULT_PAGE_SIZE = 25
MAX_PAGE_SIZE = 100

# Ensure directories exist
for directory in [CACHE_DIR, NVD_DIR, NVD_HISTORICAL_DIR, NVD_PROCESSED_DIR]:
    os.makedirs(directory, exist_ok=True)
```

project_to_pdf_highlighted.py

```
import os
from fpdf import FPDF
from pygments import highlight
from pygments.lexers import guess_lexer_for_filename, TextLexer
from pygments.formatters import HtmlFormatter

root_folder = "."

class PDF(FPDF):
    pass

def safe_text(text):
    """Replace unsupported Unicode characters with a placeholder."""
    return text.encode("latin-1", errors="replace").decode("latin-1")

def add_code_to_pdf(pdf, file_path, formatter):
    relative_path = os.path.relpath(file_path, root_folder)
    print(f"Adding: {relative_path}")

    with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
        code = f.read()

    try:
        lexer = guess_lexer_for_filename(file_path, code)
    except Exception:
        lexer = TextLexer()

    html_code = highlight(code, lexer, formatter)
    html_code = safe_text(html_code) # Replace unsupported characters

    pdf.add_page()
    pdf.set_font("Courier", "", 10) # Use default built-in font
    pdf.cell(0, 10, safe_text(relative_path), new_y="NEXT")
    pdf.ln(2)
    pdf.write_html(html_code)

def convert_project_to_pdf(root_folder, output_pdf="Project_Code.pdf"):
    pdf = PDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    formatter = HtmlFormatter(style="monokai", noclasses=True, linenos=False)

    for root, dirs, files in os.walk(root_folder):
        for file in files:
            if file.endswith((".py", ".js", ".html", ".css", ".cpp", ".java", ".ts", ".json", ".txt", ".md")):
                add_code_to_pdf(pdf, os.path.join(root, file), formatter)

    pdf.output(output_pdf)
    print(f"\n? Project converted into '{output_pdf}' successfully!")

if __name__ == "__main__":
    convert_project_to_pdf(root_folder)
```

requirements.txt

Web framework
flask>=2.3.3

HTTP client
requests>=2.31.0

HTML/XML parsing (only if you use it)
beautifulsoup4==4.12.2

Production WSGI server
gunicorn>=21.2.0

PostgreSQL adapter
psycopg2-binary>=2.9.9

vulnedu.py

```
from flask import Flask
from routes.main_routes import main_bp
from routes.api_routes import api_bp
from routes.learn_routes import learn_bp
from routes.utility_routes import utility_bp
import config
import os

def create_app():
    """Application factory pattern"""
    app = Flask(__name__)
    app.config['SECRET_KEY'] = config.SECRET_KEY

    # Register blueprints
    app.register_blueprint(main_bp)
    app.register_blueprint(api_bp, url_prefix='/api')
    app.register_blueprint(learn_bp, url_prefix='/learn')
    app.register_blueprint(utility_bp)

    # Health check endpoint for Render
    @app.route('/health')
    def health():
        return {'status': 'healthy', 'service': 'VulnEdu'}, 200

    return app

# Create the app instance at module level for Gunicorn
app = create_app()

if __name__ == "__main__":
    # This only runs in local development
    print("[Flask] Starting VulnEdu in DEVELOPMENT mode...")

    # Create required directories
    for directory in [config.CACHE_DIR, config.NVD_DIR, config.NVD_HISTORICAL_DIR,
                     config.NVD_PROCESSED_DIR]:
        os.makedirs(directory, exist_ok=True)

    # Run with basic settings for local dev
    app.run(host='0.0.0.0', port=5000, debug=True)
```

.vscode\settings.json

```
{  
  "python-envs.defaultEnvManager": "ms-python.python:system",  
  "python-envs.pythonProjects": []  
}
```

data\nvd\processed\yearly_summaries.json

```
{
  "yearly_summaries": {
    "2010": {
      "year": 2010,
      "count": 5244,
      "monthly_counts": {
        "2010-01": 155,
        "2010-02": 245,
        "2010-03": 396,
        "2010-04": 404,
        "2010-05": 374,
        "2010-06": 449,
        "2010-07": 288,
        "2010-08": 356,
        "2010-09": 287,
        "2010-10": 417,
        "2010-11": 274,
        "2010-12": 348,
        "2011-01": 249,
        "2011-02": 91,
        "2011-03": 38,
        "2011-04": 36,
        "2011-05": 12,
        "2011-06": 3,
        "2011-07": 14,
        "2011-08": 8,
        "2011-09": 22,
        "2011-10": 116,
        "2011-11": 95,
        "2011-12": 9,
        "2012-01": 1,
        "2012-02": 7,
        "2012-03": 1,
        "2012-04": 1,
        "2012-05": 8,
        "2012-06": 5,
        "2012-08": 73,
        "2012-09": 88,
        "2012-10": 11,
        "2012-11": 8,
        "2012-12": 1,
        "2013-01": 1,
        "2013-03": 1,
        "2013-06": 1,
        "2013-08": 1,
        "2013-09": 1,
        "2013-10": 1,
        "2013-11": 1,
        "2013-12": 3,
        "2014-01": 8,
        "2014-02": 3,
        "2014-04": 3,
        "2014-05": 3,
        "2014-06": 3,
        "2014-08": 3,
        "2014-10": 2,
        "2014-11": 2,
        "2014-12": 7,
        "2015-01": 7,
        "2015-03": 1,
        "2015-06": 2,
        "2015-08": 5,
        "2016-04": 1,
        "2016-05": 1,
        "2017-01": 1,
        "2017-02": 1,
        "2017-03": 1,
        "2017-04": 5,
        "2017-05": 99,
        "2017-07": 8,
        "2017-08": 2,
        "2017-09": 2,

```

```
"2017-10": 2,
"2018-02": 1,
"2019-03": 2,
"2019-06": 1,
"2019-07": 2,
"2019-09": 1,
"2019-10": 29,
"2019-11": 51,
"2020-01": 4,
"2020-02": 5,
"2020-11": 5,
"2021-05": 1,
"2021-06": 15,
"2021-10": 1,
"2022-03": 1,
"2023-01": 8,
"2023-06": 1,
"2023-09": 16,
"2024-01": 1,
"2025-07": 1,
"2025-08": 26
},
"cwe_counts": {
  "CWE-79": 611,
  "CWE-89": 588,
  "CWE-119": 521,
  "CWE-264": 325,
  "CWE-20": 320,
  "CWE-22": 275,
  "CWE-94": 243,
  "CWE-399": 217,
  "CWE-200": 170,
  "CWE-189": 125,
  "CWE-362": 76,
  "CWE-352": 76,
  "CWE-310": 64,
  "CWE-255": 60,
  "CWE-287": 57,
  "CWE-787": 41,
  "CWE-59": 33,
  "CWE-416": 30,
  "CWE-476": 25,
  "CWE-190": 23,
  "CWE-16": 21,
  "CWE-121": 19,
  "CWE-120": 17,
  "CWE-134": 14,
  "CWE-78": 14,
  "CWE-400": 11,
  "CWE-863": 7,
  "CWE-909": 7,
  "CWE-908": 6,
  "CWE-77": 5,
  "CWE-426": 5,
  "CWE-295": 5,
  "CWE-843": 4,
  "CWE-732": 4,
  "CWE-193": 4,
  "CWE-269": 4,
  "CWE-601": 4,
  "CWE-835": 3,
  "CWE-798": 3,
  "CWE-191": 3,
  "CWE-415": 3,
  "CWE-404": 3,
  "CWE-772": 3,
  "CWE-434": 3,
  "CWE-74": 3,
  "CWE-312": 2,
  "CWE-749": 2,
  "CWE-401": 2,
  "CWE-917": 2,
  "CWE-502": 2
},
"severity_counts": {
  "UNKNOWN": 4961,
  "HIGH": 133,
```

```
"MEDIUM": 91,
"CRITICAL": 51,
"LOW": 8
},
"2011": {
  "year": 2011,
  "count": 4886,
  "monthly_counts": {
    "2011-01": 140,
    "2011-02": 283,
    "2011-03": 296,
    "2011-04": 265,
    "2011-05": 279,
    "2011-06": 282,
    "2011-07": 293,
    "2011-08": 274,
    "2011-09": 350,
    "2011-10": 358,
    "2011-11": 215,
    "2011-12": 331,
    "2012-01": 168,
    "2012-02": 100,
    "2012-03": 61,
    "2012-04": 38,
    "2012-05": 89,
    "2012-06": 67,
    "2012-07": 65,
    "2012-08": 81,
    "2012-09": 65,
    "2012-10": 52,
    "2012-11": 22,
    "2012-12": 7,
    "2013-01": 7,
    "2013-02": 13,
    "2013-03": 15,
    "2013-05": 4,
    "2013-06": 20,
    "2013-07": 7,
    "2013-08": 2,
    "2013-09": 1,
    "2013-10": 30,
    "2013-11": 1,
    "2013-12": 11,
    "2014-01": 6,
    "2014-02": 36,
    "2014-03": 13,
    "2014-04": 19,
    "2014-05": 6,
    "2014-06": 6,
    "2014-08": 2,
    "2014-09": 1,
    "2014-10": 5,
    "2014-12": 13,
    "2015-01": 34,
    "2015-03": 1,
    "2015-04": 1,
    "2015-08": 3,
    "2016-04": 1,
    "2016-05": 2,
    "2017-04": 2,
    "2017-05": 136,
    "2017-07": 9,
    "2017-08": 4,
    "2017-09": 2,
    "2017-10": 6,
    "2017-12": 1,
    "2018-01": 1,
    "2018-02": 5,
    "2018-03": 1,
    "2018-05": 1,
    "2018-06": 6,
    "2018-07": 3,
    "2018-08": 2,
    "2018-09": 1,
    "2019-04": 4,
    "2019-07": 1,
```



```

"2019-08": 2,
"2019-10": 5,
"2019-11": 101,
"2019-12": 2,
"2020-01": 38,
"2020-02": 29,
"2020-03": 3,
"2020-06": 2,
"2020-11": 5,
"2021-06": 6,
"2021-10": 8,
"2022-04": 2,
"2022-07": 1,
"2022-09": 1,
"2023-01": 1,
"2023-02": 2,
"2023-09": 24,
"2023-10": 1,
"2024-01": 1,
"2024-04": 1,
"2025-06": 1,
"2025-07": 1,
"2025-08": 23
},
"cwe_counts": {
"CWE-119": 639,
"CWE-79": 529,
"CWE-20": 425,
"CWE-200": 345,
"CWE-264": 325,
"CWE-399": 277,
"CWE-89": 176,
"CWE-189": 121,
"CWE-22": 115,
"CWE-94": 112,
"CWE-416": 96,
"CWE-352": 89,
"CWE-287": 75,
"CWE-310": 71,
"CWE-59": 45,
"CWE-255": 44,
"CWE-787": 36,
"CWE-125": 36,
"CWE-16": 30,
"CWE-362": 29,
"CWE-476": 28,
"CWE-190": 22,
"CWE-78": 20,
"CWE-120": 20,
"CWE-400": 20,
"CWE-908": 12,
"CWE-134": 11,
"CWE-704": 11,
"CWE-121": 10,
"CWE-434": 9,
"CWE-295": 7,
"CWE-835": 6,
"CWE-269": 6,
"CWE-74": 6,
"CWE-863": 5,
"CWE-276": 5,
"CWE-415": 5,
"CWE-346": 5,
"CWE-426": 5,
"CWE-284": 5,
"CWE-306": 5,
"CWE-772": 5,
"CWE-732": 4,
"CWE-193": 3,
"CWE-843": 3,
"CWE-191": 3,
"CWE-326": 3,
"CWE-312": 3,
"CWE-254": 3,
"CWE-755": 3
},
"severity_counts": {

```

```
"HIGH": 129,  
"UNKNOWN": 4547,  
"MEDIUM": 136,  
"CRITICAL": 68,  
"LOW": 6  
},  
},  
"2012": {  
  "year": 2012,  
  "count": 5937,  
  "monthly_counts": {  
    "2012-01": 145,  
    "2012-02": 239,  
    "2012-03": 338,  
    "2012-04": 186,  
    "2012-05": 337,  
    "2012-06": 339,  
    "2012-07": 466,  
    "2012-08": 566,  
    "2012-09": 501,  
    "2012-10": 495,  
    "2012-11": 415,  
    "2012-12": 261,  
    "2013-01": 231,  
    "2013-02": 99,  
    "2013-03": 83,  
    "2013-04": 40,  
    "2013-05": 20,  
    "2013-06": 17,  
    "2013-07": 18,  
    "2013-08": 32,  
    "2013-09": 22,  
    "2013-10": 57,  
    "2013-11": 6,  
    "2013-12": 22,  
    "2014-01": 23,  
    "2014-02": 29,  
    "2014-03": 11,  
    "2014-04": 46,  
    "2014-05": 17,  
    "2014-06": 23,  
    "2014-07": 3,  
    "2014-08": 10,  
    "2014-09": 45,  
    "2014-10": 15,  
    "2014-11": 7,  
    "2014-12": 5,  
    "2015-01": 2,  
    "2015-02": 2,  
    "2015-04": 4,  
    "2015-05": 8,  
    "2015-06": 2,  
    "2015-08": 5,  
    "2016-04": 3,  
    "2016-05": 2,  
    "2016-06": 2,  
    "2016-12": 1,  
    "2017-03": 20,  
    "2017-04": 2,  
    "2017-05": 269,  
    "2017-06": 3,  
    "2017-07": 4,  
    "2017-08": 9,  
    "2017-09": 1,  
    "2017-10": 16,  
    "2017-11": 1,  
    "2017-12": 1,  
    "2018-01": 9,  
    "2018-02": 10,  
    "2018-05": 1,  
    "2018-06": 1,  
    "2018-07": 1,  
    "2018-10": 1,  
    "2019-05": 1,  
    "2019-06": 1,  
    "2019-07": 1,  
    "2019-08": 7,  
  }  
}
```

```
"2019-10": 9,  
"2019-11": 59,  
"2019-12": 32,  
"2020-01": 71,  
"2020-02": 56,  
"2020-03": 3,  
"2020-05": 2,  
"2020-07": 20,  
"2020-09": 5,  
"2020-11": 6,  
"2020-12": 1,  
"2021-01": 1,  
"2021-07": 10,  
"2021-12": 1,  
"2022-09": 3,  
"2023-01": 5,  
"2023-02": 2,  
"2023-03": 1,  
"2023-04": 7,  
"2023-05": 1,  
"2023-09": 35,  
"2023-10": 1,  
"2023-12": 1,  
"2024-06": 1,  
"2024-10": 1,  
"2025-07": 3,  
"2025-08": 41  
},  
"cwe_counts": {  
"CWE-79": 848,  
"CWE-119": 662,  
"CWE-264": 571,  
"CWE-20": 390,  
"CWE-200": 254,  
"CWE-89": 237,  
"CWE-399": 219,  
"CWE-352": 166,  
"CWE-94": 136,  
"CWE-22": 126,  
"CWE-189": 125,  
"CWE-287": 113,  
"CWE-310": 111,  
"CWE-255": 60,  
"CWE-78": 44,  
"CWE-416": 43,  
"CWE-787": 30,  
"CWE-362": 30,  
"CWE-434": 21,  
"CWE-400": 20,  
"CWE-295": 20,  
"CWE-59": 19,  
"CWE-269": 18,  
"CWE-16": 18,  
"CWE-190": 17,  
"CWE-134": 13,  
"CWE-284": 13,  
"CWE-611": 9,  
"CWE-121": 9,  
"CWE-476": 8,  
"CWE-125": 7,  
"CWE-522": 7,  
"CWE-798": 7,  
"CWE-120": 6,  
"CWE-276": 6,  
"CWE-863": 5,  
"CWE-74": 5,  
"CWE-732": 5,  
"CWE-843": 3,  
"CWE-77": 3,  
"CWE-502": 3,  
"CWE-862": 3,  
"CWE-306": 3,  
"CWE-668": 2,  
"CWE-311": 2,  
"CWE-835": 2,  
"CWE-601": 2,  
"CWE-23": 2,
```

```
"CWE-326": 2,
"CWE-19": 2
},
"severity_counts": {
  "UNKNOWN": 5475,
  "CRITICAL": 103,
  "HIGH": 189,
  "MEDIUM": 159,
  "LOW": 11
}
},
"2013": {
  "year": 2013,
  "count": 6819,
  "monthly_counts": {
    "2013-01": 202,
    "2013-02": 298,
    "2013-03": 339,
    "2013-04": 402,
    "2013-05": 330,
    "2013-06": 331,
    "2013-07": 458,
    "2013-08": 343,
    "2013-09": 451,
    "2013-10": 544,
    "2013-11": 391,
    "2013-12": 459,
    "2014-01": 328,
    "2014-02": 132,
    "2014-03": 186,
    "2014-04": 120,
    "2014-05": 152,
    "2014-06": 66,
    "2014-07": 19,
    "2014-08": 20,
    "2014-09": 13,
    "2014-10": 16,
    "2014-11": 23,
    "2014-12": 23,
    "2015-01": 11,
    "2015-02": 5,
    "2015-03": 5,
    "2015-04": 21,
    "2015-05": 1,
    "2015-06": 2,
    "2015-08": 6,
    "2015-09": 1,
    "2015-10": 1,
    "2015-11": 1,
    "2015-12": 1,
    "2016-02": 3,
    "2016-04": 1,
    "2016-05": 1,
    "2016-06": 1,
    "2016-07": 1,
    "2016-08": 2,
    "2016-10": 2,
    "2016-12": 1,
    "2017-01": 4,
    "2017-02": 1,
    "2017-03": 14,
    "2017-04": 5,
    "2017-05": 346,
    "2017-08": 6,
    "2017-09": 2,
    "2017-10": 7,
    "2017-11": 1,
    "2017-12": 3,
    "2018-01": 1,
    "2018-02": 8,
    "2018-04": 8,
    "2018-05": 12,
    "2018-06": 1,
    "2018-07": 20,
    "2018-08": 1,
    "2018-09": 2,
    "2018-10": 1,
```

```
"2019-02": 4,  
"2019-03": 6,  
"2019-04": 1,  
"2019-05": 1,  
"2019-06": 3,  
"2019-08": 11,  
"2019-10": 17,  
"2019-11": 95,  
"2019-12": 69,  
"2020-01": 131,  
"2020-02": 122,  
"2020-03": 2,  
"2020-04": 1,  
"2020-06": 1,  
"2020-07": 2,  
"2020-09": 2,  
"2020-11": 5,  
"2021-01": 2,  
"2021-02": 1,  
"2021-04": 2,  
"2021-05": 1,  
"2021-06": 1,  
"2021-08": 6,  
"2022-02": 2,  
"2022-05": 4,  
"2022-06": 28,  
"2022-10": 2,  
"2022-12": 1,  
"2023-01": 9,  
"2023-02": 5,  
"2023-03": 2,  
"2023-04": 4,  
"2023-05": 1,  
"2023-06": 4,  
"2023-09": 75,  
"2023-12": 1,  
"2025-06": 2,  
"2025-07": 12,  
"2025-08": 25  
},  
"cwe_counts": {  
"CWE-79": 801,  
"CWE-119": 754,  
"CWE-264": 611,  
"CWE-20": 550,  
"CWE-200": 301,  
"CWE-399": 264,  
"CWE-89": 197,  
"CWE-352": 195,  
"CWE-94": 172,  
"CWE-287": 167,  
"CWE-310": 152,  
"CWE-22": 150,  
"CWE-189": 130,  
"CWE-255": 94,  
"CWE-362": 68,  
"CWE-78": 55,  
"CWE-416": 50,  
"CWE-59": 38,  
"CWE-787": 36,  
"CWE-190": 24,  
"CWE-434": 24,  
"CWE-74": 21,  
"CWE-269": 19,  
"CWE-120": 17,  
"CWE-16": 17,  
"CWE-284": 13,  
"CWE-134": 12,  
"CWE-863": 12,  
"CWE-798": 12,  
"CWE-400": 11,  
"CWE-476": 10,  
"CWE-522": 10,  
"CWE-326": 9,  
"CWE-125": 9,  
"CWE-77": 9,  
"CWE-276": 8,
```

```
"CWE-295": 8,
"CWE-668": 7,
"CWE-611": 6,
"CWE-17": 6,
"CWE-121": 6,
"CWE-1021": 5,
"CWE-307": 5,
"CWE-404": 5,
"CWE-502": 4,
"CWE-732": 4,
"CWE-835": 4,
"CWE-330": 4,
"CWE-203": 3,
"CWE-665": 3
},
"severity_counts": {
  "UNKNOWN": 6102,
  "CRITICAL": 170,
  "HIGH": 268,
  "MEDIUM": 267,
  "LOW": 12
}
},
"2014": {
  "year": 2014,
  "count": 8999,
  "monthly_counts": {
    "2014-01": 213,
    "2014-02": 234,
    "2014-03": 338,
    "2014-04": 492,
    "2014-05": 377,
    "2014-06": 349,
    "2014-07": 634,
    "2014-08": 370,
    "2014-09": 1096,
    "2014-10": 1378,
    "2014-11": 473,
    "2014-12": 568,
    "2015-01": 474,
    "2015-02": 149,
    "2015-03": 67,
    "2015-04": 23,
    "2015-05": 34,
    "2015-06": 26,
    "2015-07": 17,
    "2015-08": 25,
    "2015-09": 10,
    "2015-10": 13,
    "2015-11": 5,
    "2015-12": 2,
    "2016-01": 4,
    "2016-02": 2,
    "2016-03": 2,
    "2016-04": 8,
    "2016-05": 9,
    "2016-06": 7,
    "2016-07": 28,
    "2016-08": 42,
    "2016-09": 1,
    "2016-10": 2,
    "2016-12": 2,
    "2017-01": 10,
    "2017-02": 5,
    "2017-03": 72,
    "2017-04": 39,
    "2017-05": 465,
    "2017-06": 40,
    "2017-07": 4,
    "2017-08": 63,
    "2017-09": 26,
    "2017-10": 49,
    "2017-11": 4,
    "2017-12": 11,
    "2018-01": 50,
    "2018-02": 26,
    "2018-03": 42,
```

```
"2018-04": 87,  
"2018-05": 5,  
"2018-06": 3,  
"2018-07": 5,  
"2018-08": 8,  
"2018-10": 2,  
"2018-11": 1,  
"2019-02": 3,  
"2019-03": 7,  
"2019-04": 8,  
"2019-05": 3,  
"2019-06": 1,  
"2019-07": 2,  
"2019-08": 28,  
"2019-09": 2,  
"2019-10": 1,  
"2019-11": 36,  
"2019-12": 40,  
"2020-01": 118,  
"2020-02": 80,  
"2020-03": 4,  
"2020-05": 1,  
"2020-06": 13,  
"2020-07": 1,  
"2020-09": 3,  
"2020-11": 3,  
"2021-08": 1,  
"2022-02": 1,  
"2022-05": 1,  
"2022-06": 26,  
"2022-07": 15,  
"2022-09": 3,  
"2022-12": 3,  
"2023-01": 55,  
"2023-02": 6,  
"2023-03": 4,  
"2023-04": 6,  
"2023-05": 4,  
"2023-06": 3,  
"2023-09": 3,  
"2023-12": 3,  
"2024-04": 2,  
"2024-06": 1,  
"2025-06": 3,  
"2025-07": 12,  
"2025-08": 1,  
"2025-09": 1  
,  
"cwe_counts": {  
  "CWE-310": 1543,  
  "CWE-79": 1043,  
  "CWE-119": 827,  
  "CWE-264": 648,  
  "CWE-20": 525,  
  "CWE-200": 453,  
  "CWE-89": 329,  
  "CWE-352": 247,  
  "CWE-399": 242,  
  "CWE-22": 221,  
  "CWE-94": 164,  
  "CWE-287": 139,  
  "CWE-255": 104,  
  "CWE-189": 103,  
  "CWE-284": 72,  
  "CWE-787": 58,  
  "CWE-362": 57,  
  "CWE-78": 54,  
  "CWE-59": 50,  
  "CWE-416": 39,  
  "CWE-77": 35,  
  "CWE-17": 33,  
  "CWE-19": 33,  
  "CWE-125": 30,  
  "CWE-476": 27,  
  "CWE-400": 25,  
  "CWE-190": 25,  
  "CWE-269": 21,
```

```
"CWE-74": 20,  
"CWE-295": 19,  
"CWE-434": 19,  
"CWE-611": 18,  
"CWE-254": 18,  
"CWE-522": 14,  
"CWE-120": 12,  
"CWE-798": 12,  
"CWE-306": 9,  
"CWE-121": 8,  
"CWE-415": 7,  
"CWE-345": 7,  
"CWE-129": 7,  
"CWE-276": 7,  
"CWE-134": 6,  
"CWE-191": 6,  
"CWE-732": 6,  
"CWE-384": 5,  
"CWE-502": 5,  
"CWE-326": 4,  
"CWE-665": 4,  
"CWE-16": 4  
},  
"severity_counts": {  
  "UNKNOWN": 7760,  
  "CRITICAL": 284,  
  "HIGH": 480,  
  "MEDIUM": 437,  
  "LOW": 38  
}  
},  
"2015": {  
  "year": 2015,  
  "count": 8766,  
  "monthly_counts": {  
    "2015-01": 209,  
    "2015-02": 340,  
    "2015-03": 383,  
    "2015-04": 509,  
    "2015-05": 374,  
    "2015-06": 466,  
    "2015-07": 625,  
    "2015-08": 573,  
    "2015-09": 488,  
    "2015-10": 717,  
    "2015-11": 361,  
    "2015-12": 564,  
    "2016-01": 294,  
    "2016-02": 130,  
    "2016-03": 31,  
    "2016-04": 113,  
    "2016-05": 74,  
    "2016-06": 31,  
    "2016-07": 16,  
    "2016-08": 18,  
    "2016-09": 26,  
    "2016-10": 32,  
    "2016-11": 10,  
    "2016-12": 18,  
    "2017-01": 47,  
    "2017-02": 28,  
    "2017-03": 87,  
    "2017-04": 93,  
    "2017-05": 511,  
    "2017-06": 82,  
    "2017-07": 44,  
    "2017-08": 188,  
    "2017-09": 161,  
    "2017-10": 74,  
    "2017-11": 8,  
    "2017-12": 14,  
    "2018-01": 20,  
    "2018-02": 33,  
    "2018-03": 46,  
    "2018-04": 130,  
    "2018-05": 12,  
    "2018-06": 2,
```



```
"2018-07": 9,
"2018-08": 6,
"2018-09": 2,
"2018-10": 13,
"2018-11": 1,
"2019-01": 7,
"2019-02": 3,
"2019-03": 34,
"2019-04": 14,
"2019-05": 4,
"2019-07": 9,
"2019-08": 91,
"2019-09": 69,
"2019-10": 88,
"2019-11": 23,
"2019-12": 12,
"2020-01": 63,
"2020-02": 48,
"2020-03": 18,
"2020-04": 6,
"2020-05": 1,
"2020-06": 1,
"2020-07": 1,
"2020-08": 3,
"2020-09": 1,
"2020-10": 2,
"2020-11": 7,
"2021-04": 1,
"2021-06": 1,
"2021-07": 3,
"2021-08": 3,
"2021-11": 33,
"2021-12": 2,
"2022-03": 2,
"2022-04": 1,
"2022-07": 34,
"2022-09": 1,
"2022-12": 2,
"2023-01": 66,
"2023-02": 15,
"2023-03": 11,
"2023-04": 7,
"2023-05": 5,
"2023-06": 12,
"2023-07": 4,
"2023-09": 11,
"2023-10": 6,
"2023-12": 3,
"2024-01": 1,
"2024-02": 1,
"2024-03": 3,
"2024-04": 1,
"2024-11": 1,
"2025-04": 2,
"2025-06": 4,
"2025-07": 12
},
"cwe_counts": {
  "CWE-119": 1088,
  "CWE-79": 992,
  "CWE-200": 753,
  "CWE-264": 619,
  "CWE-20": 555,
  "CWE-399": 356,
  "CWE-89": 326,
  "CWE-352": 288,
  "CWE-284": 216,
  "CWE-22": 195,
  "CWE-254": 135,
  "CWE-17": 111,
  "CWE-189": 107,
  "CWE-310": 82,
  "CWE-362": 72,
  "CWE-255": 70,
  "CWE-19": 69,
  "CWE-416": 67,
  "CWE-94": 64,
```

```

"CWE-787": 64,
"CWE-476": 63,
"CWE-287": 58,
"CWE-77": 58,
"CWE-78": 49,
"CWE-190": 47,
"CWE-59": 40,
"CWE-434": 40,
"CWE-125": 33,
"CWE-295": 31,
"CWE-74": 28,
"CWE-120": 26,
"CWE-601": 25,
"CWE-269": 22,
"CWE-611": 20,
"CWE-400": 16,
"CWE-345": 15,
"CWE-415": 12,
"CWE-835": 12,
"CWE-772": 10,
"CWE-798": 10,
"CWE-502": 9,
"CWE-191": 8,
"CWE-134": 8,
"CWE-285": 7,
"CWE-320": 7,
"CWE-426": 6,
"CWE-312": 6,
"CWE-276": 6,
"CWE-326": 6,
"CWE-275": 6
},
"severity_counts": {
"UNKNOWN": 6037,
"HIGH": 988,
"CRITICAL": 535,
"MEDIUM": 1100,
"LOW": 106
},
},
"2016": {
"year": 2016,
"count": 10560,
"monthly_counts": {
"2016-01": 372,
"2016-02": 259,
"2016-03": 299,
"2016-04": 534,
"2016-05": 525,
"2016-06": 472,
"2016-07": 671,
"2016-08": 350,
"2016-09": 570,
"2016-10": 652,
"2016-11": 378,
"2016-12": 505,
"2017-01": 721,
"2017-02": 688,
"2017-03": 358,
"2017-04": 394,
"2017-05": 1064,
"2017-06": 140,
"2017-07": 61,
"2017-08": 97,
"2017-09": 13,
"2017-10": 29,
"2017-11": 10,
"2017-12": 10,
"2018-01": 25,
"2018-02": 61,
"2018-03": 41,
"2018-04": 134,
"2018-05": 83,
"2018-06": 166,
"2018-07": 83,
"2018-08": 47,
"2018-09": 25,

```

```
"2018-10": 13,  
"2018-11": 3,  
"2018-12": 2,  
"2019-01": 13,  
"2019-02": 5,  
"2019-03": 16,  
"2019-04": 9,  
"2019-05": 22,  
"2019-06": 3,  
"2019-07": 7,  
"2019-08": 180,  
"2019-09": 78,  
"2019-10": 13,  
"2019-11": 17,  
"2019-12": 8,  
"2020-01": 20,  
"2020-02": 14,  
"2020-03": 8,  
"2020-04": 37,  
"2020-06": 23,  
"2020-07": 2,  
"2020-08": 4,  
"2020-09": 1,  
"2020-11": 3,  
"2020-12": 6,  
"2021-01": 8,  
"2021-03": 72,  
"2021-04": 40,  
"2021-05": 2,  
"2021-09": 3,  
"2021-12": 2,  
"2022-01": 1,  
"2022-02": 3,  
"2022-04": 1,  
"2022-06": 1,  
"2022-07": 28,  
"2022-08": 1,  
"2022-09": 2,  
"2022-10": 2,  
"2022-12": 2,  
"2023-01": 18,  
"2023-02": 4,  
"2023-03": 3,  
"2023-05": 1,  
"2023-06": 2,  
"2023-07": 1,  
"2023-08": 1,  
"2023-09": 1,  
"2023-10": 1,  
"2023-12": 1,  
"2024-01": 2,  
"2024-04": 1,  
"2024-06": 1,  
"2024-07": 1,  
"2024-10": 3,  
"2024-11": 2,  
"2025-06": 1,  
"2025-07": 4  
},  
"cwe_counts": {  
"CWE-119": 1258,  
"CWE-200": 966,  
"CWE-79": 854,  
"CWE-264": 757,  
"CWE-20": 718,  
"CWE-284": 595,  
"CWE-787": 224,  
"CWE-254": 220,  
"CWE-416": 212,  
"CWE-310": 208,  
"CWE-125": 208,  
"CWE-399": 207,  
"CWE-352": 197,  
"CWE-89": 190,  
"CWE-476": 151,  
"CWE-190": 144,  
"CWE-311": 142,
```

```
"CWE-22": 141,
"CWE-287": 108,
"CWE-255": 89,
"CWE-77": 75,
"CWE-362": 67,
"CWE-19": 67,
"CWE-611": 57,
"CWE-400": 56,
"CWE-502": 47,
"CWE-295": 47,
"CWE-601": 45,
"CWE-94": 44,
"CWE-78": 42,
"CWE-798": 41,
"CWE-189": 36,
"CWE-434": 36,
"CWE-285": 32,
"CWE-275": 27,
"CWE-269": 26,
"CWE-74": 26,
"CWE-120": 23,
"CWE-426": 22,
"CWE-369": 22,
"CWE-532": 19,
"CWE-59": 17,
"CWE-326": 17,
"CWE-918": 16,
"CWE-388": 14,
"CWE-306": 13,
"CWE-863": 13,
"CWE-704": 13,
"CWE-415": 13,
"CWE-320": 13
},
"severity_counts": {
  "CRITICAL": 1365,
  "HIGH": 4025,
  "MEDIUM": 3421,
  "LOW": 265,
  "UNKNOWN": 1484
},
},
"2017": {
  "year": 2017,
  "count": 17023,
  "monthly_counts": {
    "2017-01": 358,
    "2017-02": 352,
    "2017-03": 817,
    "2017-04": 1049,
    "2017-05": 834,
    "2017-06": 777,
    "2017-07": 1182,
    "2017-08": 1189,
    "2017-09": 1025,
    "2017-10": 1246,
    "2017-11": 1052,
    "2017-12": 1078,
    "2018-01": 957,
    "2018-02": 507,
    "2018-03": 1287,
    "2018-04": 341,
    "2018-05": 121,
    "2018-06": 477,
    "2018-07": 210,
    "2018-08": 127,
    "2018-09": 53,
    "2018-10": 36,
    "2018-11": 9,
    "2018-12": 16,
    "2019-01": 66,
    "2019-02": 10,
    "2019-03": 271,
    "2019-04": 28,
    "2019-05": 64,
    "2019-06": 50,
    "2019-07": 36,
```

```
"2019-08": 226,  
"2019-09": 24,  
"2019-10": 58,  
"2019-11": 8,  
"2019-12": 3,  
"2020-01": 4,  
"2020-02": 10,  
"2020-03": 10,  
"2020-04": 215,  
"2020-05": 17,  
"2020-06": 61,  
"2020-07": 3,  
"2020-08": 22,  
"2020-09": 1,  
"2020-10": 2,  
"2020-11": 103,  
"2020-12": 2,  
"2021-01": 1,  
"2021-03": 1,  
"2021-04": 2,  
"2021-05": 4,  
"2021-06": 101,  
"2021-07": 1,  
"2021-08": 5,  
"2021-10": 1,  
"2021-11": 5,  
"2021-12": 13,  
"2022-02": 1,  
"2022-03": 6,  
"2022-06": 109,  
"2022-07": 20,  
"2022-08": 10,  
"2022-09": 2,  
"2022-10": 2,  
"2022-12": 13,  
"2023-01": 97,  
"2023-02": 94,  
"2023-03": 4,  
"2023-05": 3,  
"2023-06": 1,  
"2023-08": 1,  
"2023-09": 84,  
"2023-11": 2,  
"2024-01": 2,  
"2024-03": 2,  
"2024-07": 4,  
"2024-10": 4,  
"2024-11": 20,  
"2024-12": 1,  
"2025-01": 4,  
"2025-03": 5,  
"2025-04": 2,  
"2025-07": 1,  
"2025-08": 1  
},  
"cwe_counts": {  
"CWE-119": 2054,  
"CWE-79": 1634,  
"CWE-200": 1274,  
"CWE-20": 1171,  
"CWE-125": 726,  
"CWE-89": 592,  
"CWE-22": 520,  
"CWE-787": 382,  
"CWE-416": 355,  
"CWE-476": 319,  
"CWE-352": 317,  
"CWE-287": 298,  
"CWE-190": 232,  
"CWE-772": 204,  
"CWE-78": 202,  
"CWE-400": 179,  
"CWE-295": 178,  
"CWE-269": 174,  
"CWE-732": 134,  
"CWE-426": 132,  
"CWE-362": 121,
```

```
"CWE-835": 120,
"CWE-74": 120,
"CWE-798": 116,
"CWE-121": 108,
"CWE-94": 107,
"CWE-502": 105,
"CWE-434": 103,
"CWE-601": 98,
"CWE-611": 98,
"CWE-120": 84,
"CWE-77": 84,
"CWE-522": 79,
"CWE-863": 71,
"CWE-918": 65,
"CWE-427": 64,
"CWE-862": 63,
"CWE-770": 54,
"CWE-617": 51,
"CWE-415": 50,
"CWE-326": 49,
"CWE-306": 46,
"CWE-399": 44,
"CWE-369": 44,
"CWE-276": 44,
"CWE-506": 44,
"CWE-284": 41,
"CWE-532": 39,
"CWE-319": 38,
"CWE-264": 37
},
"severity_counts": {
  "CRITICAL": 2095,
  "MEDIUM": 5629,
  "HIGH": 6643,
  "LOW": 313,
  "UNKNOWN": 2343
}
},
"2018": {
  "year": 2018,
  "count": 17495,
  "monthly_counts": {
    "2018-01": 650,
    "2018-02": 686,
    "2018-03": 886,
    "2018-04": 983,
    "2018-05": 953,
    "2018-06": 1148,
    "2018-07": 1887,
    "2018-08": 850,
    "2018-09": 1109,
    "2018-10": 1408,
    "2018-11": 973,
    "2018-12": 1184,
    "2019-01": 1084,
    "2019-02": 289,
    "2019-03": 493,
    "2019-04": 446,
    "2019-05": 231,
    "2019-06": 257,
    "2019-07": 133,
    "2019-08": 189,
    "2019-09": 28,
    "2019-10": 68,
    "2019-11": 35,
    "2019-12": 37,
    "2020-01": 27,
    "2020-02": 12,
    "2020-03": 31,
    "2020-04": 202,
    "2020-05": 10,
    "2020-06": 37,
    "2020-07": 2,
    "2020-08": 41,
    "2020-09": 25,
    "2020-10": 24,
    "2020-11": 64,
```

```
"2020-12": 18,  
"2021-01": 45,  
"2021-02": 1,  
"2021-03": 4,  
"2021-04": 3,  
"2021-05": 17,  
"2021-06": 5,  
"2021-07": 11,  
"2021-08": 4,  
"2021-09": 2,  
"2021-10": 4,  
"2021-11": 94,  
"2021-12": 25,  
"2022-02": 1,  
"2022-03": 3,  
"2022-05": 1,  
"2022-06": 13,  
"2022-07": 1,  
"2022-08": 5,  
"2022-09": 1,  
"2022-10": 2,  
"2022-12": 15,  
"2023-01": 19,  
"2023-02": 122,  
"2023-03": 38,  
"2023-04": 12,  
"2023-05": 18,  
"2023-06": 2,  
"2023-07": 1,  
"2023-08": 1,  
"2023-09": 361,  
"2023-10": 7,  
"2023-11": 3,  
"2023-12": 3,  
"2024-01": 2,  
"2024-02": 1,  
"2024-03": 3,  
"2024-04": 1,  
"2024-06": 1,  
"2024-09": 1,  
"2024-10": 2,  
"2024-11": 64,  
"2024-12": 38,  
"2025-01": 18,  
"2025-03": 11,  
"2025-04": 2,  
"2025-05": 3,  
"2025-06": 1,  
"2025-07": 2,  
"2025-08": 1  
},  
"cwe_counts": {  
"CWE-79": 2289,  
"CWE-20": 1203,  
"CWE-787": 1062,  
"CWE-200": 938,  
"CWE-125": 929,  
"CWE-119": 862,  
"CWE-190": 678,  
"CWE-416": 653,  
"CWE-89": 537,  
"CWE-22": 500,  
"CWE-352": 493,  
"CWE-78": 422,  
"CWE-287": 331,  
"CWE-476": 294,  
"CWE-732": 276,  
"CWE-400": 233,  
"CWE-611": 196,  
"CWE-434": 173,  
"CWE-94": 161,  
"CWE-798": 154,  
"CWE-269": 152,  
"CWE-502": 124,  
"CWE-522": 120,  
"CWE-704": 114,  
"CWE-863": 108,
```

```
"CWE-295": 104,  
"CWE-362": 102,  
"CWE-835": 100,  
"CWE-426": 89,  
"CWE-772": 87,  
"CWE-601": 87,  
"CWE-918": 87,  
"CWE-862": 85,  
"CWE-120": 81,  
"CWE-284": 79,  
"CWE-77": 77,  
"CWE-532": 77,  
"CWE-415": 70,  
"CWE-843": 62,  
"CWE-306": 60,  
"CWE-384": 59,  
"CWE-121": 54,  
"CWE-770": 52,  
"CWE-319": 50,  
"CWE-404": 50,  
"CWE-74": 49,  
"CWE-347": 47,  
"CWE-399": 46,  
"CWE-326": 43,  
"CWE-59": 43  
},  
"severity_counts": {  
  "CRITICAL": 2208,  
  "HIGH": 7048,  
  "MEDIUM": 6379,  
  "UNKNOWN": 1628,  
  "LOW": 232  
},  
},  
"2019": {  
  "year": 2019,  
  "count": 17034,  
  "monthly_counts": {  
    "2019-01": 474,  
    "2019-02": 566,  
    "2019-03": 801,  
    "2019-04": 1062,  
    "2019-05": 998,  
    "2019-06": 947,  
    "2019-07": 1452,  
    "2019-08": 1314,  
    "2019-09": 1339,  
    "2019-10": 1439,  
    "2019-11": 1320,  
    "2019-12": 1661,  
    "2020-01": 764,  
    "2020-02": 343,  
    "2020-03": 595,  
    "2020-04": 349,  
    "2020-05": 83,  
    "2020-06": 214,  
    "2020-07": 87,  
    "2020-08": 63,  
    "2020-09": 97,  
    "2020-10": 151,  
    "2020-11": 34,  
    "2020-12": 49,  
    "2021-01": 42,  
    "2021-02": 25,  
    "2021-03": 37,  
    "2021-04": 22,  
    "2021-05": 10,  
    "2021-06": 15,  
    "2021-07": 8,  
    "2021-08": 2,  
    "2021-09": 7,  
    "2021-10": 2,  
    "2021-11": 9,  
    "2021-12": 130,  
    "2022-01": 1,  
    "2022-02": 6,  
    "2022-03": 2,
```



```
"2022-04": 3,
"2022-05": 3,
"2022-06": 24,
"2022-07": 10,
"2022-08": 3,
"2022-09": 102,
"2022-10": 2,
"2022-11": 1,
"2022-12": 93,
"2023-01": 13,
"2023-02": 96,
"2023-03": 2,
"2023-04": 2,
"2023-05": 22,
"2023-06": 17,
"2023-08": 2,
"2023-09": 51,
"2023-11": 2,
"2023-12": 2,
"2024-02": 4,
"2024-03": 2,
"2024-04": 5,
"2024-06": 3,
"2024-07": 14,
"2024-09": 1,
"2024-10": 7,
"2024-11": 10,
"2024-12": 2,
"2025-01": 2,
"2025-02": 2,
"2025-03": 7,
"2025-04": 1,
"2025-05": 1,
"2025-07": 1,
"2025-08": 2
},
"cwe_counts": {
"CWE-79": 2000,
"CWE-787": 1451,
"CWE-125": 885,
"CWE-20": 687,
"CWE-416": 535,
"CWE-89": 491,
"CWE-78": 485,
"CWE-22": 476,
"CWE-352": 463,
"CWE-200": 355,
"CWE-119": 297,
"CWE-476": 279,
"CWE-287": 266,
"CWE-306": 245,
"CWE-190": 244,
"CWE-862": 235,
"CWE-522": 221,
"CWE-120": 202,
"CWE-434": 192,
"CWE-732": 182,
"CWE-400": 174,
"CWE-269": 159,
"CWE-77": 154,
"CWE-798": 154,
"CWE-502": 147,
"CWE-94": 141,
"CWE-401": 135,
"CWE-611": 132,
"CWE-532": 129,
"CWE-284": 127,
"CWE-319": 122,
"CWE-59": 120,
"CWE-601": 116,
"CWE-295": 114,
"CWE-863": 109,
"CWE-770": 107,
"CWE-276": 105,
"CWE-362": 96,
"CWE-427": 96,
"CWE-918": 89,
```

```
"CWE-755": 87,  
"CWE-843": 85,  
"CWE-74": 78,  
"CWE-908": 76,  
"CWE-327": 75,  
"CWE-311": 74,  
"CWE-264": 72,  
"CWE-312": 71,  
"CWE-122": 65,  
"CWE-426": 64  
},  
"severity_counts": {  
  "HIGH": 6524,  
  "MEDIUM": 6333,  
  "CRITICAL": 2374,  
  "LOW": 327,  
  "UNKNOWN": 1476  
}  
},  
"2020": {  
  "year": 2020,  
  "count": 20607,  
  "monthly_counts": {  
    "2020-01": 621,  
    "2020-02": 757,  
    "2020-03": 1112,  
    "2020-04": 1399,  
    "2020-05": 944,  
    "2020-06": 1504,  
    "2020-07": 1335,  
    "2020-08": 1106,  
    "2020-09": 1502,  
    "2020-10": 1394,  
    "2020-11": 1215,  
    "2020-12": 1488,  
    "2021-01": 1110,  
    "2021-02": 561,  
    "2021-03": 333,  
    "2021-04": 359,  
    "2021-05": 411,  
    "2021-06": 405,  
    "2021-07": 292,  
    "2021-08": 263,  
    "2021-09": 197,  
    "2021-10": 135,  
    "2021-11": 127,  
    "2021-12": 387,  
    "2022-01": 173,  
    "2022-02": 47,  
    "2022-03": 38,  
    "2022-04": 76,  
    "2022-05": 24,  
    "2022-06": 41,  
    "2022-07": 63,  
    "2022-08": 75,  
    "2022-09": 64,  
    "2022-10": 43,  
    "2022-11": 24,  
    "2022-12": 232,  
    "2023-01": 56,  
    "2023-02": 14,  
    "2023-03": 109,  
    "2023-04": 45,  
    "2023-05": 51,  
    "2023-06": 82,  
    "2023-07": 47,  
    "2023-08": 109,  
    "2023-09": 15,  
    "2023-10": 20,  
    "2023-11": 3,  
    "2023-12": 13,  
    "2024-01": 13,  
    "2024-02": 18,  
    "2024-03": 6,  
    "2024-04": 7,  
    "2024-05": 6,  
    "2024-06": 15,
```

```

"2024-07": 5,
"2024-08": 3,
"2024-09": 2,
"2024-10": 21,
"2024-11": 31,
"2024-12": 27,
"2025-01": 5,
"2025-02": 8,
"2025-03": 6,
"2025-04": 4,
"2025-05": 3,
"2025-07": 5,
"2025-08": 1
},
"cwe_counts": {
"CWE-79": 2650,
"CWE-787": 1596,
"CWE-125": 929,
"CWE-20": 922,
"CWE-89": 799,
"CWE-78": 592,
"CWE-22": 586,
"CWE-352": 531,
"CWE-416": 459,
"CWE-120": 407,
"CWE-287": 343,
"CWE-434": 331,
"CWE-862": 309,
"CWE-306": 297,
"CWE-400": 278,
"CWE-269": 271,
"CWE-200": 270,
"CWE-276": 265,
"CWE-476": 262,
"CWE-190": 241,
"CWE-863": 235,
"CWE-502": 232,
"CWE-798": 217,
"CWE-119": 216,
"CWE-77": 202,
"CWE-522": 181,
"CWE-362": 178,
"CWE-918": 170,
"CWE-732": 168,
"CWE-295": 159,
"CWE-427": 159,
"CWE-74": 148,
"CWE-319": 147,
"CWE-129": 146,
"CWE-611": 144,
"CWE-601": 142,
"CWE-94": 140,
"CWE-284": 126,
"CWE-59": 124,
"CWE-532": 112,
"CWE-401": 109,
"CWE-312": 106,
"CWE-327": 105,
"CWE-347": 92,
"CWE-668": 83,
"CWE-122": 82,
"CWE-121": 80,
"CWE-843": 80,
"CWE-770": 78,
"CWE-203": 78
},
"severity_counts": {
"HIGH": 7880,
"CRITICAL": 2660,
"MEDIUM": 7918,
"LOW": 517,
"UNKNOWN": 1632
}
},
"2021": {
"year": 2021,
"count": 23076,

```

```
"monthly_counts": {
  "2021-01": 742,
  "2021-02": 905,
  "2021-03": 1138,
  "2021-04": 1501,
  "2021-05": 1053,
  "2021-06": 1355,
  "2021-07": 1286,
  "2021-08": 1962,
  "2021-09": 1711,
  "2021-10": 1566,
  "2021-11": 1375,
  "2021-12": 1845,
  "2022-01": 1292,
  "2022-02": 871,
  "2022-03": 707,
  "2022-04": 427,
  "2022-05": 321,
  "2022-06": 309,
  "2022-07": 139,
  "2022-08": 204,
  "2022-09": 92,
  "2022-10": 70,
  "2022-11": 63,
  "2022-12": 262,
  "2023-01": 116,
  "2023-02": 142,
  "2023-03": 67,
  "2023-04": 61,
  "2023-05": 123,
  "2023-06": 57,
  "2023-07": 99,
  "2023-08": 42,
  "2023-09": 38,
  "2023-10": 22,
  "2023-11": 24,
  "2023-12": 10,
  "2024-01": 28,
  "2024-02": 239,
  "2024-03": 117,
  "2024-04": 42,
  "2024-05": 398,
  "2024-06": 54,
  "2024-07": 3,
  "2024-08": 13,
  "2024-09": 12,
  "2024-10": 11,
  "2024-11": 51,
  "2024-12": 18,
  "2025-01": 11,
  "2025-02": 36,
  "2025-03": 25,
  "2025-04": 11,
  "2025-05": 4,
  "2025-06": 3,
  "2025-07": 2,
  "2025-08": 1
},
"cwe_counts": {
  "CWE-79": 3414,
  "CWE-787": 1864,
  "CWE-20": 1021,
  "CWE-125": 989,
  "CWE-89": 961,
  "CWE-416": 946,
  "CWE-22": 693,
  "CWE-476": 635,
  "CWE-78": 613,
  "CWE-352": 595,
  "CWE-120": 531,
  "CWE-200": 424,
  "CWE-287": 403,
  "CWE-434": 397,
  "CWE-862": 392,
  "CWE-119": 391,
  "CWE-269": 385,
  "CWE-77": 348,
```

```
"CWE-863": 339,
"CWE-190": 336,
"CWE-502": 292,
"CWE-400": 289,
"CWE-401": 237,
"CWE-918": 223,
"CWE-121": 223,
"CWE-94": 221,
"CWE-276": 211,
"CWE-284": 200,
"CWE-798": 191,
"CWE-732": 185,
"CWE-362": 184,
"CWE-306": 181,
"CWE-74": 180,
"CWE-601": 176,
"CWE-427": 154,
"CWE-770": 153,
"CWE-522": 152,
"CWE-295": 152,
"CWE-611": 141,
"CWE-122": 140,
"CWE-668": 137,
"CWE-59": 135,
"CWE-532": 132,
"CWE-319": 119,
"CWE-203": 117,
"CWE-415": 116,
"CWE-312": 115,
"CWE-617": 115,
"CWE-639": 113,
"CWE-1321": 113
},
"severity_counts": {
  "MEDIUM": 9457,
  "HIGH": 9298,
  "CRITICAL": 2659,
  "UNKNOWN": 912,
  "LOW": 750
}
},
"2022": {
  "year": 2022,
  "count": 26722,
  "monthly_counts": {
    "2022-01": 740,
    "2022-02": 1084,
    "2022-03": 1328,
    "2022-04": 1553,
    "2022-05": 1681,
    "2022-06": 1780,
    "2022-07": 1845,
    "2022-08": 2111,
    "2022-09": 2052,
    "2022-10": 1786,
    "2022-11": 1899,
    "2022-12": 2277,
    "2023-01": 1354,
    "2023-02": 748,
    "2023-03": 664,
    "2023-04": 273,
    "2023-05": 526,
    "2023-06": 154,
    "2023-07": 133,
    "2023-08": 163,
    "2023-09": 84,
    "2023-10": 90,
    "2023-11": 91,
    "2023-12": 44,
    "2024-01": 81,
    "2024-02": 38,
    "2024-03": 36,
    "2024-04": 67,
    "2024-05": 70,
    "2024-06": 91,
    "2024-07": 118,
    "2024-08": 109,
```

```

"2024-09": 25,
"2024-10": 103,
"2024-11": 45,
"2024-12": 44,
"2025-01": 24,
"2025-02": 705,
"2025-03": 44,
"2025-04": 14,
"2025-05": 334,
"2025-06": 300,
"2025-07": 3,
"2025-08": 5,
"2025-09": 6
},
"cwe_counts": {
"CWE-79": 4144,
"CWE-787": 2923,
"CWE-89": 2139,
"CWE-125": 1147,
"CWE-416": 1051,
"CWE-22": 968,
"CWE-352": 932,
"CWE-78": 882,
"CWE-20": 852,
"CWE-862": 842,
"CWE-120": 774,
"CWE-434": 602,
"CWE-476": 596,
"CWE-200": 548,
"CWE-287": 517,
"CWE-284": 374,
"CWE-119": 374,
"CWE-863": 366,
"CWE-400": 361,
"CWE-190": 345,
"CWE-269": 337,
"CWE-362": 334,
"CWE-918": 308,
"CWE-306": 302,
"CWE-94": 276,
"CWE-502": 275,
"CWE-401": 267,
"CWE-276": 251,
"CWE-798": 249,
"CWE-77": 246,
"CWE-770": 241,
"CWE-427": 226,
"CWE-522": 210,
"CWE-601": 206,
"CWE-732": 192,
"CWE-367": 183,
"CWE-532": 183,
"CWE-122": 181,
"CWE-611": 178,
"CWE-295": 174,
"CWE-639": 170,
"CWE-707": 164,
"CWE-121": 161,
"CWE-617": 149,
"CWE-668": 147,
"CWE-74": 134,
"CWE-203": 128,
"CWE-755": 126,
"CWE-312": 120,
"CWE-59": 118
},
"severity_counts": {
"MEDIUM": 11121,
"CRITICAL": 3562,
"HIGH": 9359,
"LOW": 851,
"UNKNOWN": 1827,
"NONE": 2
}
},
"2023": {
"year": 2023,

```

```
"count": 29853,
"monthly_counts": {
  "2023-01": 742,
  "2023-02": 1224,
  "2023-03": 1874,
  "2023-04": 1945,
  "2023-05": 1979,
  "2023-06": 2070,
  "2023-07": 2078,
  "2023-08": 2199,
  "2023-09": 2018,
  "2023-10": 2572,
  "2023-11": 2381,
  "2023-12": 2604,
  "2024-01": 1374,
  "2024-02": 846,
  "2024-03": 543,
  "2024-04": 369,
  "2024-05": 1243,
  "2024-06": 254,
  "2024-07": 135,
  "2024-08": 112,
  "2024-09": 79,
  "2024-10": 64,
  "2024-11": 86,
  "2024-12": 293,
  "2025-01": 200,
  "2025-02": 120,
  "2025-03": 116,
  "2025-04": 43,
  "2025-05": 159,
  "2025-06": 41,
  "2025-07": 29,
  "2025-08": 42,
  "2025-09": 19
},
"cwe_counts": {
  "CWE-79": 5728,
  "CWE-787": 2747,
  "CWE-89": 2316,
  "CWE-352": 1323,
  "CWE-862": 1307,
  "CWE-125": 1296,
  "CWE-416": 1079,
  "CWE-22": 1053,
  "CWE-20": 929,
  "CWE-78": 871,
  "CWE-120": 730,
  "CWE-77": 716,
  "CWE-434": 692,
  "CWE-200": 582,
  "CWE-94": 552,
  "CWE-476": 531,
  "CWE-284": 522,
  "CWE-863": 507,
  "CWE-400": 502,
  "CWE-287": 492,
  "CWE-121": 446,
  "CWE-269": 430,
  "CWE-190": 358,
  "CWE-502": 345,
  "CWE-918": 325,
  "CWE-119": 301,
  "CWE-122": 298,
  "CWE-306": 290,
  "CWE-770": 270,
  "CWE-798": 252,
  "CWE-276": 246,
  "CWE-639": 230,
  "CWE-362": 221,
  "CWE-532": 210,
  "CWE-732": 210,
  "CWE-601": 209,
  "CWE-427": 206,
  "CWE-611": 200,
  "CWE-74": 199,
  "CWE-668": 189,
```

```
"CWE-522": 169,
"CWE-295": 160,
"CWE-319": 159,
"CWE-203": 156,
"CWE-59": 154,
"CWE-312": 151,
"CWE-843": 150,
"CWE-401": 147,
"CWE-285": 126,
"CWE-307": 125
},
"severity_counts": {
  "MEDIUM": 14268,
  "HIGH": 10060,
  "UNKNOWN": 803,
  "CRITICAL": 3482,
  "LOW": 1236,
  "NONE": 4
}
},
"2024": {
  "year": 2024,
  "count": 38629,
  "monthly_counts": {
    "2023-11": 2,
    "2024-01": 1142,
    "2024-02": 1759,
    "2024-03": 2640,
    "2024-04": 3239,
    "2024-05": 3386,
    "2024-06": 2752,
    "2024-07": 2894,
    "2024-08": 2708,
    "2024-09": 2420,
    "2024-10": 3378,
    "2024-11": 3776,
    "2024-12": 3025,
    "2025-01": 2283,
    "2025-02": 1148,
    "2025-03": 967,
    "2025-04": 269,
    "2025-05": 437,
    "2025-06": 134,
    "2025-07": 143,
    "2025-08": 103,
    "2025-09": 24
  },
  "cwe_counts": {
    "CWE-79": 8789,
    "CWE-89": 3308,
    "CWE-787": 1780,
    "CWE-352": 1655,
    "CWE-862": 1536,
    "CWE-416": 1248,
    "CWE-22": 1194,
    "CWE-125": 1151,
    "CWE-476": 924,
    "CWE-94": 912,
    "CWE-434": 866,
    "CWE-78": 856,
    "CWE-200": 808,
    "CWE-120": 750,
    "CWE-284": 720,
    "CWE-121": 707,
    "CWE-20": 671,
    "CWE-77": 643,
    "CWE-863": 524,
    "CWE-502": 502,
    "CWE-918": 486,
    "CWE-122": 480,
    "CWE-400": 459,
    "CWE-269": 405,
    "CWE-74": 348,
    "CWE-287": 335,
    "CWE-639": 330,
    "CWE-770": 326,
    "CWE-276": 314,
```



```
"CWE-190": 295,
"CWE-306": 270,
"CWE-601": 243,
"CWE-362": 242,
"CWE-119": 238,
"CWE-798": 225,
"CWE-427": 225,
"CWE-532": 220,
"CWE-401": 213,
"CWE-843": 169,
"CWE-285": 166,
"CWE-732": 165,
"CWE-667": 162,
"CWE-922": 159,
"CWE-59": 155,
"CWE-908": 152,
"CWE-295": 148,
"CWE-203": 134,
"CWE-266": 132,
"CWE-754": 130,
"CWE-522": 129
},
"severity_counts": {
  "UNKNOWN": 1454,
  "MEDIUM": 19823,
  "LOW": 1281,
  "HIGH": 12377,
  "CRITICAL": 3684,
  "NONE": 10
}
},
"2025": {
  "year": 2025,
  "count": 24612,
  "monthly_counts": {
    "2025-01": 1867,
    "2025-02": 1812,
    "2025-03": 2985,
    "2025-04": 3793,
    "2025-05": 3322,
    "2025-06": 3307,
    "2025-07": 3702,
    "2025-08": 3454,
    "2025-09": 370
  },
  "cwe_counts": {
    "CWE-79": 4793,
    "CWE-89": 3339,
    "CWE-74": 1764,
    "CWE-352": 1191,
    "CWE-862": 1112,
    "CWE-119": 855,
    "CWE-94": 842,
    "CWE-284": 683,
    "CWE-120": 669,
    "CWE-787": 612,
    "CWE-22": 600,
    "CWE-125": 516,
    "CWE-434": 510,
    "CWE-121": 480,
    "CWE-78": 472,
    "CWE-77": 460,
    "CWE-416": 429,
    "CWE-200": 409,
    "CWE-502": 384,
    "CWE-20": 378,
    "CWE-122": 362,
    "CWE-476": 309,
    "CWE-98": 301,
    "CWE-863": 297,
    "CWE-918": 285,
    "CWE-400": 249,
    "CWE-266": 233,
    "CWE-306": 215,
    "CWE-287": 204,
    "CWE-770": 196,
    "CWE-269": 180,
```

```
"CWE-639": 160,
"CWE-285": 151,
"CWE-601": 136,
"CWE-190": 121,
"CWE-404": 112,
"CWE-276": 109,
"CWE-798": 107,
"CWE-532": 106,
"CWE-732": 102,
"CWE-288": 100,
"CWE-362": 86,
"CWE-427": 82,
"CWE-497": 77,
"CWE-59": 77,
"CWE-295": 76,
"CWE-401": 70,
"CWE-611": 70,
"CWE-312": 69,
"CWE-73": 67
},
"severity_counts": {
  "MEDIUM": 12300,
  "CRITICAL": 2046,
  "HIGH": 7675,
  "LOW": 762,
  "UNKNOWN": 1822,
  "NONE": 7
}
},
"monthly_timeline": {
  "2010-01": 155,
  "2010-02": 245,
  "2010-03": 396,
  "2010-04": 404,
  "2010-05": 374,
  "2010-06": 449,
  "2010-07": 288,
  "2010-08": 356,
  "2010-09": 287,
  "2010-10": 417,
  "2010-11": 274,
  "2010-12": 348,
  "2011-01": 140,
  "2011-02": 283,
  "2011-03": 296,
  "2011-04": 265,
  "2011-05": 279,
  "2011-06": 282,
  "2011-07": 293,
  "2011-08": 274,
  "2011-09": 350,
  "2011-10": 358,
  "2011-11": 215,
  "2011-12": 331,
  "2012-01": 145,
  "2012-02": 239,
  "2012-03": 338,
  "2012-04": 186,
  "2012-05": 337,
  "2012-06": 339,
  "2012-07": 466,
  "2012-08": 566,
  "2012-09": 501,
  "2012-10": 495,
  "2012-11": 415,
  "2012-12": 261,
  "2013-01": 202,
  "2013-02": 298,
  "2013-03": 339,
  "2013-04": 402,
  "2013-05": 330,
  "2013-06": 331,
  "2013-07": 458,
  "2013-08": 343,
  "2013-09": 451,
  "2013-10": 544,
```

"2013-11": 391,
"2013-12": 459,
"2014-01": 213,
"2014-02": 234,
"2014-03": 338,
"2014-04": 492,
"2014-05": 377,
"2014-06": 349,
"2014-07": 634,
"2014-08": 370,
"2014-09": 1096,
"2014-10": 1378,
"2014-11": 473,
"2014-12": 568,
"2015-01": 209,
"2015-02": 340,
"2015-03": 383,
"2015-04": 509,
"2015-05": 374,
"2015-06": 466,
"2015-07": 625,
"2015-08": 573,
"2015-09": 488,
"2015-10": 717,
"2015-11": 361,
"2015-12": 564,
"2016-01": 372,
"2016-02": 259,
"2016-03": 299,
"2016-04": 534,
"2016-05": 525,
"2016-06": 472,
"2016-07": 671,
"2016-08": 350,
"2016-09": 570,
"2016-10": 652,
"2016-11": 378,
"2016-12": 505,
"2017-01": 358,
"2017-02": 352,
"2017-03": 817,
"2017-04": 1049,
"2017-05": 834,
"2017-06": 777,
"2017-07": 1182,
"2017-08": 1189,
"2017-09": 1025,
"2017-10": 1246,
"2017-11": 1052,
"2017-12": 1078,
"2018-01": 650,
"2018-02": 686,
"2018-03": 886,
"2018-04": 983,
"2018-05": 953,
"2018-06": 1148,
"2018-07": 1887,
"2018-08": 850,
"2018-09": 1109,
"2018-10": 1408,
"2018-11": 973,
"2018-12": 1184,
"2019-01": 474,
"2019-02": 566,
"2019-03": 801,
"2019-04": 1062,
"2019-05": 998,
"2019-06": 947,
"2019-07": 1452,
"2019-08": 1314,
"2019-09": 1339,
"2019-10": 1439,
"2019-11": 1320,
"2019-12": 1661,
"2020-01": 621,
"2020-02": 757,
"2020-03": 1112,

```
"2020-04": 1399,  
"2020-05": 944,  
"2020-06": 1504,  
"2020-07": 1335,  
"2020-08": 1106,  
"2020-09": 1502,  
"2020-10": 1394,  
"2020-11": 1215,  
"2020-12": 1488,  
"2021-01": 742,  
"2021-02": 905,  
"2021-03": 1138,  
"2021-04": 1501,  
"2021-05": 1053,  
"2021-06": 1355,  
"2021-07": 1286,  
"2021-08": 1962,  
"2021-09": 1711,  
"2021-10": 1566,  
"2021-11": 1375,  
"2021-12": 1845,  
"2022-01": 740,  
"2022-02": 1084,  
"2022-03": 1328,  
"2022-04": 1553,  
"2022-05": 1681,  
"2022-06": 1780,  
"2022-07": 1845,  
"2022-08": 2111,  
"2022-09": 2052,  
"2022-10": 1786,  
"2022-11": 1899,  
"2022-12": 2277,  
"2023-01": 742,  
"2023-02": 1224,  
"2023-03": 1874,  
"2023-04": 1945,  
"2023-05": 1979,  
"2023-06": 2070,  
"2023-07": 2078,  
"2023-08": 2199,  
"2023-09": 2018,  
"2023-10": 2572,  
"2023-11": 2,  
"2023-12": 2604,  
"2024-01": 1142,  
"2024-02": 1759,  
"2024-03": 2640,  
"2024-04": 3239,  
"2024-05": 3386,  
"2024-06": 2752,  
"2024-07": 2894,  
"2024-08": 2708,  
"2024-09": 2420,  
"2024-10": 3378,  
"2024-11": 3776,  
"2024-12": 3025,  
"2025-01": 1867,  
"2025-02": 1812,  
"2025-03": 2985,  
"2025-04": 3793,  
"2025-05": 3322,  
"2025-06": 3307,  
"2025-07": 3702,  
"2025-08": 3454,  
"2025-09": 370  
},  
"top_cwes": {  
"CWE-79": 41119,  
"CWE-89": 16525,  
"CWE-787": 14906,  
"CWE-119": 11337,  
"CWE-20": 11317,  
"CWE-125": 8891,  
"CWE-200": 8850,  
"CWE-352": 8758,  
"CWE-22": 7813,
```

"CWE-416": 7292,
"CWE-862": 5884,
"CWE-78": 5673,
"CWE-476": 4461,
"CWE-120": 4349,
"CWE-94": 4287,
"CWE-434": 4018,
"CWE-264": 3965,
"CWE-287": 3906,
"CWE-284": 3786,
"CWE-190": 3152,
"CWE-74": 3129,
"CWE-77": 3115,
"CWE-400": 2883,
"CWE-269": 2609,
"CWE-863": 2598,
"CWE-502": 2476,
"CWE-121": 2311,
"CWE-310": 2231,
"CWE-918": 2054,
"CWE-362": 1963,
"CWE-306": 1936,
"CWE-399": 1872,
"CWE-798": 1750,
"CWE-732": 1637,
"CWE-122": 1608,
"CWE-276": 1577,
"CWE-601": 1489,
"CWE-770": 1477,
"CWE-295": 1402,
"CWE-522": 1292,
"CWE-611": 1269,
"CWE-532": 1227,
"CWE-427": 1212,
"CWE-401": 1180,
"CWE-59": 1168,
"CWE-639": 1003,
"CWE-189": 747,
"CWE-312": 643,
"CWE-319": 635,
"CWE-203": 616,
"CWE-668": 565,
"CWE-843": 556,
"CWE-255": 521,
"CWE-285": 482,
"CWE-254": 376,
"CWE-266": 365,
"CWE-426": 323,
"CWE-617": 315,
"CWE-772": 309,
"CWE-98": 301,
"CWE-415": 276,
"CWE-835": 247,
"CWE-908": 246,
"CWE-311": 218,
"CWE-755": 216,
"CWE-367": 183,
"CWE-327": 180,
"CWE-19": 171,
"CWE-404": 170,
"CWE-707": 164,
"CWE-667": 162,
"CWE-922": 159,
"CWE-129": 153,
"CWE-17": 150,
"CWE-347": 139,
"CWE-704": 138,
"CWE-326": 133,
"CWE-307": 130,
"CWE-754": 130,
"CWE-1321": 113,
"CWE-288": 100,
"CWE-16": 90,
"CWE-497": 77,
"CWE-73": 67,
"CWE-369": 66,
"CWE-134": 64,

```
"CWE-384": 64,  
"CWE-506": 44,  
"CWE-275": 33,  
"CWE-345": 22,  
"CWE-191": 20,  
"CWE-320": 20,  
"CWE-388": 14,  
"CWE-909": 7,  
"CWE-193": 7,  
"CWE-665": 7,  
"CWE-346": 5,  
"CWE-1021": 5,  
"CWE-330": 4,  
"CWE-749": 2  
},  
"severity_distribution": {  
  "UNKNOWN": 50263,  
  "HIGH": 83076,  
  "MEDIUM": 98839,  
  "CRITICAL": 27346,  
  "LOW": 6715,  
  "NONE": 23  
},  
"total_cves": 266262,  
"processed_at": "2025-09-04T08:36:42.046719"  
}
```

database\db_manager.py

```
import os
import psycopg2
from psycopg2.extras import RealDictCursor, execute_batch
from psycopg2 import pool
from typing import List, Dict, Any, Optional
import json
from datetime import datetime

class DatabaseManager:
    """Manages PostgreSQL database connections with connection pooling - MEMORY OPTIMIZED"""

    def __init__(self):
        # Get database URL from environment variable
        self.database_url = os.environ.get('DATABASE_URL')
        self.connection_pool = None
        self.use_database = bool(self.database_url)

        if self.use_database:
            print("[Database] Database URL found, database mode ENABLED")
            self.init_connection_pool()
            self.init_tables()
        else:
            print("[Database] No DATABASE_URL found, database mode DISABLED (using memory)")

    def init_connection_pool(self):
        """Initialize connection pool with MINIMAL connections"""
        if not self.database_url:
            return False

        try:
            # Fix URL format
            database_url = self.database_url.replace('postgres://', 'postgresql://', 1)

            # Create connection pool with MINIMAL connections to save memory
            self.connection_pool = pool.SimpleConnectionPool(
                1, # Min connections
                2, # Max connections (REDUCED from 3 to 2)
                database_url
            )

            print("[Database] Connection pool initialized (1-2 connections)")
            return True
        except Exception as e:
            print(f"[Database] Connection pool error: {e}")
            self.use_database = False
            return False

    def get_connection(self):
        """Get connection from pool"""
        if self.connection_pool:
            return self.connection_pool.getconn()
        return None

    def return_connection(self, conn):
        """Return connection to pool"""
        if self.connection_pool and conn:
            self.connection_pool.putconn(conn)

    def init_tables(self):
        """Initialize database tables if they don't exist"""
        conn = self.get_connection()
        if not conn:
            return

        try:
            cursor = conn.cursor()

            # CVE table
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS cves (
            id VARCHAR(50) PRIMARY KEY,
            description TEXT,
```

```

severity VARCHAR(20),
cvss_score FLOAT,
cwe VARCHAR(50),
published VARCHAR(50),
last_modified VARCHAR(50),
reference_links JSONB,
products JSONB,
metrics JSONB,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

# Create indexes
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_severity ON cves(severity)")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_published ON cves(published)")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_cwe ON cves(cwe)")

# Cache metadata table
cursor.execute("""
CREATE TABLE IF NOT EXISTS cache_metadata (
key VARCHAR(100) PRIMARY KEY,
last_updated TIMESTAMP,
total_records INTEGER,
metadata JSONB
)
""")

conn.commit()
print("[Database] Tables initialized successfully")
except Exception as e:
print(f"[Database] Error initializing tables: {e}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def save_cves_batch(self, cves: List[Dict[str, Any]]) -> bool:
""""Save multiple CVEs to database - MEMORY OPTIMIZED""""
if not self.use_database or not cves:
return False

conn = self.get_connection()
if not conn:
return False

try:
cursor = conn.cursor()

# Prepare data for batch insert - LIMIT reference_links to save space
values = []
for cve in cves:
# Limit references to first 3 to save database space
refs = cve.get('References', [])[:3]

values.append((
cve.get('ID', ''),
cve.get('Description', ''),
cve.get('Severity', 'UNKNOWN'),
cve.get('CVSS_Score'),
cve.get('CWE'),
cve.get('Published', ''),
cve.get('lastModified', ''),
json.dumps(refs),
json.dumps([]), # Empty products to save space
json.dumps({}) # Empty metrics to save space
))

# Use ON CONFLICT to update existing records
execute_batch(cursor, """
INSERT INTO cves (id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (id) DO UPDATE SET
description = EXCLUDED.description,
severity = EXCLUDED.severity,
cvss_score = EXCLUDED.cvss_score,

```



```

cwe = EXCLUDED.cwe,
published = EXCLUDED.published,
last_modified = EXCLUDED.last_modified,
reference_links = EXCLUDED.reference_links,
products = EXCLUDED.products,
metrics = EXCLUDED.metrics,
updated_at = CURRENT_TIMESTAMP
""", values)

conn.commit()
print(f"[Database] Saved {len(cves)} CVEs to database")
return True
except Exception as e:
print(f"[Database] Error saving CVEs: {e}")
conn.rollback()
return False
finally:
cursor.close()
self.return_connection(conn)

def get_all_cves(self) -> List[Dict[str, Any]]:
"""Get all CVEs from database"""
if not self.use_database:
return []

conn = self.get_connection()
if not conn:
return []

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
cursor.execute("""
SELECT id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics
FROM cves
ORDER BY published DESC
""")

rows = cursor.fetchall()

# Convert to list of dicts
cves = []
for row in rows:
cve = {
'ID': row['id'],
'Description': row['description'],
'Severity': row['severity'],
'CVSS_Score': row['cvss_score'],
'CWE': row['cwe'],
'Published': row['published'],
'lastModified': row['last_modified'],
'References': row['reference_links'] if isinstance(row['reference_links'], list) else [],
'Products': row['products'] if isinstance(row['products'], list) else [],
'metrics': row['metrics'] if isinstance(row['metrics'], dict) else {}
}
cves.append(cve)

return cves
except Exception as e:
print(f"[Database] Error fetching CVEs: {e}")
return []
finally:
cursor.close()
self.return_connection(conn)

def update_cache_metadata(self, key: str, total_records: int, metadata: Dict = None):
"""Update cache metadata"""
if not self.use_database:
return

conn = self.get_connection()
if not conn:
return

try:
cursor = conn.cursor()

```

```

cursor.execute("""
INSERT INTO cache_metadata (key, last_updated, total_records, metadata)
VALUES (%s, CURRENT_TIMESTAMP, %s, %s)
ON CONFLICT (key) DO UPDATE SET
last_updated = CURRENT_TIMESTAMP,
total_records = EXCLUDED.total_records,
metadata = EXCLUDED.metadata
""", (key, total_records, json.dumps(metadata or {})))

conn.commit()
except Exception as e:
print(f"[Database] Error updating metadata: {e}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def get_cache_metadata(self, key: str) -> Optional[Dict]:
"""Get cache metadata"""
if not self.use_database:
return None

conn = self.get_connection()
if not conn:
return None

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
cursor.execute("""
SELECT last_updated, total_records, metadata
FROM cache_metadata
WHERE key = %s
""", (key,))

row = cursor.fetchone()
if row:
return {
'last_updated': row['last_updated'],
'total_records': row['total_records'],
'metadata': row['metadata']
}
return None
except Exception as e:
print(f"[Database] Error fetching metadata: {e}")
return None
finally:
cursor.close()
self.return_connection(conn)

def clear_all_cves(self):
"""Clear all CVE data (for refresh)"""
if not self.use_database:
return

conn = self.get_connection()
if not conn:
return

try:
cursor = conn.cursor()
cursor.execute("DELETE FROM cves")
conn.commit()
print("[Database] Cleared all CVEs from database")
except Exception as e:
print(f"[Database] Error clearing CVEs: {e}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def get_stats(self) -> Dict[str, Any]:
"""Get database statistics"""
if not self.use_database:
return {}

conn = self.get_connection()
if not conn:

```

```
return {}

try:
    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM cves")
    total_cves = cursor.fetchone()[0]

    return {
        'total_cves': total_cves,
        'database_enabled': True
    }
except Exception as e:
    print(f"[Database] Error getting stats: {e}")
    return {}
finally:
    cursor.close()
    self.return_connection(conn)

def close(self):
    """Close all connections in pool"""
    if self.connection_pool:
        self.connection_pool.closeall()
    print("[Database] Connection pool closed")
```

database\models.py

```
from typing import Dict, List, Any
```

```
class CVEDatabase:
    """CVE database model"""

    @staticmethod
    def from_api_format(api_data: Dict) -> Dict:
        """Convert API format to database format"""
        return {
            'ID': api_data.get('ID', ''),
            'Description': api_data.get('Description', ''),
            'Severity': api_data.get('Severity', 'UNKNOWN'),
            'CVSS_Score': api_data.get('CVSS_Score'),
            'CWE': api_data.get('CWE'),
            'Published': api_data.get('Published', ''),
            'lastModified': api_data.get('lastModified', ''),
            'References': api_data.get('References', []),
            'Products': api_data.get('Products', []),
            'metrics': api_data.get('metrics', {})
        }
```

```
class CWEDatabase:
    """CWE database model (for future use)"""

    @staticmethod
    def from_api_format(api_data: Dict) -> Dict:
        """Convert API format to database format"""
        return {
            'code': api_data.get('code', ''),
            'name': api_data.get('name', ''),
            'description': api_data.get('description', ''),
            'mitigations': api_data.get('mitigations', []),
            'examples': api_data.get('examples', [])
        }
```

database__init__.py

```
from .db_manager import DatabaseManager
from .models import CVEDatabase, CWEDatabase

# Global database instance
db_manager = DatabaseManager()

__all__ = ['db_manager', 'DatabaseManager', 'CVEDatabase', 'CWEDatabase']
```

models\cve_model.py

```
# Type hints for better code documentation and IDE support
from typing import Dict, List, Any, Optional
# Date/time handling for parsing CVE publication dates
from datetime import datetime

class CVEModel:
    """Data model for CVE vulnerability records"""

    def __init__(self, data: Dict[str, Any]):
        # Extract CVE identifier (e.g., "CVE-2023-12345") with empty string fallback
        self.id = data.get('ID', '')
        # Get vulnerability description/summary with empty string fallback
        self.description = data.get('Description', '')
        # Extract severity level with 'UNKNOWN' fallback for proper classification
        self.severity = data.get('Severity', 'UNKNOWN')
        # Get CVSS numeric score (can be None if not assessed)
        self.cvss_score = data.get('CVSS_Score')
        # Extract CWE weakness identifier (can be None if not mapped)
        self.cwe = data.get('CWE')
        # Get publication date string with empty string fallback
        self.published = data.get('Published', '')
        # Extract last modified date string with empty string fallback
        self.last_modified = data.get('lastModified', '')
        # Get reference URLs list with empty list fallback
        self.references = data.get('References', [])
        # Extract affected products list with empty list fallback
        self.products = data.get('Products', [])
        # Get CVSS metrics dictionary with empty dict fallback
        self.metrics = data.get('metrics', {})

    def to_dict(self) -> Dict[str, Any]:
        """Convert to dictionary format"""
        # Return all instance attributes as dictionary for JSON serialization
        return {
            'ID': self.id,
            'Description': self.description,
            'Severity': self.severity,
            'CVSS_Score': self.cvss_score,
            'CWE': self.cwe,
            'Published': self.published,
            'lastModified': self.last_modified,
            'References': self.references,
            'Products': self.products,
            'metrics': self.metrics
        }

    def get_published_date(self) -> Optional[datetime]:
        """Get published date as datetime object"""
        # Check if published date string exists
        if self.published:
            try:
                # Handle ISO format with time (e.g., "2023-01-15T10:30:00Z")
                if 'T' in self.published:
                    # Replace Z with timezone offset for proper parsing
                    return datetime.fromisoformat(self.published.replace('Z', '+0000'))
                else:
                    # Handle date-only format (e.g., "2023-01-15")
                    return datetime.strptime(self.published[:10], '%Y-%m-%d')
            except:
                # Return None if any parsing error occurs
                return None
        return None

    def is_critical(self) -> bool:
        """Check if CVE is critical severity"""
        # Case-insensitive check for exactly 'CRITICAL' severity
        return self.severity.upper() == 'CRITICAL'

    def is_high_priority(self) -> bool:
        """Check if CVE is critical or high severity"""
        # Case-insensitive check for either 'CRITICAL' or 'HIGH' severity
        return self.severity.upper() in ['CRITICAL', 'HIGH']
```

models\cwe_model.py

```
# Type hints for better code documentation and IDE support
from typing import Dict, List, Any, Optional
```

```
class CWEModel:
    """Data model for CWE weakness records"""
```

```
def __init__(self, data: Dict[str, Any]):
    # Extract CWE code (e.g., "CWE-79") with empty string fallback
    self.code = data.get('code', '')
    # Get weakness name/title with empty string fallback
    self.name = data.get('name', '')
    # Extract description text with empty string fallback
    self.description = data.get('description', '')
    # Get list of mitigation strategies with empty list fallback
    self.mitigations = data.get('mitigations', [])
    # Extract examples list with empty list fallback
    self.examples = data.get('examples', [])
    # Get relationships to other CWEs with empty list fallback
    self.relationships = data.get('relationships', [])
    # Track data source with 'unknown' fallback for provenance
    self.source = data.get('source', 'unknown')
```

```
def to_dict(self) -> Dict[str, Any]:
    """Convert to dictionary format"""
    # Return all instance attributes as dictionary for serialization
    return {
        'code': self.code,
        'name': self.name,
        'description': self.description,
        'mitigations': self.mitigations,
        'examples': self.examples,
        'relationships': self.relationships,
        'source': self.source
    }
```

```
def get_id(self) -> str:
    """Get CWE ID number from code"""
    # Check if code follows standard "CWE-123" format
    if self.code.startswith('CWE-'):
        # Return numeric part after "CWE-" prefix (index 4 onwards)
        return self.code[4:]
    # Return original code if format doesn't match
    return self.code
```

```
def get_mitre_url(self) -> str:
    """Get MITRE URL for this CWE"""
    # Extract numeric CWE ID
    cwe_id = self.get_id()
    # Build complete URL to MITRE CWE definition page
    return f"https://cwe.mitre.org/data/definitions/{cwe_id}.html"
```

```
def has_mitigations(self) -> bool:
    """Check if CWE has mitigation strategies"""
    # Verify mitigations exist AND first item isn't placeholder text
    return len(self.mitigations) > 0 and self.mitigations[0] != 'Loading detailed information...'
```

routes\api_routes.py

```
# Flask blueprint system for modular API route organization
from flask import Blueprint, jsonify, request
# Central data coordination service for CVE/CWE operations
from services.orchestrator import data_orchestrator

# Create API blueprint with name 'api' for route registration
api_bp = Blueprint('api', __name__)

@api_bp.route("/cve/<cve_id>")
def api_cve_detail(cve_id):
    """API endpoint for CVE details"""
    try:
        # Delegate CVE data retrieval to orchestrator service
        cve = data_orchestrator.get_cve_detail(cve_id)

        # Return structured JSON response with CVE data
        return jsonify({'success': True, 'data': cve})
    except Exception as e:
        # Return JSON error response with HTTP 500 status
        return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/clear-cache", methods=['POST'])
def clear_cache():
    """API endpoint to clear cache"""
    try:
        # Delegate cache clearing to orchestrator service
        data_orchestrator.clear_cache()

        # Return success confirmation for administrative feedback
        return jsonify({'success': True, 'message': 'Cache cleared'})
    except Exception as e:
        # Return JSON error response with HTTP 500 status
        return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/force-refresh")
def force_refresh():
    """Force refresh API data"""
    try:
        # Print debug message for logging refresh operations
        print("[API] Forcing data refresh...")

        # Clear existing cache to ensure fresh data
        data_orchestrator.clear_cache()

        # Import severity analyzer locally to avoid circular imports
        from services.analysis.severity_analyzer import get_severity_card_counts

        # Get fresh vulnerability statistics after cache clear
        severity_counts = get_severity_card_counts()

        # Return success with updated counts and summary message
        return jsonify({
            'success': True,
            'counts': severity_counts,
            'message': f'Refreshed with {severity_counts["total_cves"]} CVEs'
        })
    except Exception as e:
        # Import traceback for enhanced error information
        import traceback

        # Return detailed error info for debugging refresh issues
        return jsonify({
            'success': False,
            'error': str(e),
            'traceback': traceback.format_exc()
        }), 500

@api_bp.route("/data-source-status")
def data_source_status():
    """API endpoint to check current data source configuration"""
    try:
        # Get data source configuration from orchestrator
```



```

status = data_orchestrator.get_data_source_status()

# Return JSON response with system status information
return jsonify({'success': True, 'data': status})
except Exception as e:
# Return JSON error response with HTTP 500 status
return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/timeline-data")
def get_timeline_data():
    """API endpoint for dynamic timeline loading"""
    try:
        years = request.args.get('years', default=1, type=int)

        # Validate years parameter
        if years not in [1, 2, 3, 5]:
            years = 1

        print(f"[API] Loading timeline data for {years} year(s)")

        # Get timeline data from orchestrator
        timeline_data = data_orchestrator._get_timeline(years)

        return jsonify({
            'success': True,
            'timeline': timeline_data,
            'years': years
        })
    except Exception as e:
        import traceback
        return jsonify({
            'success': False,
            'error': str(e),
            'traceback': traceback.format_exc()
        }), 500

@api_bp.route("/cwe/<cwe_code>")
def api_cwe_detail(cwe_code):
    """API endpoint for CWE details ? Enhanced with MITRE scraping simulation"""
    try:
        # Hardcoded CWE database for reliable educational content
        # In production, this would fetch from MITRE or local CWE database
        cwe_details = {
            "CWE-79": {
                "name": "Cross-Site Scripting (XSS)",
                "description": "The software does not neutralize or incorrectly neutralizes user-controllable input before it is placed in output that is used as a web page that is served to other users.",
                "mitigations": [
                    "Use output encoding/escaping for all untrusted data",
                    "Implement Content Security Policy (CSP)",
                    "Validate and sanitize all input on server side"
                ],
                "examples": [
                    "<script>alert('XSS')</script> in user input field",
                    "URL parameter injection: ?search=<script>steal_cookies()</script>"
                ],
                "relationships": [
                    ["CWE-20", "ChildOf"],
                    ["CWE-80", "ParentOf"],
                    ["CWE-81", "ParentOf"]
                ],
                "source": "scraped"
            },
            "CWE-89": {
                "name": "SQL Injection",
                "description": "The software constructs all or part of an SQL command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component.",
                "mitigations": [
                    "Use parameterized queries/prepared statements",
                    "Implement input validation and sanitization",
                    "Apply principle of least privilege for database accounts"
                ],
                "examples": [
                    "' OR '1'='1 in login form",

```

```

"UNION SELECT username, password FROM users--"
},
"relationships": [
["CWE-74", "ChildOf"],
["CWE-943", "ParentOf"]
],
"source": "scraped"
},
"CWE-20": {
"name": "Improper Input Validation",
"description": "The product receives input or data, but it does not validate or
incorrectly validates that the input has the properties that are required to process the
data safely and correctly.",
"mitigations": [
"Assume all input is malicious and validate accordingly",
"Use whitelist validation rather than blacklist",
"Implement server-side validation for all inputs"
],
"examples": [
"Accepting negative numbers for array indices",
"Not validating file upload types"
],
"relationships": [
["CWE-707", "ChildOf"],
["CWE-79", "ParentOf"],
["CWE-89", "ParentOf"]
],
"source": "scraped"
}
}

# Get requested CWE data or return generic fallback structure
cwe_info = cwe_details.get(cwe_code, {
"name": f"CWE {cwe_code}",
"description": f"Detailed information for {cwe_code} would be fetched from MITRE
database.",
"mitigations": ["Contact security team for specific mitigation strategies"],
"examples": ["Examples would be loaded from MITRE database"],
"relationships": [],
"source": "placeholder"
})

# Return JSON response with CWE data
return jsonify({'success': True, 'data': cwe_info})
except Exception as e:
# Return JSON error response with HTTP 500 status
return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/test-api-connection")
def test_api_connection():
    """Test API connection and return basic stats"""
    try:
        from services.data.api_client import api_client
        from datetime import datetime

# Get cache stats
cache_stats = api_client.get_cache_stats()

# Test basic functionality
test_results = {
'api_key_configured': bool(api_client.api_key),
'cache_stats': cache_stats,
'timestamp': datetime.now().isoformat()
}

return jsonify({
'success': True,
'message': 'API connection test successful',
'data': test_results
})
except Exception as e:
return jsonify({
'success': False,
'error': str(e)
}), 500

@api_bp.route("/health")

```

```
def health_check():  
    """Simple health check endpoint"""  
    from datetime import datetime  
    return jsonify({  
        'status': 'healthy',  
        'timestamp': datetime.now().isoformat(),  
        'service': 'VulnEdu API'  
    })
```

routes\learn_routes.py

```
# Flask blueprint system for modular route organization
from flask import Blueprint, render_template, redirect, url_for
# Date/time handling for current timestamps and timezone awareness
from datetime import datetime, timezone
# CVE data retrieval service for educational content
from services.data.api_client import api_client
# CWE analysis and vendor risk assessment services
from services.analysis.cwe_processor import cwe_processor, get_vendor_risk_analysis

# Create learning blueprint with name 'learn' for route registration
learn_bp = Blueprint('learn', __name__)

# Static CWE titles for educational content - curated for learning value
CWE_TITLES = {
    "CWE-79": "Cross-Site Scripting",
    "CWE-89": "SQL Injection",
    "CWE-20": "Improper Input Validation",
    "CWE-22": "Path Traversal",
    "CWE-119": "Buffer Overflow",
    "CWE-200": "Information Exposure",
    "CWE-287": "Improper Authentication",
    "CWE-78": "OS Command Injection"
}

@learn_bp.route("/")
def learn():
    """Redirect to main learn topic"""
    # Redirect root learning requests to default educational topic
    return redirect(url_for('learn.learn_topic', topic='what-is-cve'))

@learn_bp.route("/<topic>")
def learn_topic(topic):
    """Learn topic pages"""
    # Whitelist of valid educational topics for security and content control
    valid_topics = [
        'what-is-cve', 'what-is-cve', 'cvss-scores',
        'what-is-nvd-mitre', 'cve-vs-cwe-vs-cvss'
    ]

    # Validate requested topic and redirect to default if invalid
    if topic not in valid_topics:
        return redirect(url_for('learn.learn_topic', topic='what-is-cve'))

    try:
        # Get recent CVE data for educational examples (limit to 25 for performance)
        latest_cves = api_client.get_cves_last_30_days()[:25]

        # Initialize data containers for template rendering
        cwe_dict = {}
        cwe_severity = {}
        key_cwes = []
        key_cwe_titles = {}

        # Enhanced data loading for CWE-focused educational content
        if topic == 'what-is-cve':
            # Load advanced CWE analysis data using processor
            cwe_severity = cwe_processor.get_cwe_severity_data(10)
            cwe_dict = cwe_processor.get_cwe_details()
            # Get processor's curated CWE list instead of static list
            key_cwes = cwe_processor.get_key_cwes()
            key_cwe_titles = cwe_processor.get_key_cwe_titles()

        # Attempt to use advanced vendor risk analysis for educational insights
        try:
            vendor_risk_data = get_vendor_risk_analysis()
            # Check if vendor risk data is available and properly formatted
            if vendor_risk_data and 'top_10_cwes' in vendor_risk_data:
                # Transform vendor risk data to chart-compatible format
                top_10_data = vendor_risk_data['top_10_cwes']
                severity_matrix = vendor_risk_data.get('severity_matrix', {})
        except:
            pass

        # Initialize severity data structure for charts
```

```

cwe_severity = {
'labels': top_10_data.get('labels', []),
'indices': top_10_data.get('indices', []),
'data': {
'CRITICAL': [],
'HIGH': [],
'MEDIUM': [],
'LOW': [],
'UNKNOWN': []
}
}

# Populate severity data for each CWE from vendor analysis
for cwe_code in top_10_data.get('indices', []):
cwe_severities = severity_matrix.get(cwe_code, {})
cwe_severity['data']['CRITICAL'].append(cwe_severities.get('CRITICAL', 0))
cwe_severity['data']['HIGH'].append(cwe_severities.get('HIGH', 0))
cwe_severity['data']['MEDIUM'].append(cwe_severities.get('MEDIUM', 0))
cwe_severity['data']['LOW'].append(cwe_severities.get('LOW', 0))
cwe_severity['data']['UNKNOWN'].append(cwe_severities.get('UNKNOWN', 0))

except Exception as e:
# Log vendor risk analysis errors but continue with fallback data
print(f"[Learn] Error getting vendor risk data: {e}")
# Fallback continues with processor data already loaded above
pass

else:
# Basic data loading for non-CWE topics
# Import severity analyzer locally to avoid circular imports
from services.analysis.severity_analyzer import get_severity_card_counts
cwe_severity = get_severity_card_counts()
# Use static CWE data for basic educational content
key_cwes = list(CWE_TITLES.keys())
key_cwe_titles = CWE_TITLES

# Render topic-specific template with aggregated educational data
return render_template(
f"learn/{topic}.html",
cwe_dict=cwe_dict,
cwe_severity=cwe_severity,
latest_cves=latest_cves,
key_cwes=key_cwes,
key_cwe_titles=key_cwe_titles,
now=datetime.now(timezone.utc) # Current timestamp for templates
)
except Exception as e:
# Log educational page errors for debugging
print(f"[Flask] Learn page error: {e}")
# Provide safe fallback data to ensure educational continuity
return render_template(
f"learn/{topic}.html",
cwe_dict={},
cwe_severity={},
latest_cves=[],
key_cwes=list(CWE_TITLES.keys()), # Use static CWE list as fallback
key_cwe_titles=CWE_TITLES, # Use static titles as fallback
now=datetime.now(timezone.utc)
)

```

routes\main_routes.py

```
from flask import Blueprint, render_template, request, redirect, url_for
from datetime import datetime, timezone, timedelta
import math
from services.orchestrator import data_orchestrator
from utils.filters import FilterService

main_bp = Blueprint('main', __name__)
filter_service = FilterService()

@main_bp.route("/")
def index():
    """Main dashboard route"""
    try:
        print("[Flask] Loading dashboard...")

        year = request.args.get('year', type=int)
        month = request.args.get('month', type=int)
        day = request.args.get('day', type=int)
        severity_filter = request.args.get('severity')
        search_query = request.args.get('q')
        timeline_years = request.args.get('timeline_years', default=1, type=int)

        if timeline_years not in [1, 2, 3, 5]:
            timeline_years = 1

        print(f"[Flask] Filters: year={year}, month={month}, day={day},
severity={severity_filter}, search={search_query}, timeline_years={timeline_years}")

        dashboard_data = data_orchestrator.get_dashboard_data(
            year=year,
            month=month,
            day=day,
            severity_filter=severity_filter,
            timeline_years=timeline_years,
            load_historical=True
        )

        if search_query:
            filtered_cves = filter_service.filter_by_search(dashboard_data['latest_cves'],
search_query)

        from collections import Counter
        severity_counts = Counter()
        for cve in filtered_cves:
            severity = cve.get('Severity', 'UNKNOWN').upper()
            if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
                severity_counts[severity] += 1
            else:
                severity_counts['UNKNOWN'] += 1

        dashboard_data['metrics'] = {
            'CRITICAL': severity_counts.get('CRITICAL', 0),
            'HIGH': severity_counts.get('HIGH', 0),
            'MEDIUM': severity_counts.get('MEDIUM', 0),
            'LOW': severity_counts.get('LOW', 0),
            'UNKNOWN': severity_counts.get('UNKNOWN', 0)
        }
        dashboard_data['total_cves'] = len(filtered_cves)
        dashboard_data['latest_cves'] = filtered_cves

        total = len(filtered_cves)
        if total > 0:
            dashboard_data['severity_percentage'] = {}
            for key in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']:
                percentage = round((severity_counts.get(key, 0) * 100 / total))
                dashboard_data['severity_percentage'][key] = f"{percentage}%"
            else:
                dashboard_data['severity_percentage'] = {k: "0%" for k in ['CRITICAL', 'HIGH', 'MEDIUM',
'LOW', 'UNKNOWN']}

        dashboard_data['note_text'] = f"Showing search results for '{search_query}'"
```

```

return render_template(
    "pages/dashboard.html",
    metrics=dashboard_data['metrics'],
    total_cves=dashboard_data['total_cves'],
    available_years=dashboard_data['available_years'],
    available_months=dashboard_data['available_months'],
    timeline_data_days=dashboard_data['timeline_data_days'],
    timeline_data_years=dashboard_data['timeline_data_years'],
    severity_stats=dashboard_data['metrics'],
    severity_percentage=dashboard_data['severity_percentage'],
    cwe_radar=dashboard_data['cwe_radar'],
    cwe_radar_all=dashboard_data['cwe_radar_all'],
    cwe_radar_weighted=dashboard_data['cwe_radar_weighted'],
    cwe_radar_descriptions=dashboard_data['cwe_radar_descriptions'],
    historical_loaded=dashboard_data.get('historical_loaded', False),
    year_filter=year,
    month_filter=month,
    day_filter=day,
    severity_filter=severity_filter,
    search_query=search_query,
    timeline_years=timeline_years
)

except Exception as e:
    print(f"[Flask] Dashboard error: {e}")
    import traceback
    traceback.print_exc()

return render_template(
    "pages/dashboard.html",
    metrics={'CRITICAL': 0, 'HIGH': 0, 'MEDIUM': 0, 'LOW': 0, 'UNKNOWN': 0},
    total_cves=0,
    available_years=list(range(datetime.now().year, 1998, -1)),
    available_months=list(range(1, 13)),
    timeline_data_days={'labels': [], 'values': []},
    timeline_data_years={'labels': [], 'values': []},
    severity_stats={'CRITICAL': 0, 'HIGH': 0, 'MEDIUM': 0, 'LOW': 0, 'UNKNOWN': 0},
    severity_percentage={'CRITICAL': '0%', 'HIGH': '0%', 'MEDIUM': '0%', 'LOW': '0%',
                        'UNKNOWN': '0%'},
    cwe_radar={'indices': [], 'labels': [], 'values': []},
    cwe_radar_all={},
    cwe_radar_weighted={'indices': [], 'labels': [], 'values': []},
    cwe_radar_descriptions={},
    historical_loaded=False,
    year_filter=None,
    month_filter=None,
    day_filter=None,
    severity_filter=None,
    search_query=None,
    timeline_years=1
)

@main_bp.route("/vulnerabilities")
def vulnerabilities():
    """Vulnerabilities listing page"""
    try:
        year = request.args.get('year', type=int)
        month = request.args.get('month', type=int)
        day = request.args.get('day', type=int)
        severity_filter = request.args.get('severity')
        search_query = request.args.get('q')
        page = request.args.get('page', default=1, type=int)
        per_page = 20

        print(f"[Flask] Vulnerabilities filters: year={year}, month={month}, day={day},
        severity={severity_filter}, search={search_query}")

        all_cves = data_orchestrator.get_filtered_cves(year=year, month=month, day=day)
        print(f"[Flask] Got {len(all_cves)} CVEs from orchestrator")

        if severity_filter:
            all_cves = filter_service.filter_by_severity(all_cves, severity_filter)
            print(f"[Flask] After severity filter: {len(all_cves)} CVEs")

        if search_query:
            all_cves = filter_service.filter_by_search(all_cves, search_query)

```

```

print(f"[Flask] After search filter: {len(all_cves)} CVEs")

all_cves = sorted(all_cves, key=lambda x: x.get('Published', ''), reverse=True)

total_results = len(all_cves)
total_pages = max(1, math.ceil(total_results / per_page))
current_page = max(1, min(page, total_pages))
start_index = (current_page - 1) * per_page
end_index = start_index + per_page
cves_page = all_cves[start_index:end_index]

page_numbers = filter_service.generate_page_numbers(current_page, total_pages)

note_text = filter_service.generate_note_text(year, month, day)
if severity_filter:
    note_text += f" (filtered by {severity_filter.lower()} severity)"
if search_query:
    note_text += f" (search: '{search_query}')"

print(f"[Flask] Returning {len(cves_page)} CVEs for page {current_page}")

return render_template(
    "pages/vulnerabilities.html",
    latest_cves=cves_page,
    year_filter=year,
    month_filter=month,
    day_filter=day,
    severity_filter=severity_filter,
    search_query=search_query,
    available_years=list(range(datetime.now().year, 1998, -1)),
    available_months=list(range(1, 13)),
    current_page=current_page,
    total_pages=total_pages,
    page_numbers=page_numbers,
    total_results=total_results,
    note_text=note_text
)

except Exception as e:
    print(f"[Flask] Vulnerabilities page error: {e}")
    import traceback
    traceback.print_exc()

    return render_template(
        "pages/vulnerabilities.html",
        latest_cves=[],
        total_results=0,
        note_text="Error loading vulnerabilities"
    )

@main_bp.route("/cve/<cve_id>")
def cve_detail(cve_id):
    """CVE detail page"""
    try:
        cve = data_orchestrator.get_cve_detail(cve_id)
        return render_template("pages/cve_detail.html", cve=cve)
    except Exception as e:
        print(f"[Flask] CVE detail error: {e}")
        return render_template("pages/cve_detail.html", cve={'ID': cve_id, 'Description': 'Error loading CVE details'})

```


routes\utility_routes.py

```
# Flask blueprint system for modular route organization
from flask import Blueprint, render_template

# Create utility blueprint for reference and documentation routes
utility_bp = Blueprint('utility', __name__)

@utility_bp.route("/references")
def references():
    """References and resources page"""
    # Render main references template - serves as navigation hub for documentation
    return render_template("references.html")

@utility_bp.route("/references/<section>")
def references_section(section):
    """References section pages"""
    # Whitelist of valid reference sections for security and content control
    valid_sections = ['data-sources', 'apis-feeds', 'tech-docs', 'dev-tools']

    # Validate section parameter to prevent unauthorized template access
    if section not in valid_sections:
        # Fallback to main references page for invalid sections
        return render_template("references.html")

    # Render section-specific template using validated parameter
    return render_template(f"references/{section}.html")
```

services\orchestrator.py

```
import os
import json
from typing import Dict, List, Any
from datetime import datetime, timezone, timedelta
from services.analysis.trend_analyzer import get_cve_trends_last_30_days
from services.analysis.severity_analyzer import get_severity_card_counts,
get_severity_percentages
from services.analysis.cwe_processor import get_vendor_risk_analysis,
get_weighted_cwe_analysis
from services.data.api_client import api_client
from services.cache.cache_manager import cache_manager
from services.data.data_source_config import data_source_config
import config

class DataOrchestrator:
    """Clean orchestrator using separate visualization modules with caching and dynamic data
    sources"""

    def __init__(self):
        data_source_config.log_data_source_status()
        self._vendor_risk_cache = None
        self._vendor_risk_cache_time = None

    def get_dashboard_data(self, year=None, month=None, day=None, severity_filter=None,
        timeline_years=1, load_historical=False):
        """Get all dashboard data with separate data sources for different components"""
        print(f"[Orchestrator] Loading dashboard data with filters: year={year}, month={month},
        day={day}, severity={severity_filter}, timeline_years={timeline_years},
        load_historical={load_historical}")

        try:
            # 1. CVE Trends (Last 30 days)
            print("[Orchestrator] Getting CVE trends (last 30 days) - unfiltered")
            cve_trends = self._get_always_30_day_trends()

            # 2. Vulnerabilities Over Time
            if load_historical and timeline_years > 0:
                print(f"[Orchestrator] Getting vulnerabilities over time (last {timeline_years} years) -
                HISTORICAL LOAD")
                vulnerabilities_timeline = self._get_timeline(timeline_years)
            else:
                print("[Orchestrator] Skipping historical timeline (not requested)")
                vulnerabilities_timeline = {'labels': [], 'values': [], 'total_cves': 0,
                'months_covered': 0, 'raw_data': {}}

            # 3. Filtered data for Cards and Pie Chart ONLY
            print("[Orchestrator] Getting filtered data for cards and pie chart")
            filtered_cves = self.get_filtered_cves(year=year, month=month, day=day)

            if severity_filter:
                from utils.filters import FilterService
                filter_service = FilterService()
                filtered_cves = filter_service.filter_by_severity(filtered_cves, severity_filter)

            # 4. Calculate severity counts
            from collections import Counter
            severity_counts = Counter()
            for cve in filtered_cves:
                severity = cve.get('Severity', 'UNKNOWN').upper()
                if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
                    severity_counts[severity] += 1
                else:
                    severity_counts['UNKNOWN'] += 1

            severity_metrics = {
                'CRITICAL': severity_counts.get('CRITICAL', 0),
                'HIGH': severity_counts.get('HIGH', 0),
                'MEDIUM': severity_counts.get('MEDIUM', 0),
                'LOW': severity_counts.get('LOW', 0),
                'UNKNOWN': severity_counts.get('UNKNOWN', 0),
                'total_cves': len(filtered_cves)
            }
        }
```

```

total = len(filtered_cves)
if total > 0:
    severity_percentages = {}
    for key in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']:
        percentage = round((severity_counts.get(key, 0) * 100 / total))
        severity_percentages[key] = f"{percentage}%"
    else:
        severity_percentages = {k: "0%" for k in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']}

# 5. Vendor Risk Analysis - Cache for 24 hours
cache_expired = False
if self._vendor_risk_cache_time:
    cache_age = datetime.now() - self._vendor_risk_cache_time
    if cache_age > timedelta(hours=24):
        cache_expired = True
    print("[Orchestrator] Vendor risk cache expired (>24 hours old)")

if self._vendor_risk_cache and not cache_expired:
    cache_age_str = ""
    if self._vendor_risk_cache_time:
        age = datetime.now() - self._vendor_risk_cache_time
        hours = int(age.total_seconds() / 3600)
        cache_age_str = f" (cached {hours}h ago)"
    print(f"[Orchestrator] Using cached vendor risk analysis{cache_age_str}")
    vendor_risk, weighted_vendor_risk = self._vendor_risk_cache
else:
    print("[Orchestrator] Loading vendor risk analysis for 2025 (will cache for 24 hours)")
    try:
        vendor_risk = get_vendor_risk_analysis()
        weighted_vendor_risk = get_weighted_cwe_analysis()
        self._vendor_risk_cache = (vendor_risk, weighted_vendor_risk)
        self._vendor_risk_cache_time = datetime.now()
        print("[Orchestrator] Vendor risk cached (expires in 24 hours)")
    except Exception as e:
        print(f"[Orchestrator] Error loading vendor risk analysis: {e}")
        vendor_risk = self._get_empty_vendor_risk()
        weighted_vendor_risk = {'indices': [], 'labels': [], 'values': []}

cutoff_info, explanation = data_source_config.get_data_source_cutoff()
if year and month and day:
    note_text = f"Cards & Pie Chart: {year}-{month:02d}-{day:02d} | Trends: Last 30 days"
elif year and month:
    note_text = f"Cards & Pie Chart: {year}-{month:02d} | Trends: Last 30 days"
elif year:
    note_text = f"Cards & Pie Chart: {year} data | Trends: Last 30 days"
elif severity_filter:
    note_text = f"Cards & Pie Chart: {severity_filter.lower()} severity | Trends: Last 30 days"
else:
    note_text = f"Cards & Pie Chart: Last 30 days | Trends: Last 30 days"
if load_historical:
    note_text += f" | Timeline: Last {timeline_years} year(s)"

print(f"[Orchestrator] Data loaded successfully:")
print(f" - Filtered CVEs (cards/pie): {len(filtered_cves)}")
print(f" - CVE Trends data points: {len(cve_trends.get('labels', []))}")
print(f" - Timeline data points: {len(vulnerabilities_timeline.get('labels', []))}")
print(f" - Severity counts: C={severity_metrics['CRITICAL']}, H={severity_metrics['HIGH']}, M={severity_metrics['MEDIUM']}, L={severity_metrics['LOW']}")

dashboard_data = {
    'metrics': severity_metrics,
    'total_cves': len(filtered_cves),
    'severity_percentage': severity_percentages,
    'timeline_data_days': cve_trends,
    'timeline_data_years': vulnerabilities_timeline,
    'cwe_radar': vendor_risk.get('top_10_cwes', {'indices': [], 'labels': [], 'values': []}),
    'cwe_radar_all': vendor_risk,
    'cwe_radar_weighted': weighted_vendor_risk,
    'cwe_radar_descriptions': self._get_cwe_descriptions(),
    'latest_cves': filtered_cves,
    'available_years': list(range(datetime.now().year, 2009, -1)),
    'available_months': list(range(1, 13)),

```

```

'note_text': note_text,
'historical_loaded': load_historical
}

print(f"[Orchestrator] Dashboard data structure created successfully")
return dashboard_data

except Exception as e:
print(f"[Orchestrator] Error in get_dashboard_data: {e}")
import traceback
traceback.print_exc()
raise e

def _get_empty_vendor_risk(self):
return {
'top_5_cwes': {'indices': [], 'labels': [], 'values': []},
'top_10_cwes': {'indices': [], 'labels': [], 'values': []},
'all_cwes': {'indices': [], 'labels': [], 'values': []},
'severity_matrix': {},
'cwe_30_day_counts': {},
'total_cves_analyzed': 0,
'years_analyzed': []
}

def _get_always_30_day_trends(self) -> Dict[str, Any]:
try:
return get_cve_trends_last_30_days()
except Exception as e:
print(f"[Orchestrator] Error getting 30-day trends: {e}")
return {'labels': [], 'values': [], 'total_cves': 0, 'date_range': {'start': '', 'end': ''}}

def _get_timeline(self, years=1) -> Dict[str, Any]:
try:
from services.analysis.trend_analyzer import get_vulnerabilities_over_time_last_n_years
return get_vulnerabilities_over_time_last_n_years(years)
except Exception as e:
print(f"[Orchestrator] Error getting timeline: {e}")
return {'labels': [], 'values': [], 'total_cves': 0, 'months_covered': 0, 'raw_data': {}}

def get_filtered_cves(self, year=None, month=None, day=None) -> List[Dict[str, Any]]:
print(f"[Orchestrator] Getting filtered CVEs: year={year}, month={month}, day={day}")
cutoff_info, explanation = data_source_config.get_data_source_cutoff()

if not year:
print(f"[Orchestrator] Using API data for recent data")
cves = api_client.get_cves_last_30_days()
print(f"[Orchestrator] API returned {len(cves)} CVEs")
return cves

if data_source_config.should_use_historical(year, month):
print(f"[Orchestrator] Using HISTORICAL data for {year}-{month or 'all'}")
from services.analysis.trend_analyzer import get_filtered_historical_cves
return get_filtered_historical_cves(year=year, month=month, day=day)
else:
print(f"[Orchestrator] Using API data for {year}-{month or 'all'}")
return api_client.get_cves_for_date_range(year=year, month=month, day=day)

def get_cve_detail(self, cve_id: str) -> Dict[str, Any]:
return api_client.get_cve_detail(cve_id)

def clear_cache(self):
api_client.clear_cache()
cache_manager.clear_all()
self._vendor_risk_cache = None
self._vendor_risk_cache_time = None

def get_data_source_status(self) -> Dict[str, Any]:
return data_source_config.get_status()

def _get_cwe_descriptions(self) -> Dict[str, str]:
return {}

data_orchestrator = DataOrchestrator()

```

services\analysis\cwe_processor.py

```
"""
CWE Processor - Main Interface
This file maintains backward compatibility while using the new modular CWE structure
"""

# Import all functionality from the modular CWE package
# This provides a transparent interface to the new architecture
from .cwe import (
    cwe_processor,          # Global processor instance for backward compatibility
    CWESeverityProcessor,   # Main processor class for CWE severity analysis
    get_vendor_risk_analysis, # Advanced vendor risk assessment function
    get_weighted_cwe_analysis, # Severity-weighted CWE scoring function
    get_cwe_30_day_counts,   # Recent CWE frequency analysis function
    get_cwe_title           # CWE code to title resolution utility
)

# Re-export everything for backward compatibility
# This ensures existing code can continue using the same import paths
__all__ = [
    'cwe_processor',        # Legacy global instance access
    'CWESeverityProcessor', # Class for creating custom processor instances
    'get_vendor_risk_analysis', # Comprehensive risk analysis functionality
    'get_weighted_cwe_analysis', # Weighted analysis for priority assessment
    'get_cwe_30_day_counts',   # Temporal trend analysis capabilities
    'get_cwe_title'           # Utility function for CWE title resolution
]
```

services\analysis\severity_analyzer.py

```
# Type hints for function return structures
from typing import Dict, Any
# Efficient counting data structure for frequency analysis
from collections import Counter
# API client service for retrieving recent vulnerability data
from services.data.api_client import api_client

def get_severity_card_counts() -> Dict[str, Any]:
    """Get severity counts for dashboard cards - API ONLY"""
    print("[Severity Cards] Calculating severity distribution from API...")

    # Get recent vulnerability data from API (last 30 days)
    cves = api_client.get_cves_last_30_days()

    # Count occurrences by severity level using efficient Counter
    severity_counts = Counter()
    for cve in cves:
        # Extract severity with fallback to UNKNOWN
        severity = cve.get('Severity', 'UNKNOWN').upper()
        # Validate against known CVSS severity levels
        if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
            severity_counts[severity] += 1
        else:
            # Handle invalid or missing severities as UNKNOWN
            severity_counts['UNKNOWN'] += 1

    # Build comprehensive statistics structure
    result = {
        'CRITICAL': severity_counts.get('CRITICAL', 0),
        'HIGH': severity_counts.get('HIGH', 0),
        'MEDIUM': severity_counts.get('MEDIUM', 0),
        'LOW': severity_counts.get('LOW', 0),
        'UNKNOWN': severity_counts.get('UNKNOWN', 0),
        'total_cves': len(cves),
        # Calculate total excluding unknown for certain analysis
        'total_without_unknown': (severity_counts.get('CRITICAL', 0) +
            severity_counts.get('HIGH', 0) +
            severity_counts.get('MEDIUM', 0) +
            severity_counts.get('LOW', 0))
    }

    # Log detailed breakdown for monitoring and debugging
    print(f"[Severity Cards] Counts: Critical={result['CRITICAL']}, High={result['HIGH']},
    Medium={result['MEDIUM']}, Low={result['LOW']}, Unknown={result['UNKNOWN']}")
    return result

def get_severity_percentages(counts: Dict[str, Any]) -> Dict[str, str]:
    """Calculate percentages that sum to 100% using proper rounding"""
    total = counts['total_cves']
    # Handle zero division case gracefully
    if total == 0:
        return {k: "0%" for k in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']}

    # Calculate exact floating-point percentages
    exact_percentages = {
        'CRITICAL': (counts['CRITICAL'] * 100 / total),
        'HIGH': (counts['HIGH'] * 100 / total),
        'MEDIUM': (counts['MEDIUM'] * 100 / total),
        'LOW': (counts['LOW'] * 100 / total),
        'UNKNOWN': (counts['UNKNOWN'] * 100 / total)
    }

    # Round each percentage to nearest integer
    rounded_percentages = {k: round(v) for k, v in exact_percentages.items()}

    # Check if rounding caused sum to deviate from 100%
    total_rounded = sum(rounded_percentages.values())
    difference = 100 - total_rounded

    # Apply largest remainder method to correct rounding errors
    if difference != 0:
        # Calculate fractional parts for adjustment priority
```

```

fractional_parts = {k: exact_percentages[k] - rounded_percentages[k]
for k in exact_percentages.keys()}

if difference > 0:
# Need to add percentage points to reach 100%
for _ in range(difference):
# Add to category with largest positive fractional part
max_key = max(fractional_parts.keys(), key=lambda k: fractional_parts[k])
rounded_percentages[max_key] += 1
fractional_parts[max_key] -= 1
else:
# Need to subtract percentage points to reach 100%
for _ in range(abs(difference)):
# Subtract from category with largest negative fractional part
min_key = min(fractional_parts.keys(), key=lambda k: fractional_parts[k])
rounded_percentages[min_key] -= 1
fractional_parts[min_key] += 1

# Convert to display-ready percentage strings
return {k: f"{v}%" for k, v in rounded_percentages.items()}

def get_severity_distribution_pie() -> Dict[str, Any]:
    """Get severity distribution for pie chart - includes UNKNOWN"""
    print("[Severity Distribution] Using severity card counts for pie chart...")

# Get base severity statistics
counts = get_severity_card_counts()
# Calculate accurate percentages that sum to 100%
percentages = get_severity_percentages(counts)

# Build complete chart data structure
result = {
'counts': counts,                # Raw count data
'percentages': percentages,      # Formatted percentage strings
'chart_data': {
# Labels for chart legend and display
'labels': ['Critical', 'High', 'Medium', 'Low', 'Unknown'],
# Values for chart segments
'values': [
counts['CRITICAL'],
counts['HIGH'],
counts['MEDIUM'],
counts['LOW'],
counts['UNKNOWN']
],
# Colors matching severity levels (accessible color scheme)
'colors': ['#f55855', '#f8a541', '#3b8ded', '#42d392', '#6b7280']
},
'total_including_unknown': counts['total_cves']
}

print(f"[Severity Distribution] Pie chart data generated with
{result['total_including_unknown']} total CVEs")
return result

```

services\analysis\trend_analyzer.py

```
from typing import Dict, List, Any
from datetime import datetime, timezone, timedelta
from collections import defaultdict
from services.data.api_client import api_client

def get_cve_trends_last_30_days() -> Dict[str, Any]:
    """Get CVE trends for last 30 days - API ONLY"""
    print("[CVE Trends] Fetching 30-day trends from API...")

    # Get recent vulnerability data from API
    cves = api_client.get_cves_last_30_days()

    # Initialize daily counting structure
    daily_counts = defaultdict(int)
    today = datetime.now(timezone.utc).date()

    # Pre-fill last 30 days with zero counts
    for i in range(30):
        day = today - timedelta(days=29-i)
        daily_counts[day.strftime('%Y-%m-%d')] = 0

    # Process each CVE to count by publication date
    for cve in cves:
        pub_date = cve.get('Published', '')
        if pub_date:
            try:
                if 'T' in pub_date:
                    dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
                else:
                    dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')

                date_key = dt.date().strftime('%Y-%m-%d')
                if date_key in daily_counts:
                    daily_counts[date_key] += 1
            except:
                continue

    # Build chart-ready data structure
    sorted_dates = sorted(daily_counts.keys())
    result = {
        'labels': sorted_dates,
        'values': [daily_counts[date] for date in sorted_dates],
        'total_cves': len(cves),
        'date_range': {
            'start': sorted_dates[0] if sorted_dates else '',
            'end': sorted_dates[-1] if sorted_dates else ''
        }
    }

    print(f"[CVE Trends] Generated trends: {result['total_cves']} CVEs over {len(sorted_dates)} days")
    return result

def get_vulnerabilities_over_time_last_n_years(years=1) -> Dict[str, Any]:
    """Get vulnerability timeline - ALWAYS LOADS 2025 ONLY - MEMORY OPTIMIZED"""
    print(f"[Vulnerabilities Over Time] Loading 2025 data only (ignoring years parameter for memory optimization)...")

    from services.data.data_processor import historical_loader

    # FORCE 2025 ONLY - ignore years parameter to save memory!
    current_year = datetime.now().year
    years_to_load = [current_year] # CHANGED: Always only current year!

    print(f"[Vulnerabilities Over Time] Loading years: {years_to_load}")

    monthly_counts = defaultdict(int)
    total_cves = 0

    for year in years_to_load:
        year_cves = historical_loader.get_year_data(year)
        print(f"[Vulnerabilities Over Time] Processing {year}: {len(year_cves)} CVEs")
```



```

total_cves += len(year_cves)

for cve in year_cves:
    pub_date = cve.get('Published', '')
    if pub_date:
        try:
            if 'T' in pub_date:
                dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
            else:
                dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')
            month_key = dt.strftime('%Y-%m')
            monthly_counts[month_key] += 1
        except Exception as e:
            continue

# Clear cache after processing to save memory
if str(year) in historical_loader._year_cache:
    print(f"[Vulnerabilities Over Time] Clearing cache for {year} to save memory")
    del historical_loader._year_cache[str(year)]

sorted_months = sorted(monthly_counts.items())

result = {
    'labels': [item[0] for item in sorted_months],
    'values': [item[1] for item in sorted_months],
    'total_cves': total_cves,
    'months_covered': len(sorted_months),
    'raw_data': dict(monthly_counts)
}

print(f"[Vulnerabilities Over Time] Generated timeline: {result['total_cves']} CVEs
across {result['months_covered']} months")

return result

def get_filtered_historical_cves(year: int = None, month: int = None, day: int = None)
-> List[Dict[str, Any]]:
    """Get filtered CVEs from historical data - LAZY LOADING - MEMORY OPTIMIZED"""
    print(f"[Historical Filter] Getting filtered data for year={year}, month={month},
day={day}")

    # Year is required for historical filtering
    if not year:
        print("[Historical Filter] No year specified, returning empty list")
        return []

    # Lazy import to avoid circular dependencies
    from services.data.data_processor import historical_loader

    # Load specific year data for efficient filtering - LAZY LOAD
    year_cves = historical_loader.get_year_data(year)
    print(f"[Historical Filter] Loaded {len(year_cves)} CVEs for year {year}")

    # If no month specified, return all CVEs for the year
    if not month:
        print(f"[Historical Filter] No month filter, returning all {len(year_cves)} CVEs for
{year}")
        return year_cves

    # Apply hierarchical filtering: year ? month ? day
    filtered_cves = []
    for cve in year_cves:
        pub_date = cve.get('Published', '')
        if pub_date:
            try:
                if 'T' in pub_date:
                    dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
                else:
                    dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')

                # CRITICAL: Check BOTH year and month
                if dt.year == year and dt.month == month:
                    # If day is specified, check day too
                    if day is None or dt.day == day:
                        filtered_cves.append(cve)
            except Exception as e:
                continue

```

```
print(f"[Historical Filter] Filtered result: {len(filtered_cves)} CVEs for  
{year}-{month:02d}" + (f"-{day:02d}" if day else ""))  
  
# Debug: Show sample of filtered results  
if filtered_cves:  
    sample_count = min(3, len(filtered_cves))  
    print(f"[Historical Filter] Sample of filtered CVEs:")  
    for i in range(sample_count):  
        cve = filtered_cves[i]  
        print(f"    - {cve.get('ID', 'Unknown')}: {cve.get('Published', 'No date')}")  
  
return filtered_cves
```

services\analysis\cwe\analyzer.py

```
"""
CWE Risk Analysis Functions
Handles vendor risk analysis, weighted analysis, and 30-day counts
"""

# Efficient counting for CWE frequency analysis
from collections import Counter
# Type hints for function return values
from typing import Dict, Any
# Date operations for temporal analysis
from datetime import datetime
# CWE title resolution utility
from .utils import get_cwe_title

def get_vendor_risk_analysis() -> Dict[str, Any]:
    """Get CWE analysis for 2025 ONLY - MEMORY OPTIMIZED"""
    print("[Vendor Risk Analysis] Analyzing CWE patterns for 2025 only...")
    try:
        from services.data.data_processor import historical_loader

        # ONLY LOAD 2025 - saves memory!
        current_year = datetime.now().year
        years_to_analyze = [current_year] # CHANGED: Only current year!

        cwe_counts = Counter()
        cwe_severity_matrix = {}
        total_cves = 0

        for year in years_to_analyze:
            year_cves = historical_loader.get_year_data(year)
            print(f"[Vendor Risk Analysis] Processing {year}: {len(year_cves)} CVEs")
            total_cves += len(year_cves)

            for cve in year_cves:
                cwe = cve.get('CWE')
                if cwe and cwe.startswith('CWE'):
                    cwe_counts[cwe] += 1
                    if cwe not in cwe_severity_matrix:
                        cwe_severity_matrix[cwe] = Counter()
                    severity = cve.get('Severity', 'UNKNOWN').upper()
                    cwe_severity_matrix[cwe][severity] += 1

        print("[Vendor Risk Analysis] Calculating 30-day CWE counts...")
        cwe_30_day_counts = get_cwe_30_day_counts()

        top_cwes = cwe_counts.most_common(20) if cwe_counts else []
        top_5_cwes = top_cwes[:5]
        top_10_cwes = top_cwes[:10]

        result = {
            'top_5_cwes': {
                'indices': [cwe for cwe, count in top_5_cwes],
                'labels': [get_cwe_title(cwe) for cwe, count in top_5_cwes],
                'values': [count for cwe, count in top_5_cwes]
            },
            'top_10_cwes': {
                'indices': [cwe for cwe, count in top_10_cwes],
                'labels': [get_cwe_title(cwe) for cwe, count in top_10_cwes],
                'values': [count for cwe, count in top_10_cwes]
            },
            'all_cwes': {
                'indices': [cwe for cwe, count in top_cwes],
                'labels': [get_cwe_title(cwe) for cwe, count in top_cwes],
                'values': [count for cwe, count in top_cwes]
            },
            'severity_matrix': {
                cwe: dict(severity_counts) for cwe, severity_counts in cwe_severity_matrix.items()
            },
            'cwe_30_day_counts': cwe_30_day_counts or {},
            'total_cves_analyzed': total_cves,
            'years_analyzed': years_to_analyze
        }
```

```

}

print(f"[Vendor Risk Analysis] Analyzed {result['total_cves_analyzed']} CVEs, found {len(top_cwes)} unique CWEs")
print(f"[Vendor Risk Analysis] Top 5 CWEs: {[cwe for cwe, count in top_5_cwes]}")

return result

except Exception as e:
print(f"[Vendor Risk Analysis] Error in analysis: {e}")
return {
'top_5_cwes': {'indices': [], 'labels': [], 'values': []},
'top_10_cwes': {'indices': [], 'labels': [], 'values': []},
'all_cwes': {'indices': [], 'labels': [], 'values': []},
'severity_matrix': {},
'cwe_30_day_counts': {},
'total_cves_analyzed': 0,
'years_analyzed': []
}

def get_cwe_30_day_counts() -> Dict[str, int]:
"""Get CWE counts for last 30 days from API data"""
try:
# Get recent vulnerability data for current threat assessment
from services.data.api_client import api_client
api_cves = api_client.get_cves_last_30_days()

# Count CWE frequency in recent data
cwe_30_day_counts = Counter()
for cve in api_cves:
cwe = cve.get('CWE')

# Validate CWE format before counting
if cwe and cwe.startswith('CWE'):
cwe_30_day_counts[cwe] += 1

print(f"[Vendor Risk Analysis] Found {len(cwe_30_day_counts)} unique CWEs in last 30 days")

# Convert to standard dict for JSON compatibility
return dict(cwe_30_day_counts)

except Exception as e:
# Return empty dict to allow continued operation when API unavailable
print(f"[Vendor Risk Analysis] Error getting 30-day CWE counts: {e}")
return {}

def get_weighted_cwe_analysis() -> Dict[str, Any]:
"""Get weighted CWE analysis (severity-weighted scores) - LAZY LOADING"""
print("[Vendor Risk Analysis] Calculating weighted CWE scores...")

try:
# Use same temporal scope as vendor risk analysis for consistency
current_year = datetime.now().year
years_to_analyze = [current_year - 1, current_year]

# Load historical data for weighted analysis - ONE YEAR AT A TIME
from services.data.data_processor import historical_loader

# Define severity weights for risk scoring (higher = more critical)
severity_weights = {
'CRITICAL': 4, # Highest priority
'HIGH': 3, # High priority
'MEDIUM': 2, # Medium priority
'LOW': 1, # Low priority
'UNKNOWN': 1 # Default weight for unclassified
}

# Calculate weighted scores for each CWE
cwe_weighted_scores = Counter()

for year in years_to_analyze:
year_cves = historical_loader.get_year_data(year) # Lazy load

```

```

for cve in year_cves:
    cwe = cve.get('CWE')

    # Process valid CWE identifiers
    if cwe and cwe.startswith('CWE'):
        severity = cve.get('Severity', 'UNKNOWN').upper()

    # Apply severity weight to CWE score
    weight = severity_weights.get(severity, 1)
    cwe_weighted_scores[cwe] += weight

    # Get top 10 highest weighted CWEs for strategic focus
    top_weighted = cwe_weighted_scores.most_common(10) if cwe_weighted_scores else []

    # Structure result for visualization and analysis
    result = {
        'indices': [cwe for cwe, score in top_weighted],
        'labels': [get_cwe_title(cwe) for cwe, score in top_weighted],
        'values': [score for cwe, score in top_weighted]
    }

    print(f"[Vendor Risk Analysis] Weighted analysis complete: {len(top_weighted)} CWEs")
    return result

except Exception as e:
    # Return empty structure for graceful error handling
    print(f"[Vendor Risk Analysis] Error in weighted analysis: {e}")
    return {'indices': [], 'labels': [], 'values': []}

```

services\analysis\cwe\catalog.py

```
"""
CWE Catalog Management
Handles loading and parsing CWE catalog from XML files
"""
# Operating system interface for file operations
import os
# XML parsing library for CWE catalog processing
import xml.etree.ElementTree as ET
# Type hints for return value structures
from typing import Dict, Any
# Application configuration for data directory paths
import config

class CWECatalogLoader:
    """Loads and manages CWE catalog data from XML"""

    def __init__(self):
        # Construct path to CWE catalog XML file using configuration
        self.catalog_path = os.path.join(config.DATA_DIR, 'CWE_Catalog.xml')
        # Initialize cache as None for lazy loading pattern
        self._catalog_cache = None

    def load_cwe_catalog(self) -> Dict[str, Dict[str, Any]]:
        """Load full CWE catalog from XML file"""
        # Return cached catalog if already loaded (performance optimization)
        if self._catalog_cache is not None:
            return self._catalog_cache

        # Initialize empty catalog for fallback scenarios
        catalog = {}

        # Check if catalog file exists before attempting to parse
        if not os.path.exists(self.catalog_path):
            print(f"[CWE Catalog] CWE catalog not found at {self.catalog_path}")
            self._catalog_cache = catalog
            return catalog

        try:
            print(f"[CWE Catalog] Loading CWE catalog from {self.catalog_path}")
            # Parse XML catalog file using ElementTree
            tree = ET.parse(self.catalog_path)
            root = tree.getroot()

            # Extract XML namespace for proper element querying
            namespace = self._extract_namespace(root)

            # Find all weakness entries in the XML catalog
            for weakness in root.findall(f'.//{namespace}Weakness'):
                # Parse individual weakness element into structured data
                cwe_entry = self._parse_weakness_element(weakness, namespace)
                if cwe_entry:
                    # Add parsed CWE to catalog using code as key
                    catalog[cwe_entry['code']] = cwe_entry

            print(f"[CWE Catalog] Loaded {len(catalog)} CWEs from catalog")

        except Exception as e:
            # Log error but continue with empty catalog for graceful degradation
            print(f"[CWE Catalog] Error loading CWE catalog: {e}")

            # Cache parsed catalog for future requests
            self._catalog_cache = catalog
            return catalog

    def _extract_namespace(self, root) -> str:
        """Extract namespace from XML root element"""
        # Check if root tag contains namespace information
        if root.tag.startswith('{'):
            # Extract namespace including closing brace for XPath compatibility
            return root.tag.split('}')[0] + '}'
        # Return empty string if no namespace present
        return ''
```

```

def _parse_weakness_element(self, weakness, namespace: str) -> Dict[str, Any]:
    """Parse a single weakness XML element"""
    # Extract CWE ID from XML element attributes
    cwe_id = weakness.get('ID')
    if not cwe_id:
        return None

    # Construct standard CWE code format
    cwe_code = f"CWE-{cwe_id}"
    # Get weakness name or use code as fallback
    name = weakness.get('Name', cwe_code)

    # Extract various types of weakness information using specialized methods
    description = self._extract_description(weakness, namespace)
    mitigations = self._extract_mitigations(weakness, namespace)
    examples = self._extract_examples(weakness, namespace)
    relationships = self._extract_relationships(weakness, namespace)

    # Build structured weakness data with fallback values
    return {
        'code': cwe_code,
        'name': name,
        'description': description or f"CWE-{cwe_id} weakness type",
        'mitigations': mitigations or ['Contact security team for specific mitigation strategies'],
        'examples': examples or [],
        'relationships': relationships or []
    }

def _extract_description(self, weakness, namespace: str) -> str:
    """Extract description from weakness element"""
    # Look for primary description element
    desc_elem = weakness.find(f'://{namespace}Description')
    if desc_elem is not None and desc_elem.text:
        return desc_elem.text.strip()

    # Try alternative extended description if primary not found
    extended_desc = weakness.find(f'://{namespace}Extended_Description')
    if extended_desc is not None and extended_desc.text:
        return extended_desc.text.strip()

    # Return empty string if no description found
    return ""

def _extract_mitigations(self, weakness, namespace: str) -> list:
    """Extract mitigation strategies from weakness element"""
    mitigations = []

    # Find all mitigation elements in weakness
    for mitigation_elem in weakness.findall(f'://{namespace}Mitigation'):
        # Get development phase information if available
        phase = mitigation_elem.get('Phase', '')
        # Extract mitigation description text
        description_elem = mitigation_elem.find(f'{namespace}Description')
        if description_elem is not None and description_elem.text:
            mitigation_text = description_elem.text.strip()
            # Add phase information to mitigation text for context
            if phase:
                mitigation_text = f"[{phase}] {mitigation_text}"
            mitigations.append(mitigation_text)

    return mitigations

def _extract_examples(self, weakness, namespace: str) -> list:
    """Extract examples from weakness element"""
    examples = []

    # Look for demonstrative examples in weakness
    for example_elem in weakness.findall(f'://{namespace}Demonstrative_Example'):
        # Extract introductory text from examples
        intro_elem = example_elem.find(f'{namespace}Intro_Text')
        if intro_elem is not None and intro_elem.text:
            examples.append(intro_elem.text.strip())

    return examples

```

```

def _extract_relationships(self, weakness, namespace: str) -> list:
    """Extract relationships from weakness element"""
    relationships = []

    # Find related weakness elements
    for relation_elem in weakness.findall(f'://{namespace}Related_Weakness'):
        # Get relationship nature and target CWE ID
        nature = relation_elem.get('Nature', '')
        cwe_id = relation_elem.get('CWE_ID', '')
        if nature and cwe_id:
            # Build relationship tuple with standard CWE format
            relationships.append([f"CWE-{cwe_id}", nature])

    return relationships

def get_cwe_by_code(self, cwe_code: str) -> Dict[str, Any]:
    """Get specific CWE by code"""
    # Load catalog to ensure data availability
    catalog = self.load_cwe_catalog()
    # Return found CWE or fallback structure for missing CWEs
    return catalog.get(cwe_code, {
        'code': cwe_code,
        'name': f"CWE {cwe_code}",
        'description': f"Detailed information for {cwe_code} would be fetched from MITRE database.",
        'mitigations': ["Contact security team for specific mitigation strategies"],
        'examples': ["Examples would be loaded from MITRE database"],
        'relationships': []
    })

def search_cwes_by_keyword(self, keyword: str) -> list:
    """Search CWEs by keyword in name or description"""
    # Load full catalog for comprehensive searching
    catalog = self.load_cwe_catalog()
    # Convert keyword to lowercase for case-insensitive matching
    keyword_lower = keyword.lower()

    matches = []
    # Search through all CWEs in catalog
    for cwe_code, cwe_data in catalog.items():
        # Check if keyword appears in name or description (case-insensitive)
        if (keyword_lower in cwe_data['name'].lower() or
            keyword_lower in cwe_data['description'].lower()):
            # Add matching CWE as tuple for easy unpacking
            matches.append((cwe_code, cwe_data))

    return matches

```


services\analysis\cwe\constants.py

```
"""
CWE Constants
Centralized constants and mappings for CWE analysis
"""
# Primary CWE titles mapping for core vulnerability weaknesses
# Maps CWE codes to human-readable descriptions for educational content
CWE_TITLES = {
"CWE79": "Cross-Site Scripting", # Web application XSS vulnerabilities
"CWE89": "SQL Injection", # Database injection attacks
"CWE20": "Improper Input Validation", # Input validation failures
"CWE22": "Path Traversal", # Directory traversal attacks
"CWE119": "Buffer Overflow", # Memory buffer overflow errors
"CWE200": "Information Exposure", # Unintended information disclosure
"CWE287": "Improper Authentication", # Authentication bypass vulnerabilities
"CWE120": "Buffer Copy without Checking Size", # Unchecked buffer operations
"CWE264": "Permissions, Privileges, and Access Controls", # Access control issues
"CWE193": "Off-by-one Error", # Array boundary errors
"CWE19": "Data Processing Errors", # General data handling issues
"CWE178": "Improper Handling of Case Sensitivity", # Case sensitivity bugs
"CWE17": "Code", # General code quality issues
"CWE276": "Incorrect Default Permissions", # Permission configuration errors
"CWE255": "Credentials Management", # Credential handling issues
"CWE327": "Use of a Broken or Risky Cryptographic Algorithm", # Weak cryptography
"CWE307": "Improper Restriction of Excessive Authentication Attempts", # Brute force
protection
"CWE384": "Session Fixation", # Session management vulnerabilities
"CWE88": "Argument Injection or Modification", # Command argument manipulation
"CWE78": "OS Command Injection", # Operating system command injection
"CWE59": "Improper Link Resolution Before File Access", # Symlink vulnerabilities
"CWE94": "Code Injection", # Dynamic code execution vulnerabilities
"CWE352": "Cross-Site Request Forgery", # CSRF attack vulnerabilities
"CWE434": "Unrestricted Upload", # File upload security issues
"CWE476": "NULL Pointer Dereference", # Memory access errors
"CWE862": "Missing Authorization", # Authorization bypass issues
"CWE74": "Improper Neutralization", # Input sanitization failures
"NVD-CWE-Other": "Other/Unclassified", # NVD catchall category
"NVD-CWE-noinfo": "Unclassified", # NVD unclassified vulnerabilities
"Unclassified": "Unknown" # Generic unknown classification
}

# Extended CWE titles including additional weaknesses for comprehensive analysis
# Merges base titles with specialized categories for advanced vulnerability analysis
EXTENDED_CWE_TITLES = {
**CWE_TITLES, # Include all base CWE titles
# Memory safety and buffer management weaknesses
"CWE125": "Out-of-bounds Read", # Memory read beyond boundaries
"CWE787": "Out-of-bounds Write", # Memory write beyond boundaries
"CWE416": "Use After Free", # Dangling pointer vulnerabilities
"CWE190": "Integer Overflow", # Numeric overflow conditions
"CWE191": "Integer Underflow", # Numeric underflow conditions
"CWE369": "Divide by Zero", # Division by zero errors
"CWE617": "Reachable Assertion", # Assertion failures in production
# Authentication and credential management
"CWE798": "Use of Hard-coded Credentials", # Embedded credentials in code
"CWE732": "Incorrect Permission Assignment", # File system permission errors
# Resource management and denial of service
"CWE770": "Allocation of Resources Without Limits", # Resource exhaustion
"CWE772": "Missing Release of Resource", # Resource leak vulnerabilities
"CWE775": "Missing Release of File Descriptor", # File descriptor leaks
"CWE401": "Missing Release of Memory", # Memory leak vulnerabilities
"CWE404": "Improper Resource Shutdown", # Resource cleanup failures
# Cryptographic and certificate management
"CWE295": "Improper Certificate Validation", # SSL/TLS certificate validation
"CWE326": "Inadequate Encryption Strength", # Weak encryption algorithms
"CWE347": "Improper Verification of Cryptographic Signature", # Signature validation
# Password and authentication strength
"CWE521": "Weak Password Requirements", # Insufficient password policies
"CWE522": "Insufficiently Protected Credentials", # Credential storage issues
"CWE916": "Use of Password Hash with Insufficient Computational Effort" # Weak hashing
}

# Severity weights for weighted CWE analysis and risk scoring
```

```

# Higher weights indicate higher priority for security response
SEVERITY_WEIGHTS = {
'CRITICAL': 4, # Highest priority - immediate action required
'HIGH': 3, # High priority - urgent remediation needed
'MEDIUM': 2, # Medium priority - timely remediation recommended
'LOW': 1, # Low priority - remediation as resources allow
'UNKNOWN': 1 # Unknown severity - treated as low priority for safety
}

# CWE categories for grouping related weaknesses by attack vector
# Supports categorical analysis and educational organization
CWE_CATEGORIES = {
# Injection attack vulnerabilities
'injection': ['CWE79', 'CWE89', 'CWE78', 'CWE94', 'CWE88'],
# Authentication and session management issues
'authentication': ['CWE287', 'CWE307', 'CWE384', 'CWE521', 'CWE522'],
# Authorization and access control problems
'authorization': ['CWE862', 'CWE264', 'CWE276', 'CWE732'],
# Input validation and sanitization failures
'validation': ['CWE20', 'CWE22', 'CWE178'],
# Buffer overflow and memory safety issues
'buffer_errors': ['CWE119', 'CWE120', 'CWE125', 'CWE787'],
# Memory management and resource handling
'memory_management': ['CWE416', 'CWE401', 'CWE770', 'CWE772'],
# Cryptographic implementation weaknesses
'cryptographic': ['CWE327', 'CWE295', 'CWE326', 'CWE347'],
# Information disclosure vulnerabilities
'information_exposure': ['CWE200', 'CWE255', 'CWE798'],
# Resource management and denial of service
'resource_management': ['CWE404', 'CWE775', 'CWE369']
}

# Top 25 most dangerous CWEs based on OWASP/CWE research
# Industry-standard prioritization for vulnerability assessment and education
TOP_25_CWES = [
'CWE787', # Out-of-bounds Write
'CWE79', # Cross-site Scripting
'CWE125', # Out-of-bounds Read
'CWE20', # Improper Input Validation
'CWE78', # OS Command Injection
'CWE89', # SQL Injection
'CWE416', # Use After Free
'CWE22', # Path Traversal
'CWE352', # Cross-Site Request Forgery
'CWE434', # Unrestricted Upload of File with Dangerous Type
'CWE476', # NULL Pointer Dereference
'CWE287', # Improper Authentication
'CWE190', # Integer Overflow or Wraparound
'CWE502', # Deserialization of Untrusted Data
'CWE77', # Command Injection
'CWE119', # Improper Restriction of Operations within Buffer
'CWE798', # Use of Hard-coded Credentials
'CWE862', # Missing Authorization
'CWE276', # Incorrect Default Permissions
'CWE200', # Information Exposure
'CWE522', # Insufficiently Protected Credentials
'CWE732', # Incorrect Permission Assignment for Critical Resource
'CWE611', # Improper Restriction of XML External Entity Reference
'CWE918', # Server-Side Request Forgery (SSRF)
'CWE77' # Command Injection (duplicate in original - maintaining for compatibility)
]

```

services\analysis\cwe\processor.py

```
"""
CWE Severity Processor
Main processor for CWE severity analysis and data aggregation
"""

# Efficient counting and nested dictionary structures for data analysis
from collections import Counter, defaultdict
# Type hints for function parameters and return values
from typing import Dict, List, Any
# Date operations for temporal analysis
from datetime import datetime
# Random number generation (imported but not currently used)
import random

# CWE catalog management for detailed weakness information
from .catalog import CWECatalogLoader
# Centralized CWE title mappings for consistent labeling
from .constants import CWE_TITLES

class CWESeverityProcessor:
    """Process CWE data for severity analysis"""

    def __init__(self):
        # Store reference to CWE titles for consistent labeling
        self.cwe_titles = CWE_TITLES
        # Initialize catalog loader for detailed CWE information
        self.catalog_loader = CWECatalogLoader()

    def get_cwe_severity_data(self, top_n: int = 10) -> Dict[str, Any]:
        """Get CWE severity analysis data - LAZY LOADING"""
        print(f"[CWE Processor] Analyzing CWE severity data (top {top_n})")

        # Collect vulnerability data from all available sources
        all_cves = self._collect_cve_data()

        # Return empty structure if no data available for analysis
        if not all_cves:
            print(f"[CWE Processor] No data found, returning empty structure")
            return self._get_empty_cwe_data(top_n)

        # Analyze CWE frequency and severity distributions
        cwe_severity_matrix, cwe_totals = self._analyze_cwe_severity(all_cves)

        # Get top N CWEs by total occurrence count
        top_cwes = cwe_totals.most_common(top_n)

        # Build chart-ready data structure for visualization
        result = self._build_chart_data(top_cwes, cwe_severity_matrix, len(all_cves))

        print(f"[CWE Processor] Processed {len(all_cves)} CVEs, found {len(top_cwes)} CWEs")
        print(f"[CWE Processor] Top CWEs: {[cwe for cwe, count in top_cwes]}")

        return result

    def _collect_cve_data(self) -> List[Dict]:
        """Collect CVE data from all sources - LAZY LOADING"""
        all_cves = []

        # Use lazy imports to avoid circular import issues
        try:
            from services.data.api_client import api_client
            from services.data.data_processor import historical_loader

            # Get recent data from API (last 30 days)
            try:
                api_cves = api_client.get_cves_last_30_days()
                all_cves.extend(api_cves)
            except Exception as e:
                print(f"[CWE Processor] Got {len(api_cves)} API CVEs")
            except Exception as e:
                print(f"[CWE Processor] Error getting API CVEs: {e}")
```

```

# Get last 2 years of historical data for trend analysis - ONE YEAR AT A TIME
try:
    current_year = datetime.now().year

# Load years separately to avoid memory issues
for year in [current_year - 1, current_year]:
    year_data = historical_loader.get_year_data(year) # Lazy load
    all_cves.extend(year_data)
    print(f"[CWE Processor] Got {len(year_data)} CVEs for {year}")

except Exception as e:
    print(f"[CWE Processor] Error getting historical CVEs: {e}")

except Exception as e:
    print(f"[CWE Processor] Import error: {e}")

print(f"[CWE Processor] Total CVEs to analyze: {len(all_cves)}")
return all_cves

def _analyze_cwe_severity(self, all_cves: List[Dict]) -> tuple:
    """Analyze CWE severity distribution"""
    # Nested Counter for severity distribution per CWE
    cwe_severity_matrix = defaultdict(lambda: Counter())
    # Overall CWE frequency counter
    cwe_totals = Counter()

    # Process each CVE for CWE and severity information
    for cve in all_cves:
        cwe = cve.get('CWE')

    # Validate CWE format before processing
    if cwe and cwe.startswith('CWE'):
        severity = cve.get('Severity', 'UNKNOWN').upper()

    # Count valid severities and track in matrix
    if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
        cwe_severity_matrix[cwe][severity] += 1
        cwe_totals[cwe] += 1
    else:
        # Handle unknown or invalid severities
        cwe_severity_matrix[cwe]['UNKNOWN'] += 1
        cwe_totals[cwe] += 1

    return cwe_severity_matrix, cwe_totals

def _build_chart_data(self, top_cwes: List[tuple], cwe_severity_matrix: dict,
total_cves: int) -> Dict[str, Any]:
    """Build chart data structure"""
    # Initialize parallel arrays for chart consumption
    labels = [] # Human-readable CWE titles
    indices = [] # CWE codes for reference
    data = { # Severity counts per CWE
        'CRITICAL': [],
        'HIGH': [],
        'MEDIUM': [],
        'LOW': [],
        'UNKNOWN': []
    }

    # Process each top CWE for chart data
    for cwe_code, total_count in top_cwes:
        # Resolve CWE code to human-readable title
        title = self.cwe_titles.get(cwe_code, cwe_code)
        labels.append(title)
        indices.append(cwe_code)

    # Extract severity breakdown for this CWE
    severity_counts = cwe_severity_matrix[cwe_code]
    data['CRITICAL'].append(severity_counts.get('CRITICAL', 0))
    data['HIGH'].append(severity_counts.get('HIGH', 0))
    data['MEDIUM'].append(severity_counts.get('MEDIUM', 0))
    data['LOW'].append(severity_counts.get('LOW', 0))
    data['UNKNOWN'].append(severity_counts.get('UNKNOWN', 0))

    # Return chart-ready data structure
    return {
        'labels': labels, # For chart axis labels

```

```

'indices': indices, # For data reference
'data': data, # For chart series data
'total_cves': total_cves # For context information
}

def _get_empty_cwe_data(self, top_n: int = 10) -> Dict[str, Any]:
    """Return empty data structure when no data is available"""
    return {
        'labels': [],
        'indices': [],
        'data': {
            'CRITICAL': [],
            'HIGH': [],
            'MEDIUM': [],
            'LOW': [],
            'UNKNOWN': []
        },
        'total_cves': 0
    }

def get_cwe_details(self) -> Dict[str, Dict[str, Any]]:
    """Get CWE details for the explorer"""
    # Start with key CWES and provide loading placeholders
    cwe_details = {}
    for cwe_code, title in self.cwe_titles.items():
        cwe_details[cwe_code] = {
            'name': title,
            'description': f'Click to fetch detailed information about {cwe_code}',
            'mitigations': ['Loading detailed information...'], # Loading indicator
            'examples': [],
            'relationships': []
        }

    # Add detailed CWES from XML catalog without overwriting key CWES
    catalog = self.catalog_loader.load_cwe_catalog()
    for cwe_code, cwe_data in catalog.items():
        if cwe_code not in cwe_details:
            cwe_details[cwe_code] = cwe_data

    return cwe_details

def get_key_cwes(self) -> List[str]:
    """Get list of key CWE codes"""
    # Return list of curated CWE identifiers
    return list(self.cwe_titles.keys())

def get_key_cwe_titles(self) -> Dict[str, str]:
    """Get key CWE titles mapping"""
    # Return copy of title mapping for external use
    return self.cwe_titles.copy()

def get_cwe_by_code(self, cwe_code: str) -> Dict[str, Any]:
    """Get CWE details by code"""
    # Delegate to catalog loader for individual CWE lookup
    return self.catalog_loader.get_cwe_by_code(cwe_code)

```

services\analysis\cwe\utils.py

```
"""
CWE Utility Functions
Helper functions for CWE processing and analysis
"""
# Type hints for function parameters and return values
from typing import Dict, List, Tuple, Optional
# Centralized CWE constants and category mappings
from .constants import CWE_TITLES, EXTENDED_CWE_TITLES, CWE_CATEGORIES

def get_cwe_title(cwe_code: str) -> str:
    """Get human-readable title for CWE code"""
    # Cascading lookup: primary titles, then extended, then fallback to code
    return (CWE_TITLES.get(cwe_code) or
            EXTENDED_CWE_TITLES.get(cwe_code) or
            cwe_code)

def normalize_cwe_code(cwe_input: str) -> str:
    """Normalize CWE input to standard format (CWE-123)"""
    # Handle empty or None input
    if not cwe_input:
        return ""

    # Clean input: remove whitespace and standardize case
    cwe_clean = cwe_input.strip().upper()

    # Handle different input formats and convert to standard CWE-123
    if cwe_clean.startswith('CWE-'):
        # Already in standard format
        return cwe_clean
    elif cwe_clean.startswith('CWE'):
        # Missing hyphen: CWE123 -> CWE-123
        number_part = cwe_clean[3:]
        if number_part.isdigit():
            return f"CWE-{number_part}"
    elif cwe_clean.isdigit():
        # Just number: 123 -> CWE-123
        return f"CWE-{cwe_clean}"

    # Return original if normalization not possible
    return cwe_input

def categorize_cwe(cwe_code: str) -> Optional[str]:
    """Categorize a CWE code into a broader category"""
    # Search through all defined categories for CWE membership
    for category, cwes in CWE_CATEGORIES.items():
        if cwe_code in cwes:
            return category
    # Return None if CWE doesn't match any category
    return None

def get_cwe_category_summary(cves: List[Dict]) -> Dict[str, int]:
    """Get summary of CVEs by CWE category"""
    category_counts = {}

    # Process each CVE for category classification
    for cve in cves:
        cwe = cve.get('CWE')
        if cwe:
            category = categorize_cwe(cwe)
            if category:
                # Count categorized CWES
                category_counts[category] = category_counts.get(category, 0) + 1
            else:
                # Count uncategorized CWES as 'other'
                category_counts['other'] = category_counts.get('other', 0) + 1

    return category_counts

def extract_cwe_number(cwe_code: str) -> Optional[int]:
    """Extract numeric part from CWE code"""
    try:
        # Handle different CWE formats to extract numeric ID

```

```

if cwe_code.startswith('CWE-'):
    return int(cwe_code[4:]) # CWE-123 -> 123
elif cwe_code.startswith('CWE'):
    return int(cwe_code[3:]) # CWE123 -> 123
elif cwe_code.isdigit():
    return int(cwe_code) # 123 -> 123
except (ValueError, AttributeError):
    # Return None for invalid input or conversion errors
    pass
return None

def is_valid_cwe_code(cwe_code: str) -> bool:
    """Check if a string is a valid CWE code format"""
    # Reject empty or None input
    if not cwe_code:
        return False

    # Use normalization to standardize format, then validate
    normalized = normalize_cwe_code(cwe_code)
    return (normalized.startswith('CWE-') and
            normalized[4:].isdigit() and
            len(normalized) > 4)

def get_severity_color(severity: str) -> str:
    """Get color code for severity level"""
    # Standard color mapping for security severity levels
    severity_colors = {
        'CRITICAL': '#f55855', # Red for highest severity
        'HIGH': '#f8a541', # Orange for high severity
        'MEDIUM': '#3b8ded', # Blue for medium severity
        'LOW': '#42d392', # Green for low severity
        'UNKNOWN': '#6b7280' # Gray for unknown/unclassified
    }
    # Case-insensitive lookup with gray fallback
    return severity_colors.get(severity.upper(), '#6b7280')

def format_cwe_for_display(cwe_code: str, include_title: bool = True) -> str:
    """Format CWE code for display purposes"""
    # Handle missing CWE data
    if not cwe_code:
        return "No CWE"

    # Standardize CWE format
    normalized = normalize_cwe_code(cwe_code)

    # Optionally include human-readable title
    if include_title:
        title = get_cwe_title(normalized)
        # Only add title if it's different from the code
        if title != normalized:
            return f"{normalized}: {title}"

    return normalized

def search_cwes_by_keyword(keyword: str) -> List[Tuple[str, str]]:
    """Search CWEs by keyword in titles"""
    keyword_lower = keyword.lower()
    matches = []

    # Search through extended CWE titles for keyword matches
    for cwe_code, title in EXTENDED_CWE_TITLES.items():
        # Case-insensitive search in both code and title
        if (keyword_lower in cwe_code.lower() or
            keyword_lower in title.lower()):
            matches.append((cwe_code, title))

    # Sort results by CWE code for consistent ordering
    return sorted(matches, key=lambda x: x[0])

def get_cwe_statistics(cves: List[Dict]) -> Dict[str, any]:
    """Get comprehensive CWE statistics from CVE list"""
    # Initialize comprehensive statistics structure
    stats = {
        'total_cves': len(cves),
        'cves_with_cwe': 0,
        'unique_cwes': set(), # Use set for automatic deduplication
        'top_cwes': {},
    }

```

```

'category_breakdown': {},
'severity_by_cwe': {}
}

cwe_counts = {}

# Process each CVE for comprehensive CWE analysis
for cve in cves:
    cwe = cve.get('CWE')
    severity = cve.get('Severity', 'UNKNOWN').upper()

    # Only process CVEs with valid CWE codes
    if cwe and is_valid_cwe_code(cwe):
        stats['cves_with_cwe'] += 1
        stats['unique_cwes'].add(cwe) # Set automatically handles duplicates

    # Count CWE occurrences for frequency analysis
    cwe_counts[cwe] = cwe_counts.get(cwe, 0) + 1

    # Track severity distribution per CWE for detailed analysis
    if cwe not in stats['severity_by_cwe']:
        stats['severity_by_cwe'][cwe] = {}
    stats['severity_by_cwe'][cwe][severity] = stats['severity_by_cwe'][cwe].get(severity, 0) + 1

# Generate top 10 most frequent CWES
stats['top_cwes'] = dict(sorted(cwe_counts.items(), key=lambda x: x[1],
reverse=True)[:10])

# Generate category breakdown using utility function
stats['category_breakdown'] = get_cwe_category_summary(cves)

# Convert unique CWES set to count for JSON serialization
stats['unique_cwes'] = len(stats['unique_cwes'])

return stats

```


services\analysis\cwe__init__.py

```
"""
CWE Analysis Module
Modular CWE processing and analysis functionality
"""

# Import core CWE processing class for severity analysis and data aggregation
from .processor import CWESeverityProcessor
# Import advanced analysis functions for risk assessment and trend analysis
from .analyzer import get_vendor_risk_analysis, get_weighted_cwe_analysis,
get_cwe_30_day_counts
# Import CWE catalog management class for loading weakness definitions
from .catalog import CWECatalogLoader
# Import utility functions for CWE title resolution and helper operations
from .utils import get_cwe_title

# Create global processor instance for backward compatibility and immediate use
cwe_processor = CWESeverityProcessor()

# Export main functions and classes for controlled public API
# Includes both legacy global instance and modern class-based interface
__all__ = [
    'cwe_processor',          # Global instance for backward compatibility
    'CWESeverityProcessor',   # Class for custom processor instances
    'CWECatalogLoader',      # Class for CWE catalog management
    'get_vendor_risk_analysis', # Function for vendor risk assessment
    'get_weighted_cwe_analysis', # Function for weighted CWE analysis
    'get_cwe_30_day_counts',  # Function for temporal trend analysis
    'get_cwe_title'          # Utility function for CWE title resolution
]
```

services\cache\cache_manager.py

```
import threading
from datetime import datetime, timezone, timedelta
from typing import Dict, List, Any, Optional

class CacheManager:
    """Lightweight caching system with reduced memory footprint"""

    def __init__(self):
        # Only cache metadata, not actual CVE data
        self._cache_timestamps = {}
        self._historical_loaded = False
        self._lock = threading.Lock()

        # Cache duration configuration (in seconds)
        self.API_CACHE_DURATION = 24 * 60 * 60 # 24 hours
        self.HISTORICAL_CACHE_DURATION = 7 * 24 * 60 * 60 # 7 days

    def get_api_data_30_days(self) -> Optional[List[Dict[str, Any]]]:
        """Check if API data cache is valid - returns None, DB should be checked"""
        cache_key = "api_30_days"
        with self._lock:
            if cache_key in self._cache_timestamps:
                timestamp = self._cache_timestamps.get(cache_key, 0)
                age = datetime.now().timestamp() - timestamp
                if age < self.API_CACHE_DURATION:
                    print(f"[Cache] API cache still valid (age: {int(age/60)} minutes)")
                    return True # Signal to use DB
                return None

    def set_api_data_30_days(self, data: List[Dict[str, Any]]):
        """Set cache timestamp for API data"""
        cache_key = "api_30_days"
        with self._lock:
            self._cache_timestamps[cache_key] = datetime.now().timestamp()
            print(f"[Cache] Set timestamp for API data ({len(data)} CVEs)")

    def get_historical_data(self) -> Optional[Dict[str, List[Dict]]]:
        """Check if historical data is loaded"""
        if self._historical_loaded:
            print("[Cache] Historical data marked as loaded")
            return True
        return None

    def set_historical_data(self, data: Dict[str, List[Dict]]):
        """Mark historical data as loaded"""
        with self._lock:
            self._historical_loaded = True
            total_cves = sum(len(year_data) for year_data in data.values())
            print(f"[Cache] Marked historical data as loaded ({total_cves} CVEs)")

    def get_cache_stats(self) -> Dict[str, Any]:
        """Get cache statistics"""
        with self._lock:
            stats = {
                'cached_timestamps': len(self._cache_timestamps),
                'historical_loaded': self._historical_loaded,
                'cache_ages': {}
            }

        current_time = datetime.now().timestamp()
        for key, timestamp in self._cache_timestamps.items():
            age_minutes = int((current_time - timestamp) / 60)
            stats['cache_ages'][key] = f"{age_minutes} minutes"

        return stats

    def clear_cache(self):
        """Clear all cached data"""
        with self._lock:
            self._cache_timestamps.clear()
            self._historical_loaded = False
            print("[Cache] All caches cleared")
```

```
def warm_up_cache(self):
    """Warm up cache - does nothing now since we use DB"""
    print("[Cache] Cache warm-up skipped (using database)")

# Global cache instance
cache_manager = CacheManager()
```

services\data\api_client.py

```
import requests
import json
import time
from datetime import datetime, timezone, timedelta
from typing import List, Dict, Any, Optional
import config
import threading
import logging

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class NVDApiClient:
    """Enhanced NVD API client with database support and aggressive memory optimization"""

    def __init__(self):
        self.base_url = config.NVD_API_URL
        self.api_key = config.NVD_API_KEY
        self.timeout = config.API_TIMEOUT
        self.last_request_time = 0

    # Proper rate limiting based on API key availability
    if self.api_key:
        self.min_request_interval = 30 / 50 # 50 requests per 30 seconds with key
    else:
        self.min_request_interval = 30 / 5 # 5 requests per 30 seconds without key

    # Session for connection reuse
    self.session = requests.Session()
    if self.api_key:
        self.session.headers.update({"apiKey": self.api_key})

    # Lightweight cache for metadata only
    self._cache_timestamps = {}
    self._cache_lock = threading.Lock()

    # Database integration
    from database import db_manager
    self.db = db_manager

    def _rate_limit(self):
        """Proper rate limiting with logging"""
        elapsed = time.time() - self.last_request_time
        if elapsed < self.min_request_interval:
            sleep_time = self.min_request_interval - elapsed
            time.sleep(sleep_time)
            self.last_request_time = time.time()

    def _is_cache_valid(self, cache_key: str, max_age: int = 900) -> bool:
        """Check if cache entry is still valid (default 15 minutes)"""
        with self._cache_lock:
            if cache_key not in self._cache_timestamps:
                return False
            age = time.time() - self._cache_timestamps[cache_key]
            return age < max_age

    def _set_cache_timestamp(self, cache_key: str):
        """Set cache timestamp"""
        with self._cache_lock:
            self._cache_timestamps[cache_key] = time.time()

    def get_cves_last_30_days(self) -> List[Dict[str, Any]]:
        """Get CVEs from last 30 days - OPTIMIZED for memory"""
        cache_key = "cves_30_days"

    # Try database first if available
    if self.db.use_database:
        # Check if we have recent data in database
        metadata = self.db.get_cache_metadata(cache_key)
        if metadata:
            last_updated = metadata.get('last_updated')
```

```

if last_updated:
age = datetime.now(timezone.utc) - last_updated.replace(tzinfo=timezone.utc)
if age.total_seconds() < 900: # 15 minutes
logger.info("Using database cached CVE data")
cves = self.db.get_all_cves()
if cves:
return cves

# Check memory cache timestamp
if self._is_cache_valid(cache_key, max_age=900):
logger.info("Database cache still valid, fetching from DB")
if self.db.use_database:
cves = self.db.get_all_cves()
if cves:
return cves

logger.info("Fetching fresh CVE data from API")

# Calculate date range
end_date = datetime.now(timezone.utc)
start_date = end_date - timedelta(days=30)

all_cves = []
start_index = 0
results_per_page = 2000
max_results = 10000

try:
while start_index < max_results:
params = {
"resultsPerPage": results_per_page,
"startIndex": start_index,
"pubStartDate": start_date.strftime("%Y-%m-%dT%H:%M:%S.000"),
"pubEndDate": end_date.strftime("%Y-%m-%dT%H:%M:%S.000")
}

logger.info(f"API request: startIndex={start_index}, resultsPerPage={results_per_page}")

self._rate_limit()
response = self.session.get(
self.base_url,
params=params,
timeout=self.timeout
)

if response.status_code != 200:
logger.error(f"API request failed: {response.status_code} - {response.text}")
break

data = response.json()
vulnerabilities = data.get("vulnerabilities", [])
total_results = data.get("totalResults", 0)

logger.info(f"Got {len(vulnerabilities)} CVEs (total available: {total_results})")

if not vulnerabilities:
break

# Process CVEs in SMALLER batches to reduce memory
batch = []
for item in vulnerabilities:
cve_data = item.get("cve", {})
processed = self._process_cve_item(cve_data)
if processed:
batch.append(processed)

# Save in smaller batches of 500
if len(batch) >= 500 and self.db.use_database:
self.db.save_cves_batch(batch)
all_cves.extend(batch)
batch = [] # Clear batch immediately
del batch
batch = []

# Save remaining items
if batch:
if self.db.use_database:

```

```

self.db.save_cves_batch(batch)
all_cves.extend(batch)
del batch

# Check if we've got all results or hit our limit
if len(vulnerabilities) < results_per_page or start_index + len(vulnerabilities) >=
total_results:
break

start_index += len(vulnerabilities)

# Safety limit
if start_index >= max_results:
logger.warning(f"Hit safety limit of {max_results} results")
break

# Sort by published date (newest first)
all_cves.sort(key=lambda x: x.get('Published', ''), reverse=True)

# Update cache metadata
if self.db.use_database:
self.db.update_cache_metadata(cache_key, len(all_cves))
self._set_cache_timestamp(cache_key)

logger.info(f"Successfully fetched and cached {len(all_cves)} CVEs")
return all_cves

except requests.exceptions.RequestException as e:
logger.error(f"API request exception: {e}")
# Try database as fallback
if self.db.use_database:
logger.info("Trying database as fallback")
cves = self.db.get_all_cves()
if cves:
return cves
return []
except Exception as e:
logger.error(f"Unexpected error: {e}")
# Try database as fallback
if self.db.use_database:
cves = self.db.get_all_cves()
if cves:
return cves
return []

def _process_cve_item(self, cve_data: Dict) -> Optional[Dict]:
"""Convert NVD CVE format to internal format - MEMORY OPTIMIZED"""
try:
cve_id = cve_data.get("id", "")
if not cve_id:
return None

# Extract description
description = ""
for desc in cve_data.get("descriptions", []):
if desc.get("lang") == "en":
description = desc.get("value", "")
break

# Extract severity and CVSS
severity = "UNKNOWN"
cvss_score = None
metrics = cve_data.get("metrics", {})

# Try different CVSS versions
for version in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2"]:
if version in metrics and metrics[version]:
try:
metric = metrics[version][0]
cvss_data = metric.get("cvssData", {})
potential_severity = cvss_data.get("baseSeverity", "")
potential_score = cvss_data.get("baseScore", None)

if potential_severity and potential_severity.upper() != "UNKNOWN":
severity = potential_severity.upper()
cvss_score = potential_score
break

```

```

except (IndexError, KeyError):
    continue

# Extract CWE
cwe = None
for weakness in cve_data.get("weaknesses", []):
    for desc in weakness.get("description", []):
        if desc.get("lang") == "en":
            cwe_value = desc.get("value")
            if cwe_value and cwe_value.startswith("CWE"):
                cwe = cwe_value
                break
    if cwe:
        break

# Extract references (limit to first 5 to save memory)
references = []
for ref in cve_data.get("references", [])[:5]:
    url = ref.get("url")
    if url:
        references.append(url)

return {
    "ID": cve_id,
    "Description": description or "No description available",
    "Severity": severity,
    "CVSS_Score": cvss_score,
    "CWE": cwe,
    "Published": cve_data.get("published", ""),
    "lastModified": cve_data.get("lastModified", ""),
    "References": references,
    "Products": [], # Skip products to save memory
    "metrics": {} # Skip full metrics to save memory
}
except Exception as e:
    logger.error(f"Error processing CVE: {e}")
    return None

def clear_cache(self):
    """Clear all cached data"""
    with self._cache_lock:
        self._cache_timestamps.clear()

# Clear database cache if available
if self.db.use_database:
    self.db.clear_all_cves()

logger.info("Cache cleared")

def get_cache_stats(self) -> Dict[str, Any]:
    """Get cache statistics"""
    with self._cache_lock:
        stats = {
            'cached_entries': len(self._cache_timestamps),
            'cache_ages': {}
        }

    current_time = time.time()
    for key, timestamp in self._cache_timestamps.items():
        age_minutes = int((current_time - timestamp) / 60)
        stats['cache_ages'][key] = f"{age_minutes} minutes"

# Add database stats if available
if self.db.use_database:
    stats['database'] = self.db.get_stats()

return stats

# Global API client instance
api_client = NVDApiClient()

```

services\data\data_processor.py

```
import os
import json
import gzip
from typing import List, Dict, Any
from datetime import datetime
import config

class DataProcessor:
    """Process and extract insights from historical NVD data - LOCAL FILES ONLY"""

    def __init__(self):
        self.historical_dir = config.NVD_HISTORICAL_DIR
        self.processed_dir = config.NVD_PROCESSED_DIR
        # CRITICAL: In-memory cache for SINGLE year only
        self._year_cache = {}
        self._max_cache_size = 1 # Only cache 1 year at a time

        print(f"[Historical] DataProcessor initialized")
        print(f"[Historical] Local directory: {self.historical_dir}")

        # Check what files are available
        self._check_available_files()

    def _check_available_files(self):
        """Check what historical files are available"""
        if not os.path.exists(self.historical_dir):
            print(f"[Historical] Directory does not exist: {self.historical_dir}")
            print(f"[Historical] Creating directory...")
            os.makedirs(self.historical_dir, exist_ok=True)
            return

        files = os.listdir(self.historical_dir)
        if files:
            print(f"[Historical] Found {len(files)} files: {files}")
        else:
            print(f"[Historical] WARNING: No files found in {self.historical_dir}")
            print(f"[Historical] Please add CVE-XXXX.json or CVE-XXXX.json.gz files to this directory")

    def get_year_data(self, year: int) -> List[Dict]:
        """Load CVEs from a specific year file - WITH CACHING"""
        # Check cache first
        year_str = str(year)
        if year_str in self._year_cache:
            print(f"[Historical] Using cached data for {year}")
            return self._year_cache[year_str]

        # Load from local file
        data = self._load_year_file(year)

        # Cache management - only keep 1 year in memory
        if len(self._year_cache) >= self._max_cache_size:
            # Clear oldest cache entry
            oldest_year = next(iter(self._year_cache))
            print(f"[Historical] Evicting {oldest_year} from cache to save memory")
            del self._year_cache[oldest_year]

        # Store in cache
        self._year_cache[year_str] = data
        print(f"[Historical] Cached {len(data)} CVEs for {year}")

        return data

    def _load_year_file(self, year: int) -> List[Dict]:
        """Load CVEs from a specific year file - LOCAL FILES ONLY

        This method now supports both compressed (.json.gz) and uncompressed (.json) files.
        """
        try:
            # Try different file patterns - INCLUDING GZIPPED FILES
            possible_filenames = [
                f"CVE-{year}.json.gz", # Try gzipped first (most common)
            ]
        except Exception as e:
            print(f"[Historical] Error loading year file: {e}")
            return []
```



```

f"CVE-{{year}}.json",
f"nvdcve-1.1-{{year}}.json.gz",
f"nvdcve-1.1-{{year}}.json",
f"nvdcve-2.0-{{year}}.json.gz",
f"nvdcve-2.0-{{year}}.json",
]

# Check if directory exists
if not os.path.exists(self.historical_dir):
    print(f"[Historical] Directory does not exist: {self.historical_dir}")
    return []

# Find the first available file
target_file = None
filename_used = None
is_gzipped = False

for filename in possible_filenames:
    file_path = os.path.join(self.historical_dir, filename)
    if os.path.exists(file_path):
        target_file = file_path
        filename_used = filename
        is_gzipped = filename.endswith('.gz')
        print(f"[Historical] Found file: {filename} (gzipped={is_gzipped})")
        break

if not target_file:
    print(f"[Historical] No file found for {year}. Tried: {possible_filenames}")
    print(f"[Historical] Available files: {os.listdir(self.historical_dir)}")
    return []

print(f"[Historical] Loading file: {target_file}")

# Check file size
file_size = os.path.getsize(target_file)
print(f"[Historical] File size: {file_size:,} bytes ({file_size/1024/1024:.2f} MB)")

# Load the file - HANDLE BOTH GZIPPED AND UNCOMPRESSED
if is_gzipped:
    print(f"[Historical] Opening gzipped file...")
    with gzip.open(target_file, 'rt', encoding='utf-8') as f:
        data = json.load(f)
    print(f"[Historical] Successfully loaded gzipped JSON file for {year}")
else:
    print(f"[Historical] Opening uncompressed file...")
    with open(target_file, 'r', encoding='utf-8') as f:
        data = json.load(f)
    print(f"[Historical] Successfully loaded JSON file for {year}")

# DEBUG: Print top-level structure
print(f"[Historical] DEBUG: JSON top-level keys: {list(data.keys())}")
print(f"[Historical] DEBUG: JSON type: {type(data)}")

# Process CVEs based on detected format
cves = []

# Try JSON 2.0 format first (newer format)
vulnerabilities = data.get("vulnerabilities", [])
print(f"[Historical] DEBUG: vulnerabilities field exists: {vulnerabilities is not None}")
print(f"[Historical] DEBUG: vulnerabilities length: {len(vulnerabilities) if vulnerabilities else 0}")

if vulnerabilities:
    print(f"[Historical] Found {len(vulnerabilities)} vulnerabilities in JSON 2.0 format")

# DEBUG: Show first item structure
if len(vulnerabilities) > 0:
    first_item = vulnerabilities[0]
    print(f"[Historical] DEBUG: First vulnerability keys: {list(first_item.keys()) if isinstance(first_item, dict) else 'NOT A DICT'}")
    if isinstance(first_item, dict) and 'cve' in first_item:
        print(f"[Historical] DEBUG: First CVE ID: {first_item.get('cve', {}).get('id', 'NO ID')}")

processed_count = 0
failed_count = 0

```

```

for item in vulnerabilities:
    processed = self._process_json_2_0_cve(item)
    if processed:
        cves.append(processed)
        processed_count += 1
    else:
        failed_count += 1

print(f"[Historical] DEBUG: Processed {processed_count} CVEs, {failed_count} failed")
else:
    # Try JSON 1.1 format (legacy format)
    cve_items = data.get("CVE_Items", [])
    print(f"[Historical] DEBUG: CVE_Items field exists: {cve_items is not None}")
    print(f"[Historical] DEBUG: CVE_Items length: {len(cve_items) if cve_items else 0}")

    if cve_items:
        print(f"[Historical] Found {len(cve_items)} CVE_Items in JSON 1.1 format")

    # DEBUG: Show first item structure
    if len(cve_items) > 0:
        first_item = cve_items[0]
        print(f"[Historical] DEBUG: First CVE_Item keys: {list(first_item.keys()) if isinstance(first_item, dict) else 'NOT A DICT'}")

    processed_count = 0
    failed_count = 0
    for item in cve_items:
        processed = self._process_json_1_1_cve(item)
        if processed:
            cves.append(processed)
            processed_count += 1
        else:
            failed_count += 1

    print(f"[Historical] DEBUG: Processed {processed_count} CVEs, {failed_count} failed")
    else:
        print(f"[Historical] ERROR: No recognized data format found")
        print(f"[Historical] Top-level keys: {list(data.keys())}")

    # Try to give more helpful debugging info
    for key in list(data.keys())[:5]: # First 5 keys
        value = data[key]
        print(f"[Historical] DEBUG: Key '{key}' = {type(value)} (length: {len(value) if hasattr(value, '__len__') else 'N/A'})")

    print(f"[Historical] Successfully processed {len(cves)} CVEs for {year}")

    if len(cves) == 0:
        print(f"[Historical] WARNING: Zero CVEs were processed! Check JSON structure.")

    return cves

except Exception as e:
    print(f"[Historical] ERROR loading {year}: {e}")
    import traceback
    traceback.print_exc()
    return []

def _process_json_2_0_cve(self, item: Dict) -> Dict:
    """Process JSON 2.0 format CVE - MEMORY OPTIMIZED"""
    try:
        cve_data = item.get("cve", {})
        if not cve_data:
            print(f"[Historical] DEBUG: Item missing 'cve' field, keys: {list(item.keys())}")
            return None

        cve_id = cve_data.get("id", "")

        if not cve_id:
            print(f"[Historical] DEBUG: CVE data missing 'id', keys: {list(cve_data.keys())}")
            return None

        # Extract English description
        description = ""
        for desc in cve_data.get("descriptions", []):
            if desc.get("lang") == "en":
                description = desc.get("value", "")

```

```

break

# Extract severity
severity = "UNKNOWN"
metrics = cve_data.get("metrics", {})

for version in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2"]:
    if version in metrics and metrics[version]:
        try:
            metric = metrics[version][0]
            cvss_data = metric.get("cvssData", {})
            potential_severity = cvss_data.get("baseSeverity", "")

            if potential_severity and potential_severity.upper() != "UNKNOWN":
                severity = potential_severity.upper()
                break
        except (IndexError, KeyError):
            continue

# Extract CWE
cwe = None
for weakness in cve_data.get("weaknesses", []):
    for desc in weakness.get("description", []):
        if desc.get("lang") == "en":
            value = desc.get("value", "")
            if value.startswith("CWE"):
                cwe = value
                break
    if cwe:
        break

return {
    "ID": cve_id,
    "Description": description,
    "Severity": severity,
    "CWE": cwe,
    "Published": cve_data.get("published", ""),
    "lastModified": cve_data.get("lastModified", ""),
    "References": [],
    "Products": [],
    "CVSS_Score": None,
    "metrics": {}
}

except Exception as e:
    print(f"[Historical] ERROR processing JSON 2.0 CVE: {e}")
    return None

def _process_json_1_1_cve(self, item: Dict) -> Dict:
    """Process JSON 1.1 format CVE - MEMORY OPTIMIZED"""
    try:
        cve_data = item.get("cve", {})
        if not cve_data:
            print(f"[Historical] DEBUG: Item missing 'cve' field")
            return None

        cve_id = cve_data.get("CVE_data_meta", {}).get("ID", "")

        if not cve_id:
            return None

        # Extract English description
        description = ""
        descriptions = cve_data.get("description", {}).get("description_data", [])
        for desc in descriptions:
            if desc.get("lang") == "en":
                description = desc.get("value", "")
                break

        # Extract severity
        severity = "UNKNOWN"
        impact = item.get("impact", {})

        if "baseMetricV3" in impact:
            severity = impact["baseMetricV3"].get("cvssV3", {}).get("baseSeverity",
                "UNKNOWN").upper()
        elif "baseMetricV2" in impact:

```

```

score = impact["baseMetricV2"].get("cvssV2", {}).get("baseScore", 0)
if score >= 7.0:
    severity = "HIGH"
elif score >= 4.0:
    severity = "MEDIUM"
else:
    severity = "LOW"

# Extract CWE
cwe = None
problem_types = cve_data.get("problemtype", {}).get("problemtype_data", [])
for problem_type in problem_types:
    for desc in problem_type.get("description", []):
        if desc.get("lang") == "en":
            value = desc.get("value", "")
            if value.startswith("CWE"):
                cwe = value
                break
    if cwe:
        break

return {
    "ID": cve_id,
    "Description": description,
    "Severity": severity,
    "CWE": cwe,
    "Published": cve_data.get("publishedDate", ""),
    "lastModified": cve_data.get("lastModifiedDate", ""),
    "References": [],
    "Products": [],
    "CVSS_Score": None,
    "metrics": {}
}

except Exception as e:
    print(f"[Historical] ERROR processing JSON 1.1 CVE: {e}")
    return None

# Global historical loader instance
historical_loader = DataProcessor()

```

services\data\data_source_config.py

```
# Operating system interface for file and directory operations
import os
# Date and time operations for temporal boundary calculations
from datetime import datetime, timezone
# Type annotations for structured return values
from typing import Dict, Tuple
# Application configuration for directory paths and settings
import config

class DataSourceConfig:
    """Dynamic data source configuration based on current date"""

    def __init__(self):
        # Capture current UTC timestamp for consistent temporal calculations
        self.current_date = datetime.now(timezone.utc)
        # Initialize cache variables for performance optimization
        self._cutoff_cache = None
        self._cache_date = None

    def get_data_source_cutoff(self) -> Tuple[Dict, str]:
        """
        Calculate dynamic cutoff between historical and API data
        Returns: (cutoff_info, explanation)
        """
        # Cache the cutoff calculation for the current day to improve performance
        today = self.current_date.date()
        if self._cutoff_cache and self._cache_date == today:
            return self._cutoff_cache

        # Extract current year and month for boundary calculations
        current_year = self.current_date.year
        current_month = self.current_date.month

        # Previous month is the cutoff for historical data availability
        if current_month == 1:
            # January - previous month is December of last year
            historical_cutoff_year = current_year - 1
            historical_cutoff_month = 12
            api_start_year = current_year
            api_start_month = 1
        else:
            # Any other month - previous month of same year
            historical_cutoff_year = current_year
            historical_cutoff_month = current_month - 1
            api_start_year = current_year
            api_start_month = current_month

        # Build comprehensive cutoff information structure
        cutoff_info = {
            'historical': {
                'end_year': historical_cutoff_year,
                'end_month': historical_cutoff_month,
                'years_range': list(range(2010, historical_cutoff_year + 1))
            },
            'api': {
                'start_year': api_start_year,
                'start_month': api_start_month
            },
            'current': {
                'year': current_year,
                'month': current_month,
                'date': today
            }
        }

        # Generate human-readable explanation for logging and debugging
        explanation = (
            f"Historical data: 2010 ? {historical_cutoff_year}-{historical_cutoff_month:02d}, "
            f"API data: {api_start_year}-{api_start_month:02d} onwards"
        )

        # Cache the result for same-day reuse
```

```

self._cutoff_cache = (cutoff_info, explanation)
self._cache_date = today
return cutoff_info, explanation

def should_use_historical(self, year: int, month: int = None) -> bool:
    """
    Determine if given year/month should use historical data
    """
    cutoff_info, _ = self.get_data_source_cutoff()
    historical_end = cutoff_info['historical']

    # Years clearly before cutoff use historical data
    if year < historical_end['end_year']:
        return True
    elif year == historical_end['end_year']:
        # For cutoff year, check month if provided
        if month is None:
            return True # Conservative approach for year-only queries
        return month <= historical_end['end_month']
    else:
        # Years after cutoff use API data
        return False

def should_use_api(self, year: int, month: int = None) -> bool:
    """
    Determine if given year/month should use API data
    """
    # Delegate to historical check and negate for consistency
    return not self.should_use_historical(year, month)

def get_available_historical_files(self) -> Dict[int, str]:
    """
    Get list of available historical data files
    """
    available_files = {}

    # Check if historical directory exists
    if not os.path.exists(config.NVD_HISTORICAL_DIR):
        print(f"[DataSource] Historical directory does not exist: {config.NVD_HISTORICAL_DIR}")
        return available_files

    # Check for different file patterns to accommodate various naming conventions
    patterns = [
        "CVE-{year}.json", # Simple naming
        "nvdcve-1.1-{year}.json", # JSON 1.1 format
        "nvdcve-2.0-{year}.json", # JSON 2.0 format
        "CVE-{year}.json.gz", # Compressed simple naming
        "nvdcve-1.1-{year}.json.gz", # Compressed JSON 1.1
        "nvdcve-2.0-{year}.json.gz" # Compressed JSON 2.0
    ]

    # Check years from 2010 to current year for comprehensive coverage
    current_year = self.current_date.year
    for year in range(2010, current_year + 1):
        for pattern in patterns:
            filename = pattern.format(year=year)
            filepath = os.path.join(config.NVD_HISTORICAL_DIR, filename)
            if os.path.exists(filepath):
                available_files[year] = filepath
                break # Use first matching pattern for each year

    return available_files

def log_data_source_status(self):
    """
    Log current data source configuration for monitoring and debugging
    """
    cutoff_info, explanation = self.get_data_source_cutoff()
    available_files = self.get_available_historical_files()

    # Log basic configuration information
    print(f"[DataSource] Current date: {self.current_date.strftime('%Y-%m-%d')}")
    print(f"[DataSource] Configuration: {explanation}")
    print(f"[DataSource] Historical directory: {config.NVD_HISTORICAL_DIR}")
    print(f"[DataSource] Historical files available: {len(available_files)} years")

    # Log detailed file availability information

```

```

if available_files:
print(f"[DataSource] Historical years: {min(available_files.keys())} - {max(available_files.keys())}")
print(f"[DataSource] Available files:")
for year, filepath in sorted(available_files.items()):
filename = os.path.basename(filepath)
# Get file size for monitoring storage and transfer issues
file_size = os.path.getsize(filepath) if os.path.exists(filepath) else 0
print(f"  {year}: {filename} ({file_size:,} bytes)")
else:
print(f"[DataSource] WARNING: No historical files found!")

# Check for missing files and alert about gaps in data coverage
expected_years = list(range(2010, cutoff_info['historical']['end_year'] + 1))
missing_years = [year for year in expected_years if year not in available_files]
if missing_years:
print(f"[DataSource] WARNING: Missing historical files for years: {missing_years}")

# Global instance for application-wide data source configuration
data_source_config = DataSourceConfig()

```

services\data\nvd_downloader.py

```
# Operating system interface for file and directory operations
import os
# JSON processing for vulnerability data handling
import json
# Gzip compression for efficient storage of large datasets
import gzip
# HTTP client library for NVD API communication
import requests
# Type annotations for function parameters and return values
from typing import List, Dict
# Application configuration for API credentials and paths
import config

class NVDDownloader:
    """Download and manage historical NVD data feeds"""

    def __init__(self):
        # Updated to use the correct NVD 2.0 API endpoints for historical data
        self.base_url = "https://services.nvd.nist.gov/rest/json/cves/2.0"
        # Load historical data directory from configuration
        self.historical_dir = config.NVD_HISTORICAL_DIR
        # Set generous timeout for large dataset downloads (5 minutes)
        self.timeout = 300
        # Load API key for authenticated access with higher rate limits
        self.api_key = config.NVD_API_KEY

    def download_year_via_api(self, year: int, save_as_json: bool = True) -> bool:
        """Download CVE data for a specific year using NVD API 2.0"""
        # Choose file extension based on format preference
        file_extension = ".json" if save_as_json else ".json.gz"
        filepath = os.path.join(self.historical_dir, f"CVE-{year}{file_extension}")

        # Skip download if file already exists
        if os.path.exists(filepath):
            print(f"[Downloader] {year} data already exists, skipping")
            return True

        try:
            print(f"[Downloader] Downloading {year} data via API...")
            # Use API 2.0 with full year date range for comprehensive coverage
            params = {
                "resultsPerPage": 2000, # Large page size for efficiency
                "startIndex": 0,
                "pubStartDate": f"{year}-01-01T00:00:00.000",
                "pubEndDate": f"{year}-12-31T23:59:59.999"
            }

            # Setup authentication headers if API key available
            headers = {"apiKey": self.api_key} if self.api_key else {}
            all_cves = []

            # Handle pagination to retrieve complete dataset
            while True:
                response = requests.get(
                    self.base_url,
                    params=params,
                    headers=headers,
                    timeout=self.timeout
                )

                if response.status_code == 200:
                    data = response.json()
                    vulnerabilities = data.get("vulnerabilities", [])

                    # Break if no more data available
                    if not vulnerabilities:
                        break

                    # Accumulate CVE data from this page
                    all_cves.extend(vulnerabilities)

            # Check if there are more results to fetch
```



```

total_results = data.get("totalResults", 0)
if len(all_cves) >= total_results:
    break

# Update start index for next page
params["startIndex"] += params["resultsPerPage"]

elif response.status_code == 403:
    print(f"[Downloader] API access forbidden for {year}")
    return False
else:
    print(f"[Downloader] Failed to download {year}: {response.status_code}")
    return False

# Prepare data for saving in NVD JSON 2.0 format
data_to_save = {"vulnerabilities": all_cves}

if save_as_json:
    # Save as plain JSON for fast access
    with open(filepath, 'w', encoding='utf-8') as f:
        json.dump(data_to_save, f, indent=2)
    print(f"[Downloader] Downloaded {year} successfully ({len(all_cves)} CVEs) - saved as JSON")
else:
    # Save as compressed JSON for storage efficiency
    with gzip.open(filepath, 'wt', encoding='utf-8') as f:
        json.dump(data_to_save, f, indent=2)
    print(f"[Downloader] Downloaded {year} successfully ({len(all_cves)} CVEs) - saved as compressed JSON")

return True

except Exception as e:
    print(f"[Downloader] Error downloading {year}: {e}")
    return False

def download_year(self, year: int, save_as_json: bool = True) -> bool:
    """Download CVE data for a specific year"""
    # Simple wrapper for API download method
    return self.download_year_via_api(year, save_as_json)

def download_all_years(self, years: List[int] = None, save_as_json: bool = True) -> Dict[int, bool]:
    """Download data for multiple years"""
    # Use configuration default years if none specified
    if years is None:
        years = config.HISTORICAL_YEARS

    # Track success/failure for each year
    results = {}
    for year in years:
        results[year] = self.download_year(year, save_as_json)

    # Report overall operation success
    successful = sum(1 for success in results.values() if success)
    print(f"[Downloader] Downloaded {successful}/{len(years)} years successfully")
    return results

def is_year_available(self, year: int) -> bool:
    """Check if year data is downloaded (checks both JSON and GZ formats)"""
    # Check for both possible file formats
    json_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json")
    gz_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json.gz")
    return os.path.exists(json_filepath) or os.path.exists(gz_filepath)

def get_available_years(self) -> List[int]:
    """Get list of years with downloaded data"""
    available = []
    # Check each configured historical year for data availability
    for year in config.HISTORICAL_YEARS:
        if self.is_year_available(year):
            available.append(year)
    return sorted(available)

def convert_gz_to_json(self, year: int) -> bool:
    """Convert existing .gz file to plain .json file"""
    gz_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json.gz")

```

```

json_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json")

# Check if source file exists
if not os.path.exists(gz_filepath):
    print(f"[Downloader] No .gz file found for {year}")
    return False

# Skip if target already exists
if os.path.exists(json_filepath):
    print(f"[Downloader] JSON file already exists for {year}")
    return True

try:
    print(f"[Downloader] Converting {year} from .gz to .json...")
    # Read compressed file and write as plain JSON
    with gzip.open(gz_filepath, 'rt', encoding='utf-8') as gz_file:
        data = json.load(gz_file)

    with open(json_filepath, 'w', encoding='utf-8') as json_file:
        json.dump(data, json_file, indent=2) # Pretty-print for readability

    print(f"[Downloader] Successfully converted {year} to JSON format")
    return True

except Exception as e:
    print(f"[Downloader] Error converting {year}: {e}")
    return False

def convert_all_gz_to_json(self) -> Dict[int, bool]:
    """Convert all existing .gz files to .json files"""
    results = {}
    # Convert each configured historical year
    for year in config.HISTORICAL_YEARS:
        results[year] = self.convert_gz_to_json(year)

    # Report overall conversion success
    successful = sum(1 for success in results.values() if success)
    print(f"[Downloader] Converted {successful}/{len(results)} files successfully")
    return results

```

static\css\styles.css

```
:root {
/* Background colors for different interface areas */
--bg-main: #151b24; /* main page background - dark blue-gray */
--bg-sidebar: #10151c; /* sidebar background - darker for contrast */
--bg-card: #1a222c; /* card/panel background - slightly lighter */
--card-border: #222b36; /* card border color - subtle definition */

/* Text colors for hierarchy and readability */
--text-main: #eaf4fb; /* default text color - light blue-white */
--text-muted: #8b9bb4; /* muted/secondary text color - medium gray */

/* NEW ACCENT COLORS - replacing all blues */
--primary: #FFD700; /* primary gold for links/headings */
--accent: #BCA424; /* deep tan for emphasis */
--accent-light: #F3D565; /* light khaki for highlights */

/* Severity colors following CVSS standards - DO NOT CHANGE */
--critical: #f55855; /* red: critical severity */
--high: #f8a541; /* orange: high severity */
--medium: #3b8ded; /* blue: medium severity */
--low: #4ddebe; /* teal: low severity */

/* Badge styling variables */
--badge-bg: #252e3b; /* background for general badges */
--badge-font: #fff; /* text color for badges */

/* General sizing & spacing variables for consistency */
--radius: 11px; /* consistent rounded corners */
--linegap-main: 1.6; /* reading line spacing for body text */
--linegap-heading: 1.25; /* tighter line spacing for headings */
}

/* Base typography with consistent line spacing */
body, html {
line-height: var(--linegap-main);
}

/* Headings use more compact line spacing */
h1, h2, h3, h4, h5, h6 {
line-height: var(--linegap-heading);
}

/* Apply main line-height to content blocks for consistency */
p, ul, ol, li, table, td, th, .main-content, .card-panel, .vuln-table td, .vuln-table
th, .sidebar-links a, .sidebar-submenu a, .dashboard-header, .card-title, .card-value {
line-height: var(--linegap-main);
}

/* Full page setup with dark theme foundation */
body {
margin: 0;
padding: 0;
background: var(--bg-main);
color: var(--text-main);
font-family: 'Inter', Arial, sans-serif;
}

/* Default link styling with NEW gold color */
a {
color: var(--primary);
text-decoration: none;
}

/* Fixed sidebar navigation system */
#sidebar {
position: fixed;
left: 0;
top: 0;
bottom: 0;
width: 215px;
background: var(--bg-sidebar);
padding-top: 32px;
```

```

box-shadow: 2px 0 15px rgba(10,20,25,0.09);
}

/* Logo styling within sidebar */
.sidebar-logo {
width: 160px;
margin: 0 auto 30px auto;
display: block;
}

/* Navigation menu link styling */
.sidebar-menu a {
display: block;
padding: 12px 32px 12px 32px;
color: var(--text-main);
font-weight: 500;
font-size: 1.05em;
border-left: 3px solid transparent;
}

/* Hover and active link states - NEW gold accent */
.sidebar-menu a.active,
.sidebar-menu a:hover {
background: var(--bg-card);
color: var(--accent);
border-left: 3px solid var(--accent);
}

/* Educational section in sidebar */
.sidebar-learn {
margin-left: 32px;
color: var(--text-muted);
font-size: 0.96em;
}

.sidebar-learn-title {
margin-bottom: 7px;
color: var(--primary);
font-weight: bold;
}

/* Main content area layout */
.main-content {
margin-left: 215px;
padding: 32px;
min-height: 100vh;
background: var(--bg-main);
}

/* Dashboard header with space-between alignment */
.dashboard-header {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 32px;
}

/* Top search bar with grouped elements */
.topbar-search {
display: flex;
gap: 8px;
}

/* Form input and select styling */
.topbar-search input[type="text"],
.topbar-search select {
border: none;
padding: 9px 14px;
border-radius: 5px;
background: var(--bg-card);
color: var(--text-main);
font-size: 1em;
}

/* Button styling with NEW gold colors */
.topbar-search button {
background: var(--accent);

```

```

border: none;
padding: 9px 17px;
border-radius: 6px;
color: white;
font-weight: 600;
cursor: pointer;
transition: background .18s;
}

/* Button hover state */
.topbar-search button:hover {
background: var(--primary);
}

/* Card/Panel component styling */
.card-panel {
background: var(--bg-card);
color: var(--text-main);
padding: 18px 24px;
border-radius: var(--radius);
border: 1px solid var(--card-border);
margin-bottom: 22px;
box-shadow: 0 1px 8px rgba(0,0,0,0.05);
}

/* Card title styling */
.card-title {
font-weight: 600;
margin-bottom: 10px;
color: var(--text-muted);
}

/* Card value styling for important numbers/stats */
.card-value {
font-size: 2em;
font-family: 'JetBrains Mono', monospace;
font-weight: bold;
}

/* Vulnerability table styling */
.vuln-table {
width: 100%;
border-collapse: collapse;
background: var(--bg-card);
margin-top: 18px;
border-radius: 8px;
overflow: hidden;
}

/* Table cell styling for consistent spacing */
.vuln-table th,
.vuln-table td {
padding: 11px 15px;
border-bottom: 1px solid var(--card-border);
}

/* Table header styling for differentiation */
.vuln-table th {
background: #1e2734;
color: var(--text-muted);
font-size: 0.96em;
}

/* Table content cell styling */
.vuln-table td {
color: var(--text-main);
}

/* Remove border from last table row for clean appearance */
.vuln-table tr:last-child td {
border-bottom: none;
}

/* CVE Severity badge styling - UNCHANGED */
.cve-badge {
display: inline-block;
padding: 4px 13px;

```

```

font-size: 0.84em;
border-radius: 5px;
font-weight: bold;
color: white;
margin-right: 3px;
}

/* Severity-specific background colors - UNCHANGED */
.cve-badge.CRITICAL { background: var(--critical); }
.cve-badge.HIGH { background: var(--high); }
.cve-badge.MEDIUM { background: var(--medium); }
.cve-badge.LOW { background: var(--low); }

/* Chart area container styling */
.chart-area {
width: 100%;
min-height: 220px;
margin-top: 10px;
}

/* Enhanced interactive severity card styling */
.severity-card {
background: #1a2236;
cursor: pointer;
border-radius: 18px;
padding: 20px;
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
transition: box-shadow 0.2s, transform 0.1s;
border: 1px solid transparent;
}

/* Hover state - NEW gold glow */
.severity-card:hover {
box-shadow: 0 0 0 3px rgba(255, 215, 0, 0.3);
transform: translateY(-2px);
border-color: rgba(255, 215, 0, 0.2);
}

/* Active state for click feedback */
.severity-card:active {
transform: translateY(0);
}

/* Grid layout for severity cards */
.severity-cards-grid {
display: grid;
grid-template-columns: repeat(4, 1fr);
gap: 15px;
flex: 1;
align-items: stretch;
}

/* Responsive breakpoint for smaller screens */
@media (max-width: 1200px) {
.severity-cards-grid {
grid-template-columns: repeat(2, 1fr);
}
}

```

static\js\main.js

```
document.addEventListener('DOMContentLoaded', function() {
// Timeline Chart ===
// Bar chart: CVEs discovered per month,
// for the last 5 years (data is embedded as a JSON string in the element's attribute)
const tctx = document.getElementById('timelineChart');
if (tctx) {
// Parse the 'data-timeline' attribute
// (should be an object: {month: count, ...})
const timelineData = JSON.parse(tctx.getAttribute('data-timeline') || '{}');
const labels = Object.keys(timelineData); // x-axis: months (e.g. '2021-10')
const data = Object.values(timelineData); // y-axis: CVE count for each month

new Chart(tctx, {
type: 'bar',
data: {
labels: labels,
datasets: [{
label: 'CVEs by Month (Last 5 Years)',
data: data,
backgroundColor: 'rgba(54, 162, 235, 0.5)', // soft blue bars
borderColor: 'rgba(54, 162, 235, 1)', // sharper blue borders
borderWidth: 1
}]
},
options: {
scales: { y: { beginAtZero: true } }, // y-axis always starts at zero
plugins: { legend: { display: false } } // hide the legend (single series)
}
});
}

// CWE Chart ===
// Horizontal bar chart of CWE type frequencies
// (data is also embedded as JSON in attribute)
const cweCtx = document.getElementById('cweChart');
if (cweCtx) {
// Parse the 'data-cwe' attribute
// (should be { CWE-79: count, CWE-89: count, ... })
const cweData = JSON.parse(cweCtx.getAttribute('data-cwe') || '{}');
const cweLabels = Object.keys(cweData); // y-axis: CWE code/title
const cweCounts = Object.values(cweData); // x-axis: count for each CWE

new Chart(cweCtx, {
type: 'bar',
data: {
labels: cweLabels,
datasets: [{
label: 'CWE Count',
data: cweCounts,
backgroundColor: 'rgba(255, 99, 132, 0.5)', // soft red bars
borderColor: 'rgba(255, 99, 132, 1)', // red borders
borderWidth: 1
}]
},
options: {
indexAxis: 'y', // horizontal bars
scales: { x: { beginAtZero: true } }, // x-axis starts at zero
plugins: { legend: { display: false } } // hide legend (single series)
}
});
}
});
```

static\js\core\charts.js

```
const ChartConfig = {
// Centralized color palette matching application theme and security standards
colors: {
critical: '#f55855',    // Red for critical severity (universal danger color)
high: '#f8a541',       // Orange for high severity (warning/urgency)
medium: '#3b8ded',     // Blue for medium severity (informational)
low: '#42d392',        // Green for low severity (safe/minimal risk)
unknown: '#6b7280',    // Gray for unknown/undefined severity
primary: '#63a4ff',    // Primary accent color for general chart elements
secondary: '#94a3b8'   // Secondary color for supporting elements
},

// Default chart options for consistent appearance across all charts
defaultOptions: {
responsive: true,           // Charts adapt to container size changes
maintainAspectRatio: false, // Allow flexible aspect ratios for different
containers
plugins: {
legend: {
labels: {
color: '#e2e8f0',    // Light color for legend text on dark background
font: { size: 12 }   // Consistent legend font size
}
}
},
scales: {
x: {
ticks: { color: '#94a3b8' },           // Muted color for x-axis labels
grid: { color: 'rgba(255,255,255,0.1)' } // Subtle grid lines
},
y: {
ticks: { color: '#94a3b8' },           // Muted color for y-axis labels
grid: { color: 'rgba(255,255,255,0.1)' } // Subtle grid lines
}
}
},

// Severity colors as array for easy iteration in chart data
severityColors: ['#f55855', '#f8a541', '#3b8ded', '#42d392', '#6b7280'],

// Utility function to create linear gradients for enhanced chart styling
createGradient: function(ctx, color1, color2) {
const gradient = ctx.createLinearGradient(0, 0, 0, 400); // Vertical gradient
gradient.addColorStop(0, color1); // Start color at top
gradient.addColorStop(1, color2); // End color at bottom
return gradient;
}
};

// Chart creation utilities providing factory methods for different chart types
const ChartUtils = {
// Create doughnut-style pie chart for severity distribution visualization
createPieChart: function(ctx, data, options = {}) {
return new Chart(ctx, {
type: 'doughnut',           // Doughnut style provides space for central content
data: data,
options: {
...ChartConfig.defaultOptions, // Apply base configuration
cutout: options.cutout || '65%', // Default 65% cutout for doughnut hole
...options                    // Merge any additional custom options
}
});
},

// Create line chart for trend visualization and time series data
createLineChart: function(ctx, data, options = {}) {
return new Chart(ctx, {
type: 'line',
data: data,
options: {
...ChartConfig.defaultOptions, // Apply base configuration
elements: {

```



```

line: { tension: 0.3 },      // Smooth curve without over-smoothing
point: {
radius: 4,                  // Visible points for data clarity
hoverRadius: 6              // Larger hover area for interaction
},
},
...options                  // Merge any additional custom options
});
},

// Create bar chart for categorical data comparison
createBarChart: function(ctx, data, options = {}) {
return new Chart(ctx, {
type: 'bar',
data: data,
options: {
...ChartConfig.defaultOptions, // Apply base configuration
...options                     // Merge any additional custom options
}
});
},

// Create radar chart for multi-dimensional security analysis (e.g., CWE analysis)
createRadarChart: function(ctx, data, options = {}) {
return new Chart(ctx, {
type: 'radar',
data: data,
options: {
...ChartConfig.defaultOptions, // Apply base configuration
scales: {
r: {                          // Radial scale configuration
angleLines: {
color: 'rgba(255,255,255,0.1)' // Subtle angle lines
},
grid: {
color: 'rgba(255,255,255,0.1)' // Subtle radial grid
},
pointLabels: {
color: '#a9adc1', // Muted color for category labels
font: {
size: 13, // Readable font size for labels
weight: 'bold' // Bold weight for emphasis
}
},
beginAtZero: true, // Start scale from zero for accurate comparison
ticks: { display: false } // Hide tick marks to reduce clutter
}
},
...options // Merge any additional custom options
});
},
};

```

static\js\core\interactions.js

```
// Event handlers and interactions
const Interactions = {
  // Initialize all interactions
  init: function() {
    this.initSeverityCards();
    this.initChartClicks();
    this.initSearchFilters();
    this.initTooltips();
  },

  // Severity card click handlers
  initSeverityCards: function() {
    document.querySelectorAll(".severity-card").forEach(function(card) {
      card.addEventListener("click", function() {
        var sev = card.getAttribute("data-severity");
        if (sev) {
          window.location.href = "/vulnerabilities?severity=" + encodeURIComponent(sev);
        }
      });

      card.addEventListener("mouseenter", function() {
        card.style.boxShadow = "0 0 0 3px #63a4ff44";
      });

      card.addEventListener("mouseleave", function() {
        card.style.boxShadow = "";
      });
    });
  },

  // Chart click handlers
  initChartClicks: function() {
    // These will be implemented in specific page JS files
    // This is just the base structure
  },

  // Search and filter interactions
  initSearchFilters: function() {
    // Auto-submit forms on filter change
    const filterSelects = document.querySelectorAll('select[name="severity"], select[name="year"], select[name="month"]');
    filterSelects.forEach(select => {
      select.addEventListener('change', function() {
        // Auto-submit the form when filter changes
        const form = this.closest('form');
        if (form) {
          form.submit();
        }
      });
    });
  },

  // Search input debouncing
  const searchInputs = document.querySelectorAll('input[name="q"]');
  searchInputs.forEach(input => {
    let debounceTimer;
    input.addEventListener('input', function() {
      clearTimeout(debounceTimer);
      debounceTimer = setTimeout(() => {
        // Could implement live search here
      }, 500);
    });
  });

  // Tooltip initialization
  initTooltips: function() {
    document.querySelectorAll('[data-tooltip]').forEach(element => {
      element.addEventListener('mouseenter', this.showTooltip);
      element.addEventListener('mouseleave', this.hideTooltip);
    });
  },
}
```

```
showTooltip: function(event) {
const tooltip = document.createElement('div');
tooltip.className = 'custom-tooltip';
tooltip.textContent = event.target.getAttribute('data-tooltip');
tooltip.style.cssText = `
position: absolute;
background: #1a2236;
color: #e2e8f0;
padding: 8px 12px;
border-radius: 6px;
font-size: 0.9em;
z-index: 1000;
pointer-events: none;
box-shadow: 0 4px 12px rgba(0,0,0,0.3);
`;

document.body.appendChild(tooltip);

const rect = event.target.getBoundingClientRect();
tooltip.style.left = rect.left + 'px';
tooltip.style.top = (rect.bottom + 5) + 'px';

event.target._tooltip = tooltip;
},

hideTooltip: function(event) {
if (event.target._tooltip) {
document.body.removeChild(event.target._tooltip);
delete event.target._tooltip;
}
}
};

// Initialize interactions when DOM is ready
document.addEventListener('DOMContentLoaded', function() {
Interactions.init();
});
```

static\js\pages\dashboard.js

```
document.addEventListener('DOMContentLoaded', function() {
// Initialize severity card interactions for quick filtering
initSeverityCards();

// Timeline Chart: 5-year monthly vulnerability trends
// Shows long-term patterns and seasonal variations
const timelineChartEl = document.getElementById('timelineChart');
let timelineChartInstance = null;

if (timelineChartEl) {
function drawTimelineChart() {
const ctx = timelineChartEl.getContext('2d');

// Extract data from server-injected window objects
const labels = window.timelineData.labels || [];
const data = window.timelineData.values || [];

// Destroy existing chart if it exists
if (timelineChartInstance) {
timelineChartInstance.destroy();
}

timelineChartInstance = new Chart(ctx, {
type: 'line',
data: {
labels: labels,
datasets: [{
label: 'CVEs Per Month',
data: data,
borderColor: '#63a4ff',
backgroundColor: 'rgba(99, 164, 255, 0.1)',
fill: true,
tension: 0.3,
borderWidth: 2,
pointRadius: 4,
pointHoverRadius: 6,
pointBackgroundColor: '#63a4ff'
}]
},
options: {
responsive: true,
maintainAspectRatio: false,
plugins: {
legend: { display: false },
tooltip: {
mode: 'index',
intersect: false,
callbacks: {
title: function(context) {
return 'Month: ' + context[0].label;
},
label: function(context) {
return 'CVEs: ' + context.parsed.y;
}
}
}
},
scales: {
x: {
grid: { display: false },
ticks: {
color: '#8b9bb4',
callback: function(value, index, values) {
const label = this.getLabelForValue(value);
if (!label) return null;
if (label.slice(-2) === "01") return label.slice(0, 4);
return "";
},
autoSkip: false,
maxRotation: 0,
minRotation: 0
}
}
}
}
}
```

```

},
y: {
  beginAtZero: true,
  grid: { display: false },
  ticks: {
    color: '#8b9bb4',
    stepSize: 2000,
    callback: function(value, index, ticks) {
      return value % 500 === 0 ? value : '';
    }
  }
},
interaction: {
  mode: 'nearest',
  axis: 'x',
  intersect: false
},
onClick: (evt, activeEls) => {
  if (activeEls && activeEls.length) {
    const chart = activeEls[0].element.$context.chart;
    const idx = activeEls[0].index;
    const label = chart.data.labels[idx];
    if (label && label.length >= 7) {
      const [year, month] = label.split('-');
      window.location.href = '/vulnerabilities?year=' + year + '&month=' + month;
    }
  }
});

// Initial draw
drawTimelineChart();

// Timeline Years Filter Handler
const timelineFilter = document.getElementById('timelineYearsFilter');
if (timelineFilter) {
  timelineFilter.addEventListener('change', async function() {
    const years = parseInt(this.value);
    console.log(`Loading ${years} year(s) of timeline data...`);

    // Show loading state
    timelineChartEl.style.opacity = '0.5';

    try {
      // Fetch new data from API
      const response = await fetch(`/api/timeline-data?years=${years}`);
      const result = await response.json();

      if (result.success) {
        // Update global timeline data
        window.timelineData = result.timeline;

        // Redraw chart
        drawTimelineChart();

        console.log(`Successfully loaded ${years} year(s) of data`);
      } else {
        console.error('Error loading timeline:', result.error);
        alert('Error loading timeline data. Please try again.');
```

```

const dataValues = window.timelineDataDaily.values || [];

const dayLabels = labels.map((label, idx) => {
return (idx % 7 === 0) ? label.slice(5, 10) : '';
});

new Chart(ctxDaily, {
type: 'bar',
data: {
labels: dayLabels,
datasets: [{
label: 'CVEs per Day (Last 30 Days)',
data: dataValues,
backgroundColor: 'rgba(99, 164, 255, 0.8)',
borderColor: 'rgba(99, 164, 255, 1)',
tension: 0.3,
fill: true,
pointRadius: 3,
pointHoverRadius: 6
}]
},
options: {
responsive: true,
maintainAspectRatio: false,
plugins: {
legend: { display: false },
tooltip: {
mode: 'index',
intersect: false,
callbacks: {
title: function(context) {
const idx = context[0].dataIndex;
return window.timelineDataDaily.labels && window.timelineDataDaily.labels[idx] ?
window.timelineDataDaily.labels[idx] : '';
}
}
}
},
scales: {
y: {
beginAtZero: true,
grid: { display: false },
ticks: { color: '#90caf9' }
},
x: {
grid: { display: false },
ticks: {
color: '#90caf9',
maxRotation: 0,
autoSkip: false,
maxTicksLimit: 10
}
}
},
interaction: {
mode: 'nearest',
axis: 'x',
intersect: false
},
onClick: (evt, activeElements) => {
if (activeElements && activeElements.length > 0) {
const idx = activeElements[0].index;
const fullDate = window.timelineDataDaily.labels[idx];
if (fullDate) {
const year = fullDate.slice(0, 4);
const month = parseInt(fullDate.slice(5, 7));
const day = parseInt(fullDate.slice(8, 10));
window.location.href = '/vulnerabilities?year=' + year + '&month=' + month + '&day=' +
day;
}
}
});

```

```

// Severity Doughnut Chart

```

```

if (document.getElementById('severityPie')) {
const ctxPie = document.getElementById('severityPie').getContext('2d');

new Chart(ctxPie, {
type: 'doughnut',
data: {
labels: ["Critical", "High", "Medium", "Low", "Unknown"],
datasets: [{
data: [
window.severityStats.CRITICAL || 0,
window.severityStats.HIGH || 0,
window.severityStats.MEDIUM || 0,
window.severityStats.LOW || 0,
window.severityStats.UNKNOWN || 0
],
backgroundColor: [
'#f55855',
'#f8a541',
'#3b8ded',
'#42d392',
'#6b7280'
],
borderWidth: 4,
borderColor: '#1a2236',
hoverOffset: 10
}]
},
options: {
cutout: '65%',
plugins: {
legend: { display: false },
tooltip: {
callbacks: {
label: function(context) {
const label = context.label;
const value = context.parsed;
const total = context.dataset.data.reduce((a, b) => a + b, 0);
const percentage = total > 0 ? ((value / total) * 100).toFixed(1) : 0;
return `${label}: ${value} (${percentage}%)`;
}
}
}
},
onClick: (evt, activeEls) => {
if (activeEls && activeEls.length) {
const chart = activeEls[0].element.$context.chart;
const idx = activeEls[0].index;
const label = chart.data.labels[idx];
if (label) {
window.location.href = '/vulnerabilities?severity=' + label.toUpperCase();
}
}
});

// Vendor Risk Analysis Radar Chart
if (document.getElementById('vendorRiskChart')) {
const ctx = document.getElementById('vendorRiskChart').getContext('2d');
let topN = 10;
let weighted = false;

function goToVulnsForCWE(idx) {
const cwe_key = window.cweSeverityData.indices[idx];
if (cwe_key) {
window.location.href = "/vulnerabilities?q=" + encodeURIComponent(cwe_key);
}
}

function getRadarData() {
let srcAll = window.cweRadarAll;
let source = srcAll.top_10 || srcAll['all'] || window.cweRadar || {};

if (topN === 5 && srcAll.top_5) source = srcAll.top_5;
else if (topN === 10 && srcAll.top_10) source = srcAll.top_10;
else if (topN === 'all' && srcAll.all) source = srcAll.all;

```

```

if (weighted && window.cweRadarWeighted && window.cweRadarWeighted.indices) {
let srcW = window.cweRadarWeighted;
let codes = srcW.indices, names = srcW.labels, values = srcW.values;

if (topN !== 'all' && values.length > topN) {
codes = codes.slice(0, topN);
names = names.slice(0, topN);
values = values.slice(0, topN);
}
return { codes, names, values };
}

let codes = source.indices || [];
let names = source.labels || [];
let values = source.values || [];

if (topN !== 'all' && values.length > topN) {
codes = codes.slice(0, topN);
names = names.slice(0, topN);
values = values.slice(0, topN);
}
return { codes, names, values };
}

// Enhanced tooltip with CWE details, definitions, and mitigations
function radarTooltip(context) {
const code = context.label;
const name = context.dataset.meta.names ?
context.dataset.meta.names[context.dataIndex] : "";
const val = context.dataset.data[context.dataIndex];

// Get 30-day activity count for this CWE
const cwe30DayCount = window.cwe30DayCounts &&
window.cwe30DayCounts[code] ?
window.cwe30DayCounts[code] : 0;

// Get CWE definition if available
let def = "";
if (window.cweRadarDescriptions && window.cweRadarDescriptions[code]) {
def = window.cweRadarDescriptions[code];
}

// Get mitigation information if available
let mitig = "";
if (window.cweMitigations && window.cweMitigations[code]) {
mitig = "Mitigation: " + window.cweMitigations[code];
}

// Return comprehensive tooltip information
return [
`CWE ${code}`,
`Name: ${name}`,
`Number of CVEs: ${val} (last 2 years)`,
`Number of CVEs in last 30 days: ${cwe30DayCount}`,
...(def ? [def] : []),
...(mitig ? [mitig] : []),
`Learn more: https://cwe.mitre.org/data/definitions/\${code.replace\('CWE-', ''\)}.html`
];
}

// Redraw radar chart with current filter settings
function drawRadar() {
const { codes, names, values } = getRadarData();
// Clear canvas and destroy existing chart
ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);
if (window.radarChartObj && window.radarChartObj.destroy) {
window.radarChartObj.destroy();
}

// Create new radar chart instance
window.radarChartObj = new Chart(ctx, {
type: 'radar',
data: {
labels: codes, // CWE codes as labels
datasets: [{
label: 'Vulnerability Frequency',

```



```

data: values,
backgroundColor: 'rgba(99, 164, 255, 0.2)', // Semi-transparent fill
borderColor: '#63a4ff', // Border color
pointBackgroundColor: '#63a4ff', // Point color
pointBorderColor: '#fff', // Point border
pointHoverRadius: 6, // Hover point size
pointRadius: 4, // Normal point size
pointHitRadius: 21, // Click detection area
meta: { names: names } // Store names for tooltips
}],
options: {
  responsive: true,
  maintainAspectRatio: false,
  plugins: {
    legend: { display: false },
    tooltip: {
      callbacks: { label: radarTooltip } // Use custom tooltip
    }
  },
  scales: {
    r: { // Radial scale configuration
      angleLines: { color: 'rgba(255,255,255,0.1)' }, // Subtle angle lines
      grid: { color: 'rgba(255,255,255,0.1)' }, // Subtle grid
      pointLabels: {
        color: '#a9adc1', // Label color
        font: { size: 13, weight: 'bold' } // Label font
      },
      beginAtZero: true, // Start scale at zero
      min: 0,
      ticks: { display: false } // Hide tick marks
    }
  },
  // Click handler for CWE-specific search
  onClick: (evt, activeEls) => {
    if (activeEls && activeEls.length) {
      const chart = activeEls[0].element.$context.chart;
      const idx = activeEls[0].index;
      const code = chart.data.labels[idx];
      if (code) {
        // Navigate to CWE-filtered search
        window.location.href = "/vulnerabilities?q=" + encodeURIComponent(code);
      }
    }
  });
});

// Event handler for CWE count filter dropdown
document.getElementById('radarCweCount').onchange = function() {
  topN = this.value === "all" ? "all" : parseInt(this.value, 10);
  drawRadar(); // Redraw chart with new filter
};

// Event handler for weighted scoring toggle
document.getElementById('radarWeighted').onchange = function() {
  weighted = this.checked;
  drawRadar(); // Redraw chart with weighted/unweighted data
};

// Initial chart draw with default settings
drawRadar();

// Info popover system for legend explanations
const infoIcon = document.getElementById('legendInfoIcon');
const popover = document.getElementById('legendInfoPopover');
if (infoIcon && popover) {
  function showPopover(){
    popover.style.display = "block"; // Show explanation popover
  }
  function hidePopover(){
    popover.style.display = "none"; // Hide explanation popover
  }
  // Attach multiple event types for comprehensive interaction
  infoIcon.addEventListener("click", showPopover);
  infoIcon.addEventListener("mouseenter", showPopover);

```

```

infoIcon.addEventListener("focus", showPopover);    // Keyboard accessibility
infoIcon.addEventListener("blur", hidePopover);    // Keyboard accessibility
infoIcon.addEventListener("mouseleave", hidePopover);
}
}

// Initialize severity card interactions for quick filtering
function initSeverityCards() {
document.querySelectorAll(".severity-card").forEach(function(card) {
// Click handler for severity-based navigation
card.addEventListener("click", function() {
var sev = card.getAttribute("data-severity");
if (sev) {
// Navigate to severity-filtered vulnerability list
window.location.href = "/vulnerabilities?severity=" + encodeURIComponent(sev);
}
});
});

// Mouse enter handler for visual feedback
card.addEventListener("mouseenter", function() {
card.style.boxShadow = "0 0 0 3px #63a4ff44"; // Blue glow effect
card.style.cursor = "pointer";                // Pointer cursor
});

// Mouse leave handler to remove visual effects
card.addEventListener("mouseleave", function() {
card.style.boxShadow = ""; // Remove glow effect
card.style.cursor = "";    // Reset cursor
});
});
});
});
});

```

static\js\pages\learn.js

```
// Main initialization - wait for DOM to be fully loaded before setting up interactive
features
document.addEventListener('DOMContentLoaded', function() {
// Stacked CWE Severity Bar Chart (Learn View) ===
// Only run if we have data for the chart and the proper container exists.
if (
window.learnCweSeverityData &&
Array.isArray(window.learnCweSeverityData.labels) &&
window.learnCweSeverityData.labels.length > 0 &&
document.getElementById('learnCweSeverityChart')
) {
// Get the canvas context for Chart.js rendering
const ctx = document.getElementById('learnCweSeverityChart').getContext('2d');
const data = window.learnCweSeverityData.data || {};

// Create stacked bar chart showing CWE severity distribution
new Chart(ctx, {
type: 'bar',
data: {
labels: window.learnCweSeverityData.labels, //CWE names/codes for x-axis
datasets: [
// Color-coded severity levels with industry-standard colors
{ label: 'Critical', data: data.CRITICAL || [], backgroundColor: '#f55855' }, // Red for
critical
{ label: 'High', data: data.HIGH || [], backgroundColor: '#f8a541' }, // Orange
for high
{ label: 'Medium', data: data.MEDIUM || [], backgroundColor: '#3b8ded' }, // Blue
for medium
{ label: 'Low', data: data.LOW || [], backgroundColor: '#42d392' } // Green
for low
]
},
options: {
responsive: true, // Adapt to container size changes
maintainAspectRatio: false, // Allow height adjustments
plugins: {
title: { display: true, text: 'CWEs by Severity (Key Set)' },
tooltip: { mode: 'index', intersect: false } // Show all values when hovering
},
scales: { x: { stacked: true }, y: { stacked: true } } // Stack bars for cumulative view
});
}

// Utility: HTML Encode (avoid XSS in dynamic content) ===
function htmlEncode(text) {
// Create temporary DOM element to leverage browser's built-in HTML encoding
var el = document.createElement('div');
el.innerText = text || ''; // innerText automatically encodes HTML entities
return el.innerHTML; // Return the encoded result
}

// Show "Loading..." Panel for CWE Detail ===
function showLoadingState(code) {
// Display immediate feedback with loading animation and estimated time
document.getElementById('cweDetailPanel').innerHTML = `
<div style="text-align: center; padding: 40px; color: #a9adc1;">
<div style="font-size: 2rem; margin-bottom: 16px;">?</div>
<div>Loading ${code} details from MITRE...</div>
<div style="margin-top: 8px; font-size: 0.9em;">This may take a few seconds</div>
</div>
`;
}

// Show Error/Failure Panel for CWE Detail ===
function showErrorState(code, error) {
// Display user-friendly error with fallback option to official MITRE site
document.getElementById('cweDetailPanel').innerHTML = `
<div style="text-align: center; padding: 40px; color: #f47174;">
<div style="font-size: 2rem; margin-bottom: 16px;">?</div>
<div>Failed to load ${code} details</div>
<div style="margin-top: 8px; font-size: 0.9em; color: #a9adc1;">

```

```

Error: ${htmlEncode(error)}
</div>
<div style="margin-top: 16px;">
<a href="https://cwe.mitre.org/data/definitions/${code.replace('CWE-', '')}.html"
target="_blank"
style="color: #63a4ff; text-decoration: underline;">
View on MITRE website ?
</a>
</div>
</div>
`;
}

// Fetch CWE data from backend API
async function fetchCweData(code) {
try {
// Make HTTP request to our backend CWE API endpoint
const response = await fetch(`/api/cwe/${code}`);
const result = await response.json();

// Check if API returned successful result with data
if (result.success && result.data) {
return result.data;
} else {
// API returned error or no data
throw new Error(result.error || 'Failed to fetch CWE data');
}
} catch (error) {
console.error(`Error fetching CWE ${code}:`, error);
throw error; // Re-throw for caller to handle
}
}

// Render the CWE Detail View (description, mitigation, examples, etc) ===
async function renderDetail(code) {
// Show loading state immediately for better UX
showLoadingState(code);

try {
// First check if we have local data cached
const cweDict = window.learnCweDict || {};
let entry = cweDict[code];

// If no local data or it's placeholder data, fetch from API
if (!entry ||
entry.description?.includes('Click to fetch detailed information') ||
entry.mitigations?.includes('Loading detailed information...') ||
!entry.description ||
entry.description.length < 50) { // Detect placeholder/incomplete data

console.log(`Fetching fresh data for ${code}...`);
entry = await fetchCweData(code);

// Update local cache for future use
if (window.learnCweDict) {
window.learnCweDict[code] = entry;
}
}

if (!entry) {
throw new Error('No CWE data available');
}

// Extract relationships from CWE data for navigation
let relParent = entry.relationships ?
entry.relationships.filter(r => r[1] && r[1].toLowerCase().includes('parent')).map(r =>
r[0]) : [];
let relChild = entry.relationships ?
entry.relationships.filter(r => r[1] && r[1].toLowerCase().includes('child')).map(r =>
r[0]) : [];
let relRelated = entry.relationships ?
entry.relationships.filter(r => r[1] && r[1].toLowerCase().includes('related')).map(r =>
r[0]) : [];

// Build the relationships HTML section with links to MITRE
let relationshipsHtml = '';
if (relParent.length || relChild.length || relRelated.length) {

```

```

relationshipsHtml = '<div style="margin-bottom:16px;"><strong>Relationships:</strong><ul
style="margin-left:15px;">';

// Parent relationships
if (relParent.length) {
relationshipsHtml += '<li><span style="color:#3b8ded;">Parent: </span>`;
relParent.forEach((parent, index) => {
relationshipsHtml += '<a target="_blank"
href="https://cwe.mitre.org/data/definitions/${parent.replace('CWE-', '')}.html"
style="color:#63a4ff;text-decoration:underline;">${parent}</a>`;
if (index < relParent.length - 1) relationshipsHtml += ', ';
});
relationshipsHtml += '</li>';
}

// Child relationships (limit to 3 for readability)
if (relChild.length) {
relationshipsHtml += '<li><span style="color:#3b8ded;">Children: </span>`;
relChild.slice(0, 3).forEach((child, index) => {
relationshipsHtml += '<a target="_blank"
href="https://cwe.mitre.org/data/definitions/${child.replace('CWE-', '')}.html"
style="color:#63a4ff;text-decoration:underline;">${child}</a>`;
if (index < Math.min(relChild.length, 3) - 1) relationshipsHtml += ', ';
});
if (relChild.length > 3) relationshipsHtml += '...'; // Indicate more exist
relationshipsHtml += '</li>';
}

// Related CWEs (limit to 2 for space)
if (relRelated.length) {
relationshipsHtml += '<li><span style="color:#3b8ded;">Related: </span>`;
relRelated.slice(0, 2).forEach((related, index) => {
relationshipsHtml += '<a target="_blank"
href="https://cwe.mitre.org/data/definitions/${related.replace('CWE-', '')}.html"
style="color:#63a4ff;text-decoration:underline;">${related}</a>`;
if (index < Math.min(relRelated.length, 2) - 1) relationshipsHtml += ', ';
});
if (relRelated.length > 2) relationshipsHtml += '...';
relationshipsHtml += '</li>';
}
relationshipsHtml += '</ul></div>';
}

// Build mitigations HTML section with visual indicators
let mitigationsHtml = '';
if (entry.mitigations && entry.mitigations.length > 0) {
mitigationsHtml = '<div style="margin-bottom:16px;"><strong>Key Mitigations:</strong><ul
style="margin-left:15px;color:#42d392;">`;
entry.mitigations.slice(0, 3).forEach(mitigation => { // Limit to top 3 mitigations
mitigationsHtml += '<li>? ${htmlEncode(mitigation)}</li>`; // Lightbulb emoji for tips
});
mitigationsHtml += '</ul></div>';
}

// Build examples HTML section with code styling
let examplesHtml = '';
if (entry.examples && entry.examples.length > 0) {
examplesHtml = '<div style="margin-bottom:16px;"><strong>Examples:</strong><ul
style="margin-left:15px;color:#fca311;">`;
entry.examples.slice(0, 2).forEach(example => { // Limit to 2 examples
// Truncate long examples to prevent UI overflow
const truncatedExample = example.length > 100 ? example.substring(0, 100) + '...' :
example;
examplesHtml += '<li>? <code style="background:#263159;padding:2px
6px;border-radius:3px;">${htmlEncode(truncatedExample)}</code></li>`;
});
examplesHtml += '</ul></div>';
}

// Source indicator to show data freshness
let sourceHtml = '';
if (entry.source === 'scraped') {
sourceHtml = '<div style="margin-top:12px;padding:8px
12px;background:#263159;border-radius:6px;font-size:0.9em;color:#94a3b8;"><span
style="color:#42d392;">?</span> Live data from MITRE</div>';
}

```

```

// Assemble complete detail view with all sections
document.getElementById('cweDetailPanel').innerHTML = `
<div>
<h2 style="color:#63a4ff;margin-top:0;margin-bottom:12px;">${code}:
${htmlEncode(entry.name)}</h2>
<div style="margin-bottom: 18px; color:#e2e8f2;">
<strong>Definition:</strong>
<div style="margin-top:8px;line-height:1.5;">${htmlEncode(entry.description || 'No
definition available.')}</div>
</div>
${mitigationsHtml}
${examplesHtml}
${relationshipsHtml}
<div style="margin-top:18px;color:#a9adc1;font-size:0.97em;">
<a href="https://cwe.mitre.org/data/definitions/${code.replace('CWE-', '')}.html"
target="_blank"
style="color:#63a4ff;text-decoration:underline;">
View official CWE documentation ?
</a>
</div>
${sourceHtml}
</div>
`;
} catch (error) {
console.error(`Failed to render CWE ${code}:`, error);
showErrorState(code, error.message); // Show user-friendly error
}
}

// Click-binding for CWE list items: show details on select ===
function hookList(id) {
const list = document.getElementById(id);
if (!list) return; // Exit if list doesn't exist

// Attach click handlers to all list items
Array.from(list.children).forEach(li => {
li.onclick = async function() {
// Update visual selection - clear all items first
Array.from(list.children).forEach(lui => {
lui.style.background = '';
lui.style.borderLeft = '4px solid transparent';
lui.style.color = '#e3e7f2';
});
// Highlight selected item with blue accent
this.style.background = '#212d46';
this.style.borderLeft = '4px solid #63a4ff';
this.style.color = '#63a4ff';

// Render the detail (this will handle loading and error states)
await renderDetail(this.dataset.cwe);
};
});

// Auto-select first item if available for better initial UX
if (list.children.length) {
list.children[0].click();
}
}

// Hook up both CWE lists (key set and full catalog)
hookList('cweListColumn'); // The filtered key set
hookList('cweListFull'); // The complete CWE catalog

// Toggle between key CWES and full list
document.getElementById('toggleShowAllCwe').onclick = function() {
// Determine current state from button text
let showAll = this.innerText.includes('Show All');

// Toggle visibility of the two different lists
document.getElementById('cweListColumn').style.display = showAll ? 'none' : '';
document.getElementById('cweListFull').style.display = showAll ? '' : 'none';

// Update mode indicator and button text
document.getElementById('cweListModeTip').innerText = showAll ? '(Full Catalog)' :
'(Your Key Set)';
this.innerText = showAll ? "Show Only Key CWES" : "Show All CWES";
}

```

```
// Auto-select first item in the newly visible list for continuity
let list = showAll ? document.getElementById('cweListFull') :
document.getElementById('cweListColumn');
if (list && list.children.length) {
list.children[0].click();
}
};
});
```

static\js\pages\vulnerabilities.js

```
// Wait for DOM to be fully constructed before setting up interactive features
document.addEventListener('DOMContentLoaded', function() {
// Toggle Critical/High section
// Exposed globally so HTML onclick attributes can access it easily
window.toggleCriticalHighSection = function() {
const content = document.getElementById('criticalHighContent'); // Get collapsible
content area
const icon = document.getElementById('collapseIcon'); // Get expand/collapse
indicator

// Check current visibility state and toggle
if (content.style.display === 'none') {
content.style.display = 'block'; // Show content
icon.textContent = '?'; // Down arrow indicates expanded
} else {
content.style.display = 'none'; // Hide content
icon.textContent = '?'; // Right arrow indicates collapsed
}
};

// Row hover effects for vulnerability table
// Provides visual feedback when users hover over table rows
const tableRows = document.querySelectorAll('.vuln-table tbody tr');
tableRows.forEach(row => {
// Mouse enters row - highlight with darker background
row.addEventListener('mouseenter', function() {
this.style.backgroundColor = '#212d46'; // Darker blue for hover state
});

// Mouse leaves row - restore original background
row.addEventListener('mouseleave', function() {
this.style.backgroundColor = '#1a2236'; // Original dark background
});
});

// Auto-submit forms on filter change
// Provides instant filtering without requiring submit button clicks
const filterSelects = document.querySelectorAll('select[name="severity"],
select[name="year"], select[name="month"]');
filterSelects.forEach(select => {
select.addEventListener('change', function() {
// Find the parent form element using DOM traversal
const form = this.closest('form');
if (form) {
form.submit(); // Automatically submit form when filter changes
}
});
});

// Search input handling
// Allows users to press Enter to submit search instead of clicking submit button
const searchInput = document.querySelector('input[name="q"]');
if (searchInput) { // Only attach if search input exists on page
searchInput.addEventListener('keypress', function(e) {
if (e.key === 'Enter') { // Check if Enter key was pressed
// Find parent form and submit search
const form = this.closest('form');
if (form) {
form.submit(); // Submit search form
}
}
});
}
});
```


templates\base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>{{ APP_NAME or "VulnEdu" }}</title>
<link rel="stylesheet" href="{{ url_for('static', filename='css/styles.css') }}" />
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<style>
body {
margin: 0;
font-family: 'Segoe UI', 'Arial', 'sans-serif';
background-color: #101828;
color: #f8fafc;
}
.sidebar {
width: 250px;
background: #1a2236;
color: #fff;
padding: 30px 0 0 0;
height: 100vh;
position: fixed;
left: 0; top: 0; bottom: 0;
box-shadow: 2px 0 14px rgba(0,0,0,0.1);
}
.sidebar-header {
font-weight: 700;
font-size: 1.5rem;
color: #F5E8D8;
text-align: center;
margin-bottom: 30px;
}
.sidebar-links {
padding-left: 24px;
display: flex;
flex-direction: column;
gap: 12px;
}
.sidebar-links a,
.sidebar-links .sidebar-parent {
color: #a9adc1;
text-decoration: none;
font-weight: 600;
padding: 8px 6px;
border-radius: 4px;
transition: background 0.2s, color 0.2s;
}
.sidebar-links a.active, .sidebar-links a:hover {
background: #263159;
color: #fff;
}
.sidebar-links .sidebar-parent {
padding-bottom: 4px;
}
.sidebar-submenu {
display: flex;
flex-direction: column;
gap: 4px;
margin-left: 20px;
margin-top: 3px;
}
.sidebar-submenu a {
font-weight: 520;
font-size: 0.97em;
padding: 7px 7px 7px 18px;
color: #a9adc1;
border-radius: 3px;
transition: background 0.2s, color 0.2s;
}
.sidebar-submenu a.active, .sidebar-submenu a:hover {
background: #263159;

```

```

color: #F5E8D8;
}
.mainwrap {
margin-left: 250px;
min-height: 100vh;
background: #101828;
box-sizing: border-box;
}
header {
padding: 0;
background: transparent;
color: inherit;
}
main {
max-width: 1500px;
margin: 0 auto;
padding: 36px 40px;
}
footer {
padding: 16px;
background: #151e37;
color: #94a3b8;
font-size: 0.91em;
text-align: right;
}
</style>
{% block head %}{% endblock %}
</head>
<body>
<aside class="sidebar">
<div class="sidebar-header">VulnEdu</div>
<div class="sidebar-links">
<a href="{{ url_for('main.index') }}" class="{% if request.endpoint == 'main.index' %}active{% endif %}">Dashboard</a>
<a href="{{ url_for('main.vulnerabilities') }}" class="{% if request.endpoint == 'main.vulnerabilities' %}active{% endif %}">Vulnerabilities</a>

<div class="sidebar-parent" style="font-size:1.05em; color:#a9adc1;">Learn</div>
<div class="sidebar-submenu">
<a href="{{ url_for('learn.learn_topic', topic='what-is-cwe') }}" class="{% if request.endpoint == 'learn.learn_topic' and request.view_args.topic == 'what-is-cwe' %}active{% endif %}">
What is a CWE?
</a>
<a href="{{ url_for('learn.learn_topic', topic='what-is-cve') }}" class="{% if request.endpoint == 'learn.learn_topic' and request.view_args.topic == 'what-is-cve' %}active{% endif %}">
What is a CVE?
</a>
<a href="{{ url_for('learn.learn_topic', topic='cvss-scores') }}" class="{% if request.endpoint == 'learn.learn_topic' and request.view_args.topic == 'cvss-scores' %}active{% endif %}">
Understanding CVSS Scores
</a>
<a href="{{ url_for('learn.learn_topic', topic='what-is-nvd-mitre') }}" class="{% if request.endpoint == 'learn.learn_topic' and request.view_args.topic == 'what-is-nvd-mitre' %}active{% endif %}">
What is NVD & MITRE?
</a>
<a href="{{ url_for('learn.learn_topic', topic='cve-vs-cwe-vs-cvss') }}" class="{% if request.endpoint == 'learn.learn_topic' and request.view_args.topic == 'cve-vs-cwe-vs-cvss' %}active{% endif %}">
CVE vs CWE vs CVSS
</a>
</div>

<a href="{{ url_for('utility.references') }}" class="{% if request.endpoint == 'utility.references' %}active{% endif %}">References & Resources</a>
</div>
</aside>

<div class="mainwrap">
<header></header>
<main>
{% block content %}{% endblock %}
</main>
<footer>© 2025 VulnEdu · Educational CVE Analysis Tool</footer>

```

</div>

<script src="{{ url_for('static', filename='js/main.js') }}"></script>
{% block scripts %}{% endblock %}
</body>
</html>

templates\references.html

```
{% extends "base.html" %}

{% block content %}
<div style="max-width:1400px;margin:0 auto 32px auto;padding-top:20px;">
<!-- Page Header -->
<h1 style="color:#63a4ff;">References & Resources</h1>
<p style="color:#e2e8f0;font-size:1.1em;line-height:1.6;margin-bottom:32px;">
This page contains all the authoritative sources, APIs, documentation,
and resources that power VulnEdu. Understanding these resources helps you dive deeper
into cybersecurity
vulnerability research and build your own security tools.
</p>

<!-- Quick Navigation - SCROLL TO SECTIONS ON SAME PAGE -->
<div style="background:#1a2236;border-radius:12px;padding:20px;margin-bottom:32px;">
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Quick Navigation</h3>
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(200px,1fr));gap:12px;">
<a href="#data-sources" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">?? Primary Data Sources</a>
<a href="#apis-feeds" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">? APIs & Data Feeds</a>
<a href="#tech-docs" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">?? Technical
Documentation</a>
<a href="#dev-tools" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">?? Development Tools</a>
</div>
</div>

<!-- INCLUDE YOUR EXISTING TEMPLATE FILES - CORRECTED FILENAMES -->
<div id="data-sources">
{% include 'references/data-sources.html' %}
</div>

<div id="apis-feeds">
{% include 'references/apis-feeds.html' %}
</div>

<div id="tech-docs">
{% include 'references/tech-docs.html' %}
</div>

<div id="dev-tools">
{% include 'references/dev_tools.html' %}
</div>

<!-- Attribution -->
<section style="background:#1a2236;border-radius:14px;padding:28px;">
<h2 style="color:#63a4ff;margin:0 0 20px 0;">Attribution & Acknowledgments</h2>
<div style="color:#e2e8f0;line-height:1.6;">
<p>VulnEdu is built using publicly available data from government and nonprofit
organizations dedicated to improving cybersecurity. We acknowledge:</p>
<ul style="margin-left:20px;color:#a9adc1;">
<li><strong>NIST</strong> for maintaining the National Vulnerability Database</li>
<li><strong>MITRE Corporation</strong> for the CVE and CWE programs</li>
<li><strong>FIRST.org</strong> for the CVSS scoring methodology</li>
<li><strong>OWASP</strong> for educational resources and best practices</li>
<li>The global <strong>cybersecurity research community</strong> for continuous
improvements</li>
</ul>
</div>
</section>
</div>

<div style="text-align:center; margin:2.5em 0 1em 0;">
<a href="{{ url_for('main.index') }}" style="color:#63a4ff; padding:0.8em 2em;
border-radius:6px; font-weight:600; text-decoration:none;">
? Back to Dashboard
</a>
</div>
```

```
<script>
// Smooth scrolling for anchor links
document.addEventListener('DOMContentLoaded', function() {
const links = document.querySelectorAll('a[href^="#"]');
for (const link of links) {
link.addEventListener('click', function(e) {
e.preventDefault();
const targetId = this.getAttribute('href').substring(1);
const targetElement = document.getElementById(targetId);
if (targetElement) {
targetElement.scrollIntoView({
behavior: 'smooth',
block: 'start'
});
}
});
}
});
</script>
{% endblock %}
```

templates\components\severity_cards.html

```
<div class="severity-cards-grid">

<!-- Critical severity card - highest priority vulnerabilities -->
<div class="severity-card" data-severity="CRITICAL" style="cursor: pointer;">
<!-- Dynamic count display - populated by server-side template engine -->
<div style="font-size:2.2rem;font-weight:800;color:#f47174;">
{{ metrics.CRITICAL }}
</div>
<!-- Severity label with consistent styling -->
<div style="color:#fff;margin-top:7px;font-size:0.95em;font-weight:600;">Critical</div>
</div>

<!-- High severity card - second priority level -->
<div class="severity-card" data-severity="HIGH" style="cursor: pointer;">
<!-- Orange color for high-attention vulnerabilities -->
<div style="font-size:2.2rem;font-weight:800;color:#fca311;">
{{ metrics.HIGH }}
</div>
<!-- Standard label formatting -->
<div style="color:#fff;margin-top:7px;font-size:0.95em;font-weight:600;">High</div>
</div>

<!-- Medium severity card - moderate priority vulnerabilities -->
<div class="severity-card" data-severity="MEDIUM" style="cursor: pointer;">
<!-- Blue color indicating medium-level attention needed -->
<div style="font-size:2.2rem;font-weight:800;color:#3b8ded;">
{{ metrics.MEDIUM }}
</div>
<!-- Consistent white text for readability -->
<div style="color:#fff;margin-top:7px;font-size:0.95em;font-weight:600;">Medium</div>
</div>

<!-- Low severity card - lowest priority vulnerabilities -->
<div class="severity-card" data-severity="LOW" style="cursor: pointer;">
<!-- Green color for low-priority items -->
<div style="font-size:2.2rem;font-weight:800;color:#42d392;">
{{ metrics.LOW }}
</div>
<!-- Final label with matching typography -->
<div style="color:#fff;margin-top:7px;font-size:0.95em;font-weight:600;">Low</div>
</div>

</div> <!-- End severity cards grid -->
```

templates\components\charts\line_chart.html

```
<div style="display: flex; gap: 25px;">

<!-- CVE Trends Daily SparkLine Card - compact 30-day trend visualization -->
<div class="trend-sevi-blocks" style="display: flex; flex-direction: column;
justify-content: center; background: #1a2236; border-radius: 18px; padding: 20px;
width: 380px; height: 110px;">

<!-- Chart title with blue accent color and consistent typography -->
<div style="font-weight: 700; color: #63a4ff; margin-bottom: 8px; font-size: 1.02rem;
letter-spacing: 0.15px; text-align: left;">
CVE Trends (Last 30 days)
</div>

<!-- Canvas element for sparkline chart - fixed height for compact display -->
<canvas id="dailyTrendChart" height="70"></canvas>
</div>

<!-- Vulnerabilities Over Time - main timeline chart section -->
<section style="background: #1a2236; border-radius: 14px; padding: 30px 30px 10px 30px;
flex: 2; display: flex; flex-direction: column; min-width: 350px; min-height: 250px;
height: 300px;">

<!-- Section title matching CVE card styling for visual consistency -->
<div style="font-weight: 700; color: #63a4ff; margin-bottom: 8px; font-size: 1.02rem;
letter-spacing: 0.15px; text-align: left;">
Vulnerabilities Over Time
</div>

<!-- Chart container with relative positioning for chart.js absolute positioning -->
<div style="position: relative; height: 250px;">
<!-- Main timeline chart canvas - JavaScript will target this ID -->
<canvas id="timelineChart"></canvas>
</div>
</section>

</div> <!-- End chart container -->
```

templates\components\charts\pie_chart.html

```
<section style="background:#1a2236; border-radius:14px; padding:10px 30px 20px 30px;
flex:1.00; display:flex; flex-direction:row; align-items:center; justify-content:center;
min-height:270px;">
<div style="display:flex; flex-direction:column; justify-content:center; height:150px;
min-width:280px;">
<div style="font-weight:700; color:#F5E8D8; margin-bottom:15px; font-size: 1.02rem;
letter-spacing:0.15px; text-align:left; width:100%;">
Severity Distribution
</div>
<div style="position:relative; width:230px; height:230px; min-width:230px; display:
flex; align-items:center; justify-content:center;">
<canvas id="severityPie" width="230" height="230"></canvas>
<div
style="position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);color:#e3e7f2;t
ext-align:center;pointer-events:none;width:110px;z-index:2;">
<span style="font-size:1.0rem;font-weight:500;line-height:1;">Total CVE's</span>
<br>
<span style="font-size:1.2rem;font-weight:500;letter-spacing: 1px;">{{ total_cves
}}</span>
</div>
</div>
</div>

<div style="display:flex; flex-direction:column; justify-content:center;
align-items:flex-start; margin-left: 5px; height:150px; min-width:125px; position:
relative; top:20px;">
<div style="display:flex;align-items:left;gap:8px;font-size:1.06em;
margin-bottom:16px;">
<span
style="display:inline-block;width:18px;height:18px;border-radius:50%;background:#f55855;
"></span>
<span style="color:#e3e7f2;min-width:67px;">Critical</span>
<span style="color:#e3e7f2; font-weight:500;">{{ severity_percentage.CRITICAL }}</span>
</div>
<div style="display:flex;align-items:left;gap:8px;font-size:1.06em;
margin-bottom:16px;">
<span
style="display:inline-block;width:18px;height:18px;border-radius:50%;background:#f8a541;
"></span>
<span style="color:#e3e7f2;min-width:67px;">High</span>
<span style="color:#e3e7f2; font-weight:500;">{{ severity_percentage.HIGH }}</span>
</div>
<div style="display:flex;align-items:left;gap:8px;font-size:1.06em;
margin-bottom:16px;">
<span
style="display:inline-block;width:18px;height:18px;border-radius:50%;background:#3b8ded;
"></span>
<span style="color:#e3e7f2;min-width:67px;">Medium</span>
<span style="color:#e3e7f2; font-weight:500;">{{ severity_percentage.MEDIUM }}</span>
</div>
<div style="display:flex;align-items:left;gap:8px;font-size:1.06em;
margin-bottom:16px;">
<span
style="display:inline-block;width:18px;height:18px;border-radius:50%;background:#42d392;
"></span>
<span style="color:#e3e7f2;min-width:67px;">Low</span>
<span style="color:#e3e7f2; font-weight:500;">{{ severity_percentage.LOW }}</span>
</div>
<div style="display:flex;align-items:left;gap:8px;font-size:1.06em;">
<span
style="display:inline-block;width:18px;height:18px;border-radius:50%;background:#6b7280;
"></span>
<span style="color:#e3e7f2;min-width:67px;">Unknown</span>
<span style="color:#e3e7f2; font-weight:500;">{{ severity_percentage.UNKNOWN }}</span>
</div>
</div>
</section>
```


templates\components\charts\radar_chart.html

```
<section style="background:#1a2236; border-radius:14px; padding:30px; flex:1;
min-height:430px;">
<div style="font-weight:700; color:#F5E8D8; margin-bottom:8px; font-size:1.02rem;
letter-spacing:0.15px; text-align:left;">
Vendor Risk Analysis
</div>

<div style="margin-bottom:12px; display:flex; gap:16px; flex-wrap:wrap;
align-items:center;">
<label>
<select id="radarCweCount" style="padding:5px 12px; border-radius:7px; border:1px solid
#263159; background:#222d44; color:#cdd7f7;">
<option value="5">Top 5 CWEs</option>
<option value="10" selected>Top 10 CWEs</option>
</select>
</label>
<label style="margin-left:8px;">
<input type="checkbox" id="radarWeighted" style="vertical-align:middle;">
<span style="color:#e2e8f0; font-size:0.99em;">Weight by Severity</span>
</label>
</div>

<div style="display: flex;">
<div style="position: relative; height:340px; margin-bottom:8px; width:60%;">
<canvas id="vendorRiskChart" width="400" height="320"></canvas>
</div>

<div id="radarReadout" style="height:340px; padding:18px 16px 8px 26px; display:block;
color:#afc1ff; font-size:0.95em; min-width:256px; width:50%;">
<div style="background:#232b45; border-radius:12px; padding:18px 20px 20px 20px;
color:#cdd7ff; margin-left:6px;" aria-labelledby="legendTitle" role="region"
tabindex="0">
<div style="display:flex;align-items:center;gap:8px;">
<span id="legendTitle" style="font-size:1.08em;font-weight:700;color:#F5E8D8;">
<svg id="legendInfoIcon" tabindex="0" aria-label="More info" width="20" height="20"
viewBox="0 0 20 20" style="cursor:pointer;margin-right:7px;vertical-align:-4px;">
<circle cx="10" cy="10" r="9" fill="#D4BA7F"/>
<text x="10" y="14" text-anchor="middle" font-size="13" font-weight="700"
fill="#fff">?</text>
</svg>
How to read this chart?
</span>
</div>

<ul style="margin:14px 0 0 18px; color:#afc1ff; font-size:0.98em;list-style:none;">
<li style="margin-bottom:9px;">
<span
style="font-family:sans-serif;font-size:1em;margin-right:8px;vertical-align:-1px;">?</sp
an>
Each axis is a <span style="color:#E6D5A8;font-weight:600;">CATEGORY</span>.
</li>
<li style="margin-bottom:9px;">
<span
style="font-family:sans-serif;font-size:1em;margin-right:8px;vertical-align:-1px;">?</sp
an>
The blue area <span style="color:#E6D5A8;font-weight:700;">shows concentration of
vulnerabilities</span> of the type!
</li>
<li style="margin-bottom:9px;">
<span
style="font-family:sans-serif;font-size:1em;margin-right:8px;vertical-align:-1px;">??</s
pan>
<span style="color:#f8a541;font-weight:700;">Click</span> on any spike for details or
filtering.
</li>
<li style="margin-bottom:9px;">
<span
style="font-family:sans-serif;font-size:1em;margin-right:8px;vertical-align:-1px;">??</s
pan>
<span style="color:#f55855;font-weight:700;">Long spikes highlight your <span
style="color:#f55855;">top risk areas</span>.</span>
</li>
```


templates\components\filters\search_filters.html

```
<form style="display:flex;gap:18px;" method="get">
<!-- Text search input for CVE identifiers and keywords -->
<input type="text"
name="q"
value="{{ search_query or '' }}"
placeholder="? Search CVEs"
style="padding:9px;font-size:1rem;background:#222d44;color:#cdd7f7;border:none;border-radius:7px;">

<!-- Severity level filter dropdown with state preservation -->
<select name="severity"
style="background:#222d44;color:#cdd7f7;padding:9px;border:none;border-radius:7px;">
<!-- Default option for no severity filtering -->
<option value="" {% if not severity_filter %}selected{% endif %}>All Severities</option>
<!-- Critical severity option with conditional selection -->
<option value="CRITICAL" {% if severity_filter == 'CRITICAL' %}selected{% endif
%}>Critical</option>
<!-- High severity option -->
<option value="HIGH" {% if severity_filter == 'HIGH' %}selected{% endif %}>High</option>
<!-- Medium severity option -->
<option value="MEDIUM" {% if severity_filter == 'MEDIUM' %}selected{% endif
%}>Medium</option>
<!-- Low severity option -->
<option value="LOW" {% if severity_filter == 'LOW' %}selected{% endif %}>Low</option>
</select>

<!-- Year filter dropdown with dynamic option generation -->
<select name="year"
style="background:#222d44;color:#cdd7f7;padding:9px;border:none;border-radius:7px;">
<!-- Default option for no year filtering -->
<option value="" {% if not year_filter %}selected{% endif %}>All Years</option>
<!-- Dynamic year options from server-provided available_years list -->
{% for y in available_years %}
<option value="{{ y }}" {% if year_filter == y %}selected{% endif %}>{{ y }}</option>
{% endfor %}
</select>

<!-- Month filter dropdown with formatted numeric display -->
<select name="month"
style="background:#222d44;color:#cdd7f7;padding:9px;border:none;border-radius:7px;">
<!-- Default option for no month filtering -->
<option value="" {% if not month_filter %}selected{% endif %}>All Months</option>
<!-- Dynamic month options with zero-padded formatting -->
{% for m in available_months %}
<option value="{{ m }}" {% if month_filter == m %}selected{% endif %}>{{
"%02d"|format(m) }}</option>
{% endfor %}
</select>

<!-- Submit button to apply all selected filter criteria - LIGHT PURPLE -->
<button type="submit"
style="background:#9370DB;color:white;border:none;padding:10px
18px;border-radius:7px;font-weight:700;">
Filter
</button>
</form> <!-- End CVE search and filter form -->
```

templates\layouts\dashboard_layout.html

```
{% extends "base.html" %}

<!-- Head block injection for dashboard-specific styling -->
{% block head %}
<style>
/* Dashboard-specific styles embedded in template for co-location and performance */

/* Main dashboard grid container - vertical organization with consistent spacing */
.dashboard-grid {
display: grid;
gap: 25px;
margin-bottom: 25px;
}

/* Top dashboard row - horizontal flex layout for metrics and quick stats */
.dashboard-row-1 {
display: flex;
gap: 25px;
}

/* Main dashboard row - horizontal flex with stretch alignment for equal heights */
.dashboard-row-2 {
display: flex;
gap: 25px;
align-items: stretch;
}

/* Trend visualization blocks - compact containers for sparklines and metrics */
.trend-sevi-blocks {
display: flex;
flex-direction: column;
justify-content: center;
background: #1a2236;
border-radius: 18px;
padding: 20px;
width: 380px;
height: 110px;
}

/* Severity cards grid - 4-column layout for vulnerability severity display */
.severity-cards-grid {
display: grid;
grid-template-columns: repeat(4, 1fr);
gap: 15px;
flex: 1;
align-items: stretch;
}

/* Individual severity card styling - interactive cards with hover effects */
.severity-card {
background: #1a2236;
cursor: pointer;
border-radius: 18px;
padding: 20px;
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
transition: box-shadow 0.2s;
}

/* Responsive breakpoint for tablets and smaller laptops */
@media (max-width: 1200px) {
/* Convert horizontal dashboard rows to vertical stacking */
.dashboard-row-1,
.dashboard-row-2 {
flex-direction: column;
}

/* Reduce severity cards from 4 to 2 columns for better mobile readability */
.severity-cards-grid {
grid-template-columns: repeat(2, 1fr);
}
```

```
}  
}  
}</style>  
{% endblock %}  
  
<!-- Content block placeholder - child templates inject dashboard components here -->  
{% block content %}  
{% endblock %}
```

templates\layouts\learn_layout.html

```
<!-- Learning module template extending base site template for consistent navigation -->
{% extends "base.html" %}

<!-- Head block injection for learning platform-specific styling -->
{% block head %}
<style>
/* Learning platform-specific styles embedded for co-location and performance */

/* Main learning container - centered with maximum width constraint */
.learn-container {
max-width: 1400px;
margin: 0 auto 32px auto;
padding-top: 20px;
}

/* Two-column CSS Grid layout for sidebar navigation and main content */
.learn-grid {
display: grid;
grid-template-columns: 300px 1fr;
gap: 36px;
}

/* Left sidebar container for CWE navigation and controls */
.learn-sidebar {
background: #1a2236;
border-radius: 14px;
padding: 18px 0 18px 0;
height: fit-content;
}

/* Main content area for educational materials and documentation */
.learn-content {
background: #1a2236;
border-radius: 14px;
padding: 28px 26px;
min-height: 390px;
}

/* Scrollable container for CWE list with maximum height constraint */
.cwe-list-scroll {
max-height: 550px;
overflow-y: auto;
list-style: none;
padding: 0;
margin: 0;
}

/* Individual CWE list item styling with interactive states */
.cwe-list-item {
padding: 8px 24px;
cursor: pointer;
font-weight: 500;
color: #e3e7f2;
user-select: none;
border-left: 4px solid transparent;
transition: background 0.13s, border-left 0.13s, color 0.13s;
}

/* Hover state for CWE list items - provides immediate visual feedback */
.cwe-list-item:hover {
background: #212d46;
color: #ffffff;
}

/* Selected state for CWE list items - blue accent border and text */
.cwe-list-item.selected {
background: #212d46;
border-left: 4px solid #63a4ff;
color: #63a4ff;
}

/* Toggle button styling with micro-interaction effects */
```

```

.toggle-button {
margin: 16px 0 0 24px;
background: #263159;
color: #63a4ff;
padding: 8px 18px;
border-radius: 8px;
border: none;
cursor: pointer;
font-weight: 600;
font-size: 0.97em;
transition: background 0.2s, transform 0.1s;
}

/* Toggle button hover state with subtle lift effect */
.toggle-button:hover {
background: #2d3d6b;
transform: translateY(-1px);
}

/* Responsive breakpoint for tablets and mobile devices */
@media (max-width: 1024px) {
/* Convert two-column grid to single-column stacking layout */
.learn-grid {
grid-template-columns: 1fr;
gap: 24px;
}
}
</style>
{% endblock %}

<!-- Content block placeholder - child templates inject learning content here -->
{% block content %}
{% endblock %}

```

templates\learn\cve-vs-cwe-vs-cvss.html

```
{% extends "base.html" %}
{% block content %}

<!-- Embedded CSS styles for this specific educational page -->
<style>
/* Container for Venn diagram and legend layout */
.venn-container {
display: flex;
align-items: center;
justify-content: center;
margin: 22px 0;
gap: 40px; /* Space between diagram and legend */
}

/* Main Venn diagram container with relative positioning */
.venn-diagram {
position: relative;
width: 450px;
height: 350px;
padding: 20px;
}

/* Base circle styling for all three frameworks */
.circle {
position: absolute; /* Allows precise positioning for overlap */
border-radius: 50%; /* Creates perfect circles */
opacity: 0.85; /* Transparency for intersection visibility */
display: flex;
align-items: center;
justify-content: center;
font-weight: bold;
font-size: 12px;
text-align: center;
line-height: 1.2;
border: 3px solid; /* Border color set individually */
}

/* CVE circle - Blue theme for vulnerability identifiers */
.cve-circle {
width: 180px;
height: 180px;
background: rgba(99, 164, 255, 0.6); /* Semi-transparent blue */
border-color: #63a4ff;
top: 30px;
left: 60px; /* Positioned to overlap with other circles */
color: #63a4ff;
}

/* CWE circle - Teal theme for weakness categories */
.cwe-circle {
width: 180px;
height: 180px;
background: rgba(56, 178, 172, 0.6); /* Semi-transparent teal */
border-color: #38b2ac;
top: 30px;
left: 190px; /* Overlaps with CVE circle */
color: #38b2ac;
}

/* CVSS circle - Red theme for severity scoring */
.cvss-circle {
width: 180px;
height: 180px;
background: rgba(245, 101, 101, 0.6); /* Semi-transparent red */
border-color: #f56565;
top: 150px;
left: 125px; /* Positioned to create triple intersection */
color: #f56565;
}

/* Framework name labels positioned on circles */
.label {
```



```

position: absolute;
font-weight: bold;
font-size: 14px;
z-index: 10; /* Appear above circles */
}

/* Individual label positioning for each framework */
.cve-label {
top: 70px;
left: 105px;
color: #63a4ff;
}

.cwe-label {
top: 70px;
left: 255px;
color: #38b2ac;
}

.cvss-label {
top: 250px;
left: 165px;
color: #f56565;
}

/* Example identifiers for each framework */
.example {
position: absolute;
font-size: 10px;
background: rgba(0, 0, 0, 0.8); /* Dark background for readability */
padding: 3px 6px;
border-radius: 4px;
white-space: nowrap; /* Prevent text wrapping */
z-index: 5;
}

/* Example positioning for each framework */
.cve-example {
top: 95px;
left: 75px;
color: #e2e8f0;
}

.cwe-example {
top: 95px;
left: 270px;
color: #e2e8f0;
}

.cvss-example {
top: 275px;
left: 150px;
color: #e2e8f0;
}

/* Labels for intersection areas showing relationships */
.intersection-label {
position: absolute;
font-size: 10px;
background: rgba(255, 255, 255, 0.9); /* High contrast for visibility */
color: #1a202c;
padding: 3px 6px;
border-radius: 4px;
font-weight: 600;
z-index: 15; /* Above all other elements */
}

/* Positioning for two-way intersections */
.cve-cwe {
top: 115px;
left: 185px;
}

.cve-cvss {
top: 175px;
left: 115px;
}

```

```

.cwe-cvss {
top: 175px;
left: 255px;
}

/* Center intersection where all three frameworks meet */
.center-intersection {
top: 165px;
left: 170px;
background: rgba(255, 215, 0, 0.9); /* Gold for emphasis */
color: #1a202c;
font-size: 9px;
}

/* Legend styling with cybersecurity theme */
.legend {
color: #e3e8f0;
background: linear-gradient(135deg, #232b45 0%, #1e2640 100%); /* Dark gradient */
border-radius: 12px;
box-shadow: 0 6px 24px rgba(0,0,0,0.3); /* Subtle shadow */
border: 1px solid rgba(99, 164, 255, 0.1);
padding: 18px 24px;
margin: 0 0 18px 0;
width: 350px;
}

/* Legend title styling */
.legend-title {
font-weight: bold;
font-size: 16px;
margin-bottom: 14px;
color: #cdd7ff;
}

/* Individual legend items layout */
.legend-item {
display: flex;
align-items: center;
margin-bottom: 8px;
}

/* Small colored circles in legend */
.legend-circle {
width: 16px;
height: 16px;
border-radius: 50%;
margin-right: 12px;
border: 2px solid #333;
flex-shrink: 0; /* Maintain size */
}

/* Legend circle colors matching main diagram */
.legend-cve {
background: rgba(99, 164, 255, 0.6);
border-color: #63a4ff;
}

.legend-cwe {
background: rgba(56, 178, 172, 0.6);
border-color: #38b2ac;
}

.legend-cvss {
background: rgba(245, 101, 101, 0.6);
border-color: #f56565;
}

/* Legend text styling */
.legend-text {
font-size: 13px;
}

.legend-name {
margin-bottom: 0;
}

```

```

/* Warning items get red color treatment */
.warning-item .legend-name {
color: #e74c3c;
}
</style>

<!-- Main content container with max width and centering -->
<div style="max-width:1400px;margin:0 auto 32px auto;padding-top:18px;">
<!-- Page title with CVE blue color scheme -->
<h1 style="color:#63a4ff;">CVE vs CWE vs CVSS</h1>

<!-- Introduction paragraph explaining the three frameworks -->
<p style="color:#e2e8f0;font-size:1.07em;">
Vulnerability analysis uses three frame-works: <b>CVE</b>, <b>CWE</b>, and <b>CVSS</b>
each provides a unique way to understand risk and weaknesses.
</p>

<!-- Comparison table showing framework roles and examples -->
<table style="color:#e3e8f0;background:linear-gradient(135deg, #232b45 0%, #1e2640
100%);border-radius:12px;width:75%;margin-bottom:18px; border-collapse:collapse;border:
1px solid rgba(99, 164, 255, 0.1);box-shadow: 0 6px 24px rgba(0,0,0,0.3);">
<!-- Table header with framework categories -->
<tr style="background:#232b45;">
<th style="text-align:center;padding:12px 8px;border-radius:12px 0 0 0;">Framework</th>
<th style="text-align:center;padding:12px 8px;">Role in Security</th>
<th style="text-align:center;padding:12px 8px;border-radius:0 12px 0 0;">Sample</th>
</tr>
<!-- CVE row with blue color scheme -->
<tr style="background:rgba(38, 49, 89, 0.4);">
<td style="text-align:center;padding:10px 8px;"><strong
style="color:#63a4ff;">CVE</strong></td>
<td style="text-align:center;padding:10px 8px;">Specific vulnerability identifier</td>
<td style="text-align:center;padding:10px 8px;">CVE-2021-34527</td>
</tr>
<!-- CWE row with teal color scheme -->
<tr style="background:rgba(38, 49, 89, 0.4);">
<td style="text-align:center;padding:10px 8px;"><strong
style="color:#38b2ac;">CWE</strong></td>
<td style="text-align:center;padding:10px 8px;">Type/classification of weakness</td>
<td style="text-align:center;padding:10px 8px;">CWE-79 (Cross-Site Scripting)</td>
</tr>
<!-- CVSS row with red color scheme -->
<tr style="background:rgba(38, 49, 89, 0.4);">
<td style="text-align:center;padding:10px 8px;border-radius:0 0 0 12px;"><strong
style="color:#f56565;">CVSS</strong></td>
<td style="text-align:center;padding:10px 8px;">Numerical severity score</td>
<td style="text-align:center;padding:10px 8px;border-radius:0 0 12px 0;">9.8
(Critical)</td>
</tr>
</table>

<!-- Section explaining framework relationships -->
<h2 style="color:#cdd7ff;margin-top:32px;">How Are They Connected?</h2>
<ul style="color:#e3e8f0;margin-left:17px;">
<li><b>CVE</b> records the bug.</li>
<li><b>CWE</b> classifies what kind of bug it is.</li>
<li><b>CVSS</b> tells you how dangerous it is.</li>
</ul>

<!-- Venn diagram section header -->
<h2 style="color:#cdd7ff;">Visual Relationship (Venn Diagram)</h2>
<div class="venn-container">
<!-- Main Venn diagram with three overlapping circles -->
<div class="venn-diagram">
<!-- Three framework circles with transparency for overlaps -->
<div class="circle cve-circle"></div>
<div class="circle cwe-circle"></div>
<div class="circle cvss-circle"></div>

<!-- Framework name labels positioned on circles -->
<div class="label cve-label">CVE</div>
<div class="label cwe-label">CWE</div>
<div class="label cvss-label">CVSS</div>

<!-- Example identifiers for each framework -->
<div class="example cve-example">CVE-2021-34527</div>
<div class="example cwe-example">CWE-79</div>

```

```

<div class="example cvss-example">9.8 Critical</div>

<!-- Labels showing what each intersection represents -->
<div class="intersection-label cve-cwe">ID + Type</div>
<div class="intersection-label cve-cvss">ID + Score</div>
<div class="intersection-label cwe-cvss">Type + Risk</div>
<div class="intersection-label center-intersection">Complete Analysis</div>
</div>

<!-- Legend explaining how to interpret the diagram -->
<div class="legend">
<div class="legend-title">How to read this diagram?</div>
<!-- General explanation of frameworks -->
<div class="legend-item cve-item">
<div class="legend-text">
<div class="legend-name">Each circle is a <strong style="color:
#3498db;">FRAMEWORK</strong>.</div>
</div>
</div>
<!-- CVE framework explanation with color coding -->
<div class="legend-item cve-item">
<div class="legend-circle legend-cve"></div>
<div class="legend-text">
<div class="legend-name"><strong style="color:#63a4ff;">CVE</strong> - Specific
vulnerability ID</div>
</div>
</div>
<!-- CWE framework explanation -->
<div class="legend-item cwe-item">
<div class="legend-circle legend-cwe"></div>
<div class="legend-text">
<div class="legend-name"><strong style="color:#38b2ac;">CWE</strong> - Weakness
category/type</div>
</div>
</div>
<!-- CVSS framework explanation -->
<div class="legend-item cvss-item">
<div class="legend-circle legend-cvss"></div>
<div class="legend-text">
<div class="legend-name"><strong style="color:#f56565;">CVSS</strong> - Severity score
(0-10)</div>
</div>
</div>
<!-- Explanation of intersection meanings -->
<div style="margin-top: 14px; font-size: 12px; color: #a9adc1;">
<strong>Overlaps show relationships:</strong><br>
? <strong>CVE + CWE:</strong> Specific vuln with its type<br>
? <strong>CVE + CVSS:</strong> Specific vuln with severity<br>
? <strong>CWE + CVSS:</strong> Weakness type and risk level<br>
? <strong>Center:</strong> Complete vulnerability analysis
</div>
</div>
</div>

<!-- Real-world example using Log4Shell vulnerability -->
<h2 style="color:#cdd7ff;margin-top:32px;">Real-World Example: Log4Shell</h2>
<div
style="background:#232b45;border-radius:12px;padding:24px;margin-bottom:24px;border-left
:4px solid #f47174;">
<!-- Three-column grid showing each framework's role -->
<div style="display:grid;grid-template-columns:1fr 1fr 1fr;gap:20px;color:#e3e8f0;">
<!-- CVE component breakdown -->
<div style="background:#263159;padding:18px;border-radius:8px;border-top:3px solid
#63a4ff;">
<h4 style="color:#63a4ff;margin:0 0 10px 0;">CVE Component</h4>
<div
style="font-family:monospace;color:#cdd7ff;font-weight:bold;margin-bottom:8px;">CVE-2021-
44228</div>
<div style="font-size:0.95em;line-height:1.4;">
This is the unique identifier that security teams, researchers, and databases use to
track this specific vulnerability. When someone says "CVE-2021-44228," everyone knows
exactly which bug they're talking about.
</div>
</div>
<!-- CVE component breakdown -->
<div style="background:#263159;padding:18px;border-radius:8px;border-top:3px solid
#38b2ac;">

```

```

<h4 style="color:#38b2ac;margin:0 0 10px 0;">CWE Component</h4>
<div
style="font-family:monospace;color:#cdd7ff;font-weight:bold;margin-bottom:8px;">CWE-502:
Deserialization</div>
<div style="font-size:0.95em;line-height:1.4;">
This tells us the <em>type</em> of weakness. It's a deserialization flaw?meaning the
software unsafely processes external data. This helps developers understand the pattern
and prevent similar bugs.
</div>
</div>
<!-- CVSS component breakdown -->
<div style="background:#263159;padding:18px;border-radius:8px;border-top:3px solid
#f47174;">
<h4 style="color:#f47174;margin:0 0 10px 0;">CVSS Component</h4>
<div
style="font-family:monospace;color:#cdd7ff;font-weight:bold;margin-bottom:8px;">CVSS
3.1: 10.0 (Critical)</div>
<div style="font-size:0.95em;line-height:1.4;">
This tells us <em>how dangerous</em> it is. A 10.0 score means maximum severity?easy to
exploit, severe impact. This helps prioritize patching efforts.
</div>
</div>
</div>
<!-- Summary showing how all frameworks work together -->
<div
style="margin-top:20px;padding:16px;background:#1a2236;border-radius:8px;color:#a9adc1;"
>
<strong style="color:#fca311;">Together:</strong> CVE-2021-44228 is a critical (CVSS
10.0) deserialization vulnerability (CWE-502) in Apache Log4j2 that allows remote code
execution. The combination tells the complete story: what it is, how bad it is, and what
type of fix is needed.
</div>
</div>

<!-- Practical applications for security teams -->
<h2 style="color:#cdd7ff;">Why This Matters for Security Teams</h2>
<div style="display:grid;grid-template-columns:1fr 1fr;gap:24px;margin-bottom:24px;">
<!-- Incident response applications -->
<div style="background:#1a2236;border-radius:12px;padding:24px;">
<h3 style="color:#42d392;margin:0 0 16px 0;">For Incident Response</h3>
<ul style="color:#e3e8f0;line-height:1.6;margin-left:16px;">
<li><strong>CVE ID</strong> helps you quickly find patches, advisories, and detection
rules</li>
<li><strong>CWE type</strong> suggests what kind of exploitation to look for in
logs</li>
<li><strong>CVSS score</strong> helps you prioritize which vulnerabilities to address
first</li>
</ul>
</div>
<!-- Development team applications -->
<div style="background:#1a2236;border-radius:12px;padding:24px;">
<h3 style="color:#3b8ded;margin:0 0 16px 0;">For Development Teams</h3>
<ul style="color:#e3e8f0;line-height:1.6;margin-left:16px;">
<li><strong>CVE examples</strong> show real-world consequences of coding mistakes</li>
<li><strong>CWE patterns</strong> guide secure coding practices and code reviews</li>
<li><strong>CVSS metrics</strong> help understand which flaws pose the greatest business
risk</li>
</ul>
</div>
</div>

<!-- Common misunderstandings section -->
<h2 style="color:#cdd7ff;">Common Misconceptions</h2>
<div style="background:#232b45;border-radius:12px;padding:24px;margin-bottom:24px;">
<div style="display:grid;gap:16px;">
<!-- Misconception about CVE vs CWE -->
<div style="padding:16px;background:#2d1b1e;border-left:4px solid
#f47174;border-radius:0 8px 8px 0;">
<div style="color:#f47174;font-weight:bold;margin-bottom:8px;">? "CVE and CWE are the
same thing"</div>
<div style="color:#e3e8f0;">CVE identifies a specific vulnerability instance, while CWE
categorizes the type of weakness. One CWE can apply to thousands of different
CVEs.</div>
</div>
<!-- Misconception about CVSS accuracy -->
<div style="padding:16px;background:#2d1b1e;border-left:4px solid
#f47174;border-radius:0 8px 8px 0;">

```

```

<div style="color:#f47174;font-weight:bold;margin-bottom:8px;">? "CVSS scores are always accurate"</div>
<div style="color:#e3e8f0;">CVSS provides a baseline, but environmental factors matter. A "Low" score vulnerability might be "Critical" in your specific environment.</div>
</div>
<!-- Misconception about only focusing on high severity -->
<div style="padding:16px;background:#2d1b1e;border-left:4px solid #f47174;border-radius:0 8px 8px 0;">
<div style="color:#f47174;font-weight:bold;margin-bottom:8px;">? "You only need to worry about Critical/High CVEs"</div>
<div style="color:#e3e8f0;">Medium and Low severity vulnerabilities can be chained together for major impact. Consider the full attack surface.</div>
</div>
</div>
</div>

<!-- Practical quick reference guide -->
<h2 style="color:#cdd7ff;">Quick Reference Cheat Sheet</h2>
<div style="background: linear-gradient(135deg, #232b45 0%, #1e2640 100%);border-radius:12px;padding:24px;margin-bottom:24px;border: 1px solid rgba(99, 164, 255, 0.1);">
<!-- Responsive grid layout for different screen sizes -->
<div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(280px,1fr));gap:20px;">
<!-- CVE workflow guidance -->
<div style="background:#263159;padding:16px;border-radius:8px;">
<h4 style="color:#63a4ff;margin:0 0 12px 0;">When you see a CVE...</h4>
<div style="color:#e3e8f0;font-size:0.95em;line-height:1.5;">
<strong>1.</strong> Check if it affects your software<br>
<strong>2.</strong> Look up the CVSS score for priority<br>
<strong>3.</strong> Review the CWE to understand the root cause<br>
<strong>4.</strong> Find patches or mitigations in references
</div>
</div>
<!-- CWE workflow guidance -->
<div style="background:#263159;padding:16px;border-radius:8px;">
<h4 style="color:#38b2ac;margin:0 0 12px 0;">When you see a CWE...</h4>
<div style="color:#e3e8f0;font-size:0.95em;line-height:1.5;">
<strong>1.</strong> Learn the weakness pattern<br>
<strong>2.</strong> Review your code for similar issues<br>
<strong>3.</strong> Implement preventive measures<br>
<strong>4.</strong> Update coding standards and reviews
</div>
</div>
<!-- CVSS workflow guidance -->
<div style="background:#263159;padding:16px;border-radius:8px;">
<h4 style="color:#f47174;margin:0 0 12px 0;">When you see a CVSS score...</h4>
<div style="color:#e3e8f0;font-size:0.95em;line-height:1.5;">
<strong>1.</strong> Use it for initial prioritization<br>
<strong>2.</strong> Consider your environment specifics<br>
<strong>3.</strong> Factor in business impact<br>
<strong>4.</strong> Don't ignore lower scores entirely
</div>
</div>
</div>
</div>
</div>

<!-- Summary reminder box emphasizing framework integration -->
<div style="background:#1a2236;border-radius:12px;padding:20px;text-align:center;color:#a9adc1;">
<div style="font-size:1.1em;margin-bottom:8px;">
<strong style="color:#cdd7ff;">Remember:</strong> These frameworks work together
</div>
<div>
Use CVE for tracking, CWE for understanding, and CVSS for prioritizing.
The combination gives you the complete picture needed for effective vulnerability management.
</div>
</div>
</div>
</div>

<!-- Navigation back to main dashboard using Flask URL generation -->
<div style="text-align:center; margin:2.5em 0 1em 0;">
<!-- Dynamic URL generation ensures proper routing -->
<a href="{{ url_for('main.index') }}" style="color:#63a4ff; padding:0.8em 2em; border-radius:6px; font-weight:600; text-decoration:none;">

```

[? Back to Dashboard](#)

</div>

<!-- End of content block -->

{% endblock %}

templates\learn\cvss-scores.html

```
{% extends "base.html" %}
{% block content %}

<!-- Main container with responsive max-width -->
<div style="max-width:1400px;margin:0 auto 32px auto;padding-top:20px;">

<!-- Page title with CVSS blue theme -->
<h1 style="color:#63a4ff;">Understanding CVSS Scores</h1>

<!-- Introduction explaining CVSS purpose and usage -->
<div style="color:#e2e8f0; font-size:1.07em; margin-bottom:28px; margin-top:9px;
line-height:1.66;">
CVSS scores help you measure, compare, and prioritize vulnerabilities. Use the range and
severity to decide: Patch now, or monitor?
</div>

<!-- Section header for score breakdown -->
<h2 style="color:#cdd7ff;">Score Breakdown</h2>

<!-- Interactive table showing CVSS score ranges and severities -->
<table id="cvss-score-table" style="color:#e3e8f0;background:linear-gradient(135deg,
#232b45 0%, #1e2640 100%);border-radius:12px;width:75%;margin-bottom:18px;
border-collapse:collapse;border: 1px solid rgba(99, 164, 255, 0.1);box-shadow: 0 6px
24px rgba(0,0,0,0.3);">

<!-- Table header with column titles -->
<tr style="background:#232b45;">
<th style="text-align:center;padding:12px 8px;border-radius:12px 0 0 0;">Score
Range</th>
<th style="text-align:center;padding:12px 8px;">Severity</th>
<th style="text-align:center;padding:12px 8px;">What this means</th>
<th style="text-align:center;padding:12px 8px;border-radius:0 12px 0 0;">Examples</th>
</tr>

<!-- Critical severity row (9.0-10.0) with red color scheme -->
<tr class="cvss-score-row" data-severity="CRITICAL" tabindex="0"
style="background:rgba(38, 49, 89, 0.4);cursor:pointer;">
<td style="text-align:center;padding:10px 8px;">9.0 ? 10.0</td>
<td style="text-align:center;padding:10px 8px;">
<span class="cvss-badger" style="background:#f55855;">Critical</span>
</td>
<td class="cvss-meaning-cell" style="text-align:center;padding:10px 8px;">Immediate
action needed.</td>
<td style="text-align:center;padding:10px 8px;"><a href="{{
url_for('main.vulnerabilities', severity='CRITICAL') }}" class="cvss-link-cve"
style="color:#f55855;text-decoration:underline;">Show Critical CVEs</a></td>
</tr>

<!-- High severity row (7.0-8.9) with orange color scheme -->
<tr class="cvss-score-row" data-severity="HIGH" tabindex="0" style="background:rgba(38,
49, 89, 0.4);cursor:pointer;">
<td style="text-align:center;padding:10px 8px;">7.0 ? 8.9</td>
<td style="text-align:center;padding:10px 8px;">
<span class="cvss-badger" style="background:#f8a541;">High</span>
</td>
<td class="cvss-meaning-cell" style="text-align:center;padding:10px 8px;">Patch as soon
as possible.</td>
<td style="text-align:center;padding:10px 8px;"><a href="{{
url_for('main.vulnerabilities', severity='HIGH') }}" class="cvss-link-cve"
style="color:#f8a541;text-decoration:underline;">Show High CVEs</a></td>
</tr>

<!-- Medium severity row (4.0-6.9) with blue color scheme -->
<tr class="cvss-score-row" data-severity="MEDIUM" tabindex="0"
style="background:rgba(38, 49, 89, 0.4);cursor:pointer;">
<td style="text-align:center;padding:10px 8px;">4.0 ? 6.9</td>
<td style="text-align:center;padding:10px 8px;">
<span class="cvss-badger" style="background:#3b8ded;">Medium</span>
</td>
<td class="cvss-meaning-cell" style="text-align:center;padding:10px 8px;">Review and
patch soon.</td>
```



```

<td style="text-align:center;padding:10px 8px;"><a href="{url_for('main.vulnerabilities', severity='MEDIUM') }}" class="cvss-link-cve" style="color:#3b8ded;text-decoration:underline;">Show Medium CVEs</a></td>
</tr>

<!-- Low severity row (0.1-3.9) with green color scheme -->
<tr class="cvss-score-row" data-severity="LOW" tabindex="0" style="background:rgba(38, 49, 89, 0.4);cursor:pointer;">
<td style="text-align:center;padding:10px 8px;border-radius:0 0 0 12px;">0.1 ? 3.9</td>
<td style="text-align:center;padding:10px 8px;">
<span class="cvss-badger" style="background:#42d392;">Low</span>
</td>
<td class="cvss-meaning-cell" style="text-align:center;padding:10px 8px;">Monitor for now.</td>
<td style="text-align:center;padding:10px 8px;border-radius:0 0 12px 0;"><a href="{url_for('main.vulnerabilities', severity='LOW') }}" class="cvss-link-cve" style="color:#42d392;text-decoration:underline;">Show Low CVEs</a></td>
</tr>
</table>

<!-- Educational section explaining CVSS methodology -->
<h2 style="color:#cdd7ff;">How CVSS v3.1 Works</h2>
<div style="background:#232b45;padding:24px;border-radius:12px;margin-bottom:28px;border:1px solid rgba(99, 164, 255, 0.1);">

<!-- Introduction to CVSS calculation process -->
<div style="color:#e2e8f0;line-height:1.6;margin-bottom:20px;">
<p>CVSS (Common Vulnerability Scoring System) v3.1 calculates vulnerability severity using 8 metrics across 3 groups:</p>
</div>

<!-- Three-column grid showing CVSS metric groups -->
<div style="display:grid;grid-template-columns:1fr 1fr 1fr;gap:24px;margin-bottom:24px;">

<!-- Exploitability metrics group -->
<div style="background:#263159;border-radius:8px;padding:20px;border-top:3px solid #63a4ff;">
<h3 style="color:#63a4ff;margin:0 0 12px 0;text-align:center;">Exploitability Metrics</h3>
<div style="color:#e2e8f0;font-size:0.95em;line-height:1.5;">
<p style="margin-bottom:8px;"><strong>How easy is it to exploit?</strong></p>
<ul style="margin:0;padding-left:16px;color:#a9adc1;">
<li>Attack Vector</li>
<li>Attack Complexity</li>
<li>Privileges Required</li>
<li>User Interaction</li>
</ul>
</div>
</div>

<!-- Impact metrics group -->
<div style="background:#263159;border-radius:8px;padding:20px;border-top:3px solid #42d392;">
<h3 style="color:#42d392;margin:0 0 12px 0;text-align:center;">Impact Metrics</h3>
<div style="color:#e2e8f0;font-size:0.95em;line-height:1.5;">
<p style="margin-bottom:8px;"><strong>What damage can be done?</strong></p>
<ul style="margin:0;padding-left:16px;color:#a9adc1;">
<li>Confidentiality Impact</li>
<li>Integrity Impact</li>
<li>Availability Impact</li>
</ul>
</div>
</div>

<!-- Scope metric group -->
<div style="background:#263159;border-radius:8px;padding:20px;border-top:3px solid #f8a541;">
<h3 style="color:#f8a541;margin:0 0 12px 0;text-align:center;">Scope</h3>
<div style="color:#e2e8f0;font-size:0.95em;line-height:1.5;">
<p style="margin-bottom:8px;"><strong>Boundary consideration</strong></p>
<ul style="margin:0;padding-left:16px;color:#a9adc1;">
<li>Unchanged: Same component</li>
<li>Changed: Affects other components</li>
</ul>
</div>
</div>

```

```

</div>
</div>

<!-- Mathematical formula explanation -->
<div style="background:#1a2236;padding:12px;border-radius:8px;border-left:4px solid
#63a4ff;font-family:monospace;font-size:0.9em;">
<strong>BaseScore = min(10, Impact + Exploitability)</strong><br>
Where Impact and Exploitability are calculated from the 8 metrics above.
</div>
</div>

<!-- Interactive calculator section -->
<h2 style="color:#cdd7ff;">CVSS v3.1 Calculator <span
style="font-size:0.82em;color:#a9adc1;">(interactive demo)</span></h2>

<!-- Main calculator wrapper with styling -->
<div id="cvss-calc-wrapper" style="background:linear-gradient(135deg,#232b45 0%, #1e2640
100%);padding:24px;border-radius:12px;margin-bottom:28px;box-shadow:0 6px 24px
rgba(0,0,0,0.3);border:1px solid rgba(99, 164, 255, 0.1);">

<!-- Three-column grid for metric input groups -->
<div style="display:grid;grid-template-columns:1fr 1fr
1fr;gap:24px;margin-bottom:32px;">

<!-- Exploitability metrics input group -->
<div style="background:#263159;border-radius:8px;padding:20px;border-top:3px solid
#63a4ff;">
<h3 style="color:#63a4ff;margin:0 0 16px 0;text-align:center;">Exploitability</h3>
<form id="cvss-calc-form" autocomplete="off">
<div style="display:flex;flex-direction:column;gap:12px;">

<!-- Attack Vector dropdown with tooltip -->
<div class="cvss-input-group">
<label class="cvss-label">Attack Vector <span class="cvss-info" title="How can the
attacker reach the vulnerable component?"></span></label>
<select name="AV" class="cvss-select">
<option value="N">? Network (Remote)</option>
<option value="A">? Adjacent Network</option>
<option value="L">? Local Access</option>
<option value="P">? Physical Access</option>
</select>
</div>

<!-- Attack Complexity dropdown -->
<div class="cvss-input-group">
<label class="cvss-label">Attack Complexity <span class="cvss-info" title="How difficult
is the attack to execute?"></span></label>
<select name="AC" class="cvss-select">
<option value="L">? Low (Easy to exploit)</option>
<option value="H">? High (Complex conditions)</option>
</select>
</div>

<!-- Privileges Required dropdown -->
<div class="cvss-input-group">
<label class="cvss-label">Privileges Required <span class="cvss-info" title="What access
level does attacker need?"></span></label>
<select name="PR" class="cvss-select">
<option value="N">? None</option>
<option value="L">? Low (Basic user)</option>
<option value="H">? High (Admin privileges)</option>
</select>
</div>

<!-- User Interaction dropdown -->
<div class="cvss-input-group">
<label class="cvss-label">User Interaction <span class="cvss-info" title="Does a user
need to do something?"></span></label>
<select name="UI" class="cvss-select">
<option value="N">? None (Automatic)</option>
<option value="R">? Required (User must click/act)</option>
</select>
</div>
</div>
</div>

<!-- Impact metrics input group -->

```

```

<div style="background:#263159;border-radius:8px;padding:20px;border-top:3px solid
#42d392;">
<h3 style="color:#42d392;margin:0 0 16px 0;text-align:center;">Impact</h3>
<div style="display:flex;flex-direction:column;gap:12px;">

<!-- Confidentiality Impact dropdown -->
<div class="cvss-input-group">
<label class="cvss-label">Confidentiality Impact <span class="cvss-info" title="Can
attacker read protected information?">?</span></label>
<select name="C" class="cvss-select">
<option value="H">? High (Full disclosure)</option>
<option value="L">? Low (Limited disclosure)</option>
<option value="N">? None</option>
</select>
</div>

<!-- Integrity Impact dropdown -->
<div class="cvss-input-group">
<label class="cvss-label">Integrity Impact <span class="cvss-info" title="Can attacker
modify data?">?</span></label>
<select name="I" class="cvss-select">
<option value="H">?? High (Full modification)</option>
<option value="L">? Low (Limited modification)</option>
<option value="N">? None</option>
</select>
</div>

<!-- Availability Impact dropdown -->
<div class="cvss-input-group">
<label class="cvss-label">Availability Impact <span class="cvss-info" title="Can
attacker disrupt service?">?</span></label>
<select name="A" class="cvss-select">
<option value="H">? High (Complete shutdown)</option>
<option value="L">?? Low (Performance impact)</option>
<option value="N">? None</option>
</select>
</div>
</div>
</div>

<!-- Scope metric input group -->
<div style="background:#263159;border-radius:8px;padding:20px;border-top:3px solid
#f8a541;">
<h3 style="color:#f8a541;margin:0 0 16px 0;text-align:center;">Scope</h3>
<div style="display:flex;flex-direction:column;gap:12px;">

<!-- Scope dropdown -->
<div class="cvss-input-group">
<label class="cvss-label">Scope <span class="cvss-info" title="Does vulnerability affect
other components?">?</span></label>
<select name="S" class="cvss-select">
<option value="U">? Unchanged (Same component)</option>
<option value="C">? Changed (Affects others)</option>
</select>
</div>

<!-- Calculate button triggers form submission -->
<button type="submit" class="cvss-calculate-btn">
<span>? Calculate CVSS Score</span>
</button>
</div>
</div>
</form>
</div>

<!-- Score display and explanation section -->
<div style="display:grid;grid-template-columns:1fr 2fr;gap:32px;align-items:center;">

<!-- Left column: Score visualization -->
<div style="display:flex;flex-direction:column;align-items:center;">
<!-- Dynamic badge showing current score and severity -->
<div id="cvss-calc-badge" aria-live="polite" style="margin-bottom:8px;"></div>

<!-- Container for Chart.js gauge -->
<div style="position:relative;display:flex;align-items:center;justify-content:center;">
<canvas id="cvssScoreGauge" width="180" height="120" style="background:none;"></canvas>
</div>

```

```

<!-- Score label below gauge -->
<div id="cvssGaugeLabel"
style="text-align:center;margin-top:8px;color:#a9adc1;font-size:0.95em;"></div>
</div>

<!-- Right column: Calculation explanation -->
<div id="cvssExplanation" style="color:#e2e8f0;line-height:1.6;">
<h4 style="color:#63a4ff;margin:0 0 12px 0;">Calculation Details:</h4>
<p style="margin-bottom:12px;">Adjust the metrics above to see how different attack
scenarios affect the CVSS score. The calculator uses the official CVSS v3.1 formula.</p>
<div style="background:#1a2236;padding:12px;border-radius:8px;border-left:4px solid
#63a4ff;">
<strong style="color:#fca311;">Tip:</strong> Higher exploitability and impact scores
result in higher overall CVSS scores. Scope changes can significantly affect the final
calculation.
</div>
</div>
</div>
</div>
</div>

<!-- Chart.js library from CDN for gauge visualization -->
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<script>
// Enhanced tooltip handling for help icons
document.querySelectorAll('.cvss-info').forEach(function(el){
el.setAttribute('tabindex','0'); // Make focusable for keyboard navigation
el.style.display = 'inline-block';
el.style.transition = 'color 0.18s';
el.style.color = '#63a4ff';
el.style.borderRadius = '50%';
el.style.fontWeight = '900';
el.style.cursor = 'help';
// Hover effect for visual feedback
el.addEventListener('mouseover',function(){el.style.color='#fca311'});
el.addEventListener('mouseout',function(){el.style.color='#63a4ff'});
});

// Global variable to track current gauge chart instance
let gaugeChart = null;

// Severity-specific styling and advice configuration
const scoreAdvice = {
'CRITICAL': {
msg: "Critical severity: Immediate action required. Patch now or take system offline if
patch unavailable.",
color: "#f55855"
},
'HIGH': {
msg: "High severity: Patch quickly, ideally within days. High risk of active
exploitation.",
color: "#f8a541"
},
'MEDIUM': {
msg: "Medium severity: Review and patch soon, especially if combined with other
vulnerabilities.",
color: "#3b8ded"
},
'LOW': {
msg: "Low severity: Monitor and patch in next maintenance cycle unless actively
targeted.",
color: "#42d392"
}
};

// Function to create/update the gauge visualization
function drawScoreGauge(score, label, severity) {
const ctx = document.getElementById("cvssScoreGauge").getContext("2d");
// Destroy existing chart to prevent memory leaks
if (gaugeChart) gaugeChart.destroy();

// Create gradient for visual appeal
const gradient = ctx.createLinearGradient(0, 0, 180, 0);
gradient.addColorStop(0, scoreAdvice[severity].color);
gradient.addColorStop(1, scoreAdvice[severity].color + '80');

```

```

// Configure Chart.js doughnut as semicircle gauge
gaugeChart = new Chart(ctx, {
  type: 'doughnut',
  data: {
    datasets: [{
      data: [score, 10 - score], // Score portion and remaining portion
      backgroundColor: [gradient, "rgba(35, 43, 69, 0.3)"], // Active and inactive colors
      borderWidth: 0,
      cutout: "75%" // Creates ring shape instead of solid circle
    }]
  },
  options: {
    circumference: 180, // Half circle (semicircle)
    rotation: 270, // Start from top
    plugins: {
      legend: { display: false }, // Hide default legend
      tooltip: { enabled: false } // Disable tooltips for cleaner look
    },
    animation: {
      animateRotate: true, // Smooth rotation animation
      duration: 800 // 800ms animation duration
    }
  }
});

// Update gauge label with score and severity
document.getElementById("cvssGaugeLabel").innerHTML = `
<div style="font-size: 2.2em; font-weight: 800; color: ${scoreAdvice[severity].color};
text-shadow: 0 2px 4px rgba(0,0,0,0.3);">${score.toFixed(1)}</div>
<div style="font-weight: 600; color: ${scoreAdvice[severity].color}; font-size: 1em;
margin-top: 4px;">${label}</div>`;

// Update explanation section with context-specific advice
document.getElementById("cvssExplanation").innerHTML = `
<h4 style="color:#63a4ff;margin:0 0 12px 0;">Score Analysis:</h4>
<p style="margin-bottom:12px;">${scoreAdvice[severity].msg}</p>
<div style="background:#1a2236;padding:12px;border-radius:8px;border-left:4px solid
${scoreAdvice[severity].color};">
<strong style="color:#fca311;">Score: ${score.toFixed(1)}/10 (${label})</strong>
</div>`;

// Function to update the severity badge display
function updateCalcBadge(score, severity) {
  const badge = document.getElementById("cvss-calc-badge");
  if (!severity) { badge.style.display = "none"; return; }

  // Icons for different severity levels
  const icons = { CRITICAL: '?', HIGH: '??', MEDIUM: '?', LOW: '??' };

  // Create styled badge with gradient background
  badge.innerHTML = `<div style="background: linear-gradient(135deg,
${scoreAdvice[severity].color}, ${scoreAdvice[severity].color}dd);
color: white;
padding: 8px 16px;
border-radius: 20px;
font-weight: 700;
font-size: 1em;
display: inline-flex;
align-items: center;
gap: 8px;
box-shadow: 0 4px 16px rgba(0,0,0,0.2);
border: 1px solid rgba(255,255,255,0.2);">
${icons[severity]} ${score.toFixed(1)} ? ${severity.charAt(0) +
severity.slice(1).toLowerCase()}
</div>`;
  badge.style.display = '';
}

// Function to reset calculator to default state
function resetCalculator() {
  const form = document.getElementById('cvss-calc-form');
  form.reset();

  // Set default values for worst-case scenario (Critical rating)
  form.AV.value = 'N'; // Network access
  form.AC.value = 'L'; // Low complexity

```

```

form.PR.value = 'N'; // No privileges required
form.UI.value = 'N'; // No user interaction
form.C.value = 'H'; // High confidentiality impact
form.I.value = 'H'; // High integrity impact
form.A.value = 'H'; // High availability impact
form.S.value = 'U'; // Unchanged scope

// Clear visual displays
document.getElementById("cvss-calc-badge").style.display = "none";
document.getElementById("cvssGaugeLabel").innerHTML = '';
if (gaugeChart) gaugeChart.destroy();

// Reset explanation to default text
document.getElementById("cvssExplanation").innerHTML = `
<h4 style="color:#63a4ff;margin:0 0 12px 0;">Calculation Details:</h4>
<p style="margin-bottom:12px;">Adjust the metrics above to see how different attack
scenarios affect the CVSS score. The calculator uses the official CVSS v3.1 formula.</p>
<div style="background:#1a2236;padding:12px;border-radius:8px;border-left:4px solid
#63a4ff;">
<strong style="color:#fca311;">Tip:</strong> Higher exploitability and impact scores
result in higher overall CVSS scores. Scope changes can significantly affect the final
calculation.
</div>`;
}

// CVSS v3.1 calculation using official NIST formula
const cvssCalcForm = document.getElementById('cvss-calc-form');
cvssCalcForm.addEventListener('submit', function(e){
e.preventDefault(); // Prevent form submission and page reload

// Extract metric values using official CVSS v3.1 scoring weights
const AV = {'N': 0.85, 'A': 0.62, 'L': 0.55, 'P': 0.2}[this.AV.value];
const AC = {'L': 0.77, 'H': 0.44}[this.AC.value];
const PR_vals = {'N': 0.85, 'L': 0.62, 'H': 0.27}; // Base privileges required values
const UI = {'N': 0.85, 'R': 0.62}[this.UI.value];
const S = this.S.value; // Scope affects calculation

// Scope affects Privileges Required values (CVSS specification requirement)
let PR = PR_vals[this.PR.value];
if (S === 'C' && this.PR.value === 'L') PR = 0.68; // Changed scope, low privileges
if (S === 'C' && this.PR.value === 'H') PR = 0.50; // Changed scope, high privileges

// Impact metric values
const C = {'H': 0.56, 'L': 0.22, 'N': 0}[this.C.value]; // Confidentiality
const I = {'H': 0.56, 'L': 0.22, 'N': 0}[this.I.value]; // Integrity
const A = {'H': 0.56, 'L': 0.22, 'N': 0}[this.A.value]; // Availability

// Calculate ISS (Impact Sub Score) using CIA triad
const ISS = 1 - (1 - C) * (1 - I) * (1 - A);

// Calculate Impact score based on scope
let impact;
if (S === 'U') {
// Unchanged scope formula
impact = 6.42 * ISS;
} else {
// Changed scope formula (more complex calculation)
impact = 7.52 * (ISS - 0.029) - 3.25 * Math.pow(ISS - 0.02, 15);
}

// Calculate Exploitability score from attack metrics
const exploitability = 8.22 * AV * AC * PR * UI;

// Calculate final Base Score using CVSS v3.1 formula
let baseScore;
if (impact <= 0) {
baseScore = 0; // No impact means zero score
} else {
if (S === 'U') {
// Unchanged scope: simple addition
baseScore = Math.min(impact + exploitability, 10);
} else {
// Changed scope: apply multiplier
baseScore = Math.min(1.08 * (impact + exploitability), 10);
}
}
}

```

```

// Round to one decimal place per CVSS standard
baseScore = Math.round(baseScore * 10) / 10;

// Determine severity category based on score ranges
let severity = "LOW";
if (baseScore >= 9.0) severity = "CRITICAL";
else if (baseScore >= 7.0) severity = "HIGH";
else if (baseScore >= 4.0) severity = "MEDIUM";

// Update UI with calculated results
updateCalcBadge(baseScore, severity);
drawScoreGauge(baseScore, severity.charAt(0) + severity.slice(1).toLowerCase(),
severity);
});

// Calculate initial score when page loads
document.addEventListener('DOMContentLoaded', function() {
cvssCalcForm.dispatchEvent(new Event('submit'));
});
</script>

<!-- CSS styles for calculator components -->
<style>
/* Severity badge styling with consistent colors */
.cvss-badge {
color:#fff;
border-radius:12px;
padding:3px 13px;
font-size:0.93em;
font-weight:600;
display:inline-block;
}

/* Interactive table row states */
.cvss-score-row:focus, .cvss-score-row:active {
outline:none;
background:rgba(35, 43, 69, 0.8) !important; /* Darker on focus/active */
}

.cvss-score-row:hover {
background:rgba(35, 43, 69, 0.6) !important; /* Lighter on hover */
}

/* Help icon styling with interactive states */
.cvss-info {
cursor:help !important;
border-bottom:1px dotted #63a4ff;
margin-left:2px;
font-size:0.97em;
}

/* Form input group layout */
.cvss-input-group {
display: flex;
flex-direction: column;
gap: 4px;
}

/* Form label styling */
.cvss-label {
font-weight: 600;
color: #cdd7ff;
font-size: 0.95em;
display: flex;
align-items: center;
gap: 6px;
}

/* Dropdown select styling */
.cvss-select {
background: #232b45;
border: 1px solid rgba(99, 164, 255, 0.2);
border-radius: 8px;
padding: 8px 12px;
color: #e3e8f0;
font-size: 0.95em;
cursor: pointer;

```

```

transition: all 0.2s ease;
outline: none;
}

/* Dropdown hover state */
.cvss-select:hover {
border-color: rgba(99, 164, 255, 0.4);
background: #2a3650;
}

/* Dropdown focus state for accessibility */
.cvss-select:focus {
border-color: #63a4ff;
box-shadow: 0 0 0 2px rgba(99, 164, 255, 0.2);
}

/* Dropdown option styling */
.cvss-select option {
background: #1e2640;
color: #e3e8f0;
padding: 8px;
}

/* Calculate button styling */
.cvss-calculate-btn {
background: linear-gradient(135deg, #63a4ff 0%, #4d8ef7 100%);
border: none;
border-radius: 10px;
padding: 12px 18px;
color: white;
font-weight: 700;
font-size: 1em;
cursor: pointer;
transition: all 0.2s ease;
box-shadow: 0 4px 16px rgba(99, 164, 255, 0.3);
}

/* Calculate button hover effect */
.cvss-calculate-btn:hover {
transform: translateY(-1px); /* Subtle lift effect */
box-shadow: 0 6px 20px rgba(99, 164, 255, 0.4);
}

/* Calculate button active state */
.cvss-calculate-btn:active {
transform: translateY(0); /* Return to original position */
}

/* Reset button styling (if needed) */
.cvss-reset-btn {
background: linear-gradient(135deg, #6b7280 0%, #4b5563 100%);
border: none;
border-radius: 10px;
padding: 10px 18px;
color: white;
font-weight: 600;
font-size: 0.95em;
cursor: pointer;
transition: all 0.2s ease;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2);
}

.cvss-reset-btn:hover {
transform: translateY(-1px);
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.3);
background: linear-gradient(135deg, #4b5563 0%, #374151 100%);
}

.cvss-reset-btn:active {
transform: translateY(0);
}

/* Responsive design breakpoints */
@media (max-width: 1024px) {
/* Tablet layout: stack calculator columns */
#cvss-calc-wrapper > div:first-child {
grid-template-columns: 1fr;
}
}

```



```

gap: 24px;
}

#cvss-calc-wrapper > div:last-child {
grid-template-columns: 1fr;
gap: 24px;
text-align: center;
}
}

@media (max-width: 768px) {
/* Mobile layout: single column for all grids */
div[style*="grid-template-columns:1fr 1fr 1fr"] {
grid-template-columns: 1fr !important;
}
}
</style>

<!-- Navigation back to main dashboard -->
<div style="text-align:center; margin:2.5em 0 1em 0;">
<a href="{{ url_for('main.index') }}" style="color:#63a4ff; padding:0.8em 2em;
border-radius:6px; font-weight:600; text-decoration:none;">
? Back to Dashboard
</a>
</div>

</div>
<!-- End content block -->
{% endblock %}

```

templates\learn\index.html

```
{% extends "layouts/learn_layout.html" %}

{% block content %}
<!-- Main container for CWE exploration interface -->
<div class="learn-container">
<!-- Page title with CVSS blue theme for consistency -->
<h1 style="color:#63a4ff;">CWEs by Severity - Explore Common Weaknesses</h1>

<!-- Chart section with styled container -->
<div style="margin:32px 0 48px 0; background:#1a2236; border-radius:14px;
padding:30px;">
<!-- Chart title with consistent styling -->
<div style="font-weight:700; color:#63a4ff; font-size:1.02rem; margin-bottom:10px;">
CWEs by Severity
</div>

<!-- Chart canvas container with fixed dimensions for consistency -->
<div style="position:relative;height:320px;width:100%;">
<!-- Conditional rendering: Only show chart if data exists -->
{% if cwe_severity.labels|length > 0 %}
<!-- Canvas element for Chart.js rendering -->
<canvas id="learnCweSeverityChart"></canvas>
{% else %}
<!-- Fallback message when no CWE data is available -->
<div style="color:#a9adc1;text-align:center;padding:30px 0;">
No CWE data available. Please check your CWE XML or try again later.
</div>
{% endif %}
</div>
</div>

<!-- Main content grid: sidebar + content area -->
<div class="learn-grid">
<!-- Left sidebar containing scrollable CWE list -->
<aside class="learn-sidebar">
<!-- Sidebar header with consistent styling -->
<div style="padding-left:24px;font-weight:700;color:#63a4ff;margin-bottom:10px;">CWE
List</div>

<!-- Conditional rendering: Only show list if CWE data exists -->
{% if cwe_dict|length > 0 %}
<!-- Scrollable list of all available CWE codes -->
<ul id="cweListColumn" class="cwe-list-scroll">
<!-- Loop through all CWE entries from server data -->
{% for code, entry in cwe_dict.items() %}
<!-- Individual CWE list item with data attribute for JavaScript -->
<li data-cwe="{{ code }}" class="cwe-list-item">
{{ code }} <!-- Display CWE code (e.g. CWE-79) -->
</li>
{% endfor %}
</ul>
{% else %}
<!-- Fallback message when CWE dictionary is empty -->
<div style="padding:18px 22px;color:#a9adc1;">
No CWE data loaded from MITRE XML file.
</div>
{% endif %}
</aside>

<!-- Main content section for displaying CWE details -->
<section class="learn-content">
<!-- Dynamic detail panel updated by JavaScript -->
<div id="cweDetailPanel">
<!-- Default placeholder text before user interaction -->
<p style="color:#a9adc1;">Select a CWE code to see its official description, examples,
and mitigations.</p>
</div>
</section>
</div>
</div>

<!-- Data injection: Transfer server data to client-side JavaScript -->
```

```
<script>
// Convert Python objects to JavaScript using Jinja2 tojson filter
// Global variables accessible by external JavaScript files
window.learnCweSeverityData = {{ cwe_severity | tojson }}; <!-- Chart data for severity
distribution -->
window.learnCweDict = {{ cwe_dict | tojson }}; <!-- Complete CWE dictionary with
details -->
</script>

<!-- Load external JavaScript file containing interaction logic -->
<!-- Uses Flask url_for to generate proper static file URLs -->
<script src="{{ url_for('static', filename='js/pages/learn.js') }}"></script>
{% endblock %}
```

templates\learn\what-is-cve.html

```
{% extends "base.html" %}
{% block content %}
<!-- Main container with centered layout and responsive max-width -->
<div style="max-width:1400px; margin:0 auto 32px auto; padding-top:20px;">
<!-- Page Title & Intro -->
<!-- Blue color scheme matches the site's cybersecurity theme -->
<h1 style="color:#63a4ff;">What is a CVE?</h1>
<!-- Comprehensive introduction explaining CVE purpose and importance -->
<p style="color:#e2e8f0; font-size:1.12em;">
<b>CVE</b> stands for <em>Common Vulnerabilities and Exposures</em>.
It is the universal system?managed by
<!-- Color-coded organization names for visual distinction -->
<span style="color:#79a3f8;">MITRE</span> and
<span style="color:#fca045;">NVD</span>?for giving unique names to
publicly known cybersecurity vulnerabilities.
<br><br>
<b>Why do CVEs matter?</b>
They allow security teams, researchers, and companies to discuss and fix the <u>same</u>
problem consistently using a single ID.
Every CVE describes a specific security flaw, explains its risk, and helps you learn
about real-world threats and how to manage them.
</p>

<!-- CVE Anatomy Interactive Sample -->
<!-- Educational section showing real CVE structure -->
<h2 style="color:#cdd7ff; margin-top:36px;">CVE Anatomy: What's Inside a Vulnerability
Record</h2>
<!-- Container with blue left border to emphasize importance -->
<div id="cve-sample-section"
style="background:#232b45; border-radius:16px; padding:34px 28px; margin-bottom:26px;
border-left:6px solid #63a4ff;">
<!-- Flexible layout for CVE metadata fields -->
<div style="display:flex; flex-wrap:wrap; gap:22px; align-items:center;">
<!-- Interactive CVE ID field - uses famous Log4Shell as example -->
<span class="field-label" data-field="id" tabindex="0"
style="background:#263159;padding:7px
15px;border-radius:8px;color:#63a4ff;cursor:pointer;font-family:monospace;">
ID: <b>CVE-2021-44228</b>
</span>
<!-- Publication date field with hover functionality -->
<span class="field-label" data-field="published" tabindex="0"
style="background:#263159;padding:7px 15px;border-radius:8px; color:#cdd7ff;
cursor:pointer;">
Published: <b>2021-12-09</b>
</span>
<!-- Critical severity with red background for urgency -->
<span class="field-label" data-field="severity" tabindex="0"
style="background:#f55855;padding:7px 15px;border-radius:8px; color:#fff;
font-weight:600; cursor:pointer;">
Severity: Critical
</span>
<!-- CWE classification with blue background -->
<span class="field-label" data-field="cwe" tabindex="0"
style="background:#3b8ded;padding:7px 15px;border-radius:8px; color:#fff;
font-weight:600; cursor:pointer;">
CWE: CWE-502
</span>
</div>
<!-- Description field spans full width -->
<div style="margin-top:18px;">
<span class="field-label" data-field="desc" tabindex="0"
style="background:#263159;padding:7px
15px;border-radius:8px;color:#e2e8f0;cursor:pointer;">
Description: Apache Log4j2 Remote code execution via JNDI lookup ("Log4Shell")
</span>
</div>
<!-- Reference links section -->
<div style="margin-top:13px; color:#a9adc1;">
<span class="field-label" data-field="refs" tabindex="0"
style="background:#263159;padding:5px 11px;border-radius:8px;cursor:pointer;">
References:
</span>
</div>
</div>
```

```

<!-- External links to authoritative sources -->
<a href="https://nvd.nist.gov/vuln/detail/CVE-2021-44228"
target="_blank"
style="color:#63a4ff; margin-left:10px;">
NVD
</a>
<a href="https://logging.apache.org/log4j/2.x/security.html" target="_blank"
style="color:#63a4ff; margin-left:18px;">
Vendor Advisory
</a>
</div>

<!-- Tooltip Box - Hidden by default, positioned absolutely -->
<div id="cve-sample-tooltip" style="display:none; position:absolute; background:#101828;
color:#eaf4fb; border-radius:8px; padding:13px 15px; font-size:1.01em; min-width:210px;
max-width:350px; box-shadow:0 3px 18px #0008; z-index:200; pointer-events:none;"></div>
</div>

<!-- Section: How to Read & Interpret a CVE -->
<h2 style="color:#cdd7ff;">How to Read a CVE Entry & Interpret Its Details</h2>
<!-- Flexible layout with main content and sidebar tips -->
<div style="display:flex; flex-wrap:wrap; gap:30px; align-items:flex-start;">
<!-- Main content area with step-by-step guide -->
<ul style="background:#1a2236; border-radius:13px; padding:26px 28px 16px 26px;
color:#e3e8f0; font-size:1.09em; margin-bottom:22px;flex:2;">
<li><b>ID:</b> Unique reference for searching in vulnerability databases and security
tools.</li>
<li><b>Description:</b> Review the summary?does it match your environment and
risks?</li>
<li><b>Severity & Score:</b> The higher (e.g., <b>Critical</b>), the more urgent. Check
CVSS number if present.</li>
<li><b>CWE:</b> Type or category of bug?helps guide fixes and mitigations. E.g.,
Deserialization, XSS</li>
<li><b>References:</b> Advisories?ALWAYS consult for patches or step-by-step
guidance.</li>
<li><b>Products:</b> (If present) Specific affected software; always check compatibility
before action.</li>
</ul>
<!-- Sidebar with helpful tips and related links -->
<div style="flex:1; min-width:210px;">
<!-- Green accent color for tips section -->
<div style="background:#232b45; border-radius:13px; padding:18px 19px 19px 20px;
color:#42d392; font-size:0.99em;">
<b>Tips:</b>
<!-- Practical advice list -->
<ul style="font-size:0.99em; color:#a9adc1; margin-top:7px;">
<li>Never treat all CVEs the same?focus on critical/high scores first.</li>
<li>Check advisory links before taking action.</li>
<li>Look up the CWE type ("What is a CWE?") to prevent similar issues in your own
code.</li>
</ul>
<!-- Related learning links using Flask url_for routing -->
<div style="margin-top:11px;">
<a href="{{ url_for('learn.learn_topic', topic='what-is-cwe') }}" style="color:#63a4ff;
text-decoration:underline;">What is a CWE?</a> |
<a href="{{ url_for('learn.learn_topic', topic='cvss-scores') }}" style="color:#63a4ff;
text-decoration:underline;">CVSS Score Guide</a>
</div>
</div>
</div>
</div>
</div>

<!-- Interactive Today's CVEs -->
<h2 style="color:#cdd7ff; margin-top:38px;">Today's CVEs</h2>
<!-- Collapsible section for recent vulnerabilities -->
<div style="background:#1a2236; border-radius:14px; padding:20px 28px 24px 26px;
margin-bottom:18px;">
<!-- Header with toggle functionality and navigation link -->
<div style="display: flex; justify-content: space-between; align-items: center;
margin-bottom:9px; cursor: pointer;" onclick="toggleTodaysCVEs()">
<div style="color:#94a3b8;font-size:1.01em;">
<span>Showing most recent vulnerabilities published in the last 24 hours.</span>
<!-- Link to full vulnerabilities page -->
<a href="{{ url_for('main.vulnerabilities') }}" style="margin-left:17px;
color:#63a4ff;text-decoration:underline;font-weight:600;">SHOW ALL TODAY CVEs ?</a>
</div>
<!-- Collapsible indicator arrow -->

```

```

<span id="todaysCvesIcon" style="color: #a9adc1; font-size: 1.2rem;">?</span>
</div>
<!-- Content area that can be toggled -->
<div id="todaysCvesContent" style="display: block;">
<div style="display: grid; gap: 17px;">
<!-- Jinja2 logic to filter today's CVEs -->
{% set todays_cves = [] %}
{% for cve in latest_cves[:10] %}
<!-- Extract date from CVE published field -->
{% set date_str = cve.get('Published', '')[:10] %}
<!-- Compare with current date -->
{% if date_str == (now.strftime('%Y-%m-%d')) %}
<!-- Add to today's list if dates match -->
{% set _ = todays_cves.append(cve) %}
{% endif %}
{% endfor %}

<!-- Conditional rendering based on data availability -->
{% if todays_cves %}
<!-- Display first 5 CVEs from today -->
{% for cve in todays_cves[:5] %}
<!-- CVE card with severity-based color coding -->
<div style="background:#263159; border-radius:12px; padding:18px 19px 14px 19px;
border-left:4px solid {% if cve.get('Severity') == 'CRITICAL' %}#f47174{% else
%}#fca311{% endif %};">
<!-- Card header with CVE details -->
<div style="display:flex; justify-content:space-between; align-items:flex-start;
margin-bottom:10px;">
<div style="flex:1;">
<!-- CVE ID, severity badge, and CWE info -->
<div style="display:flex; gap:11px; align-items:center;">
<!-- Clickable CVE ID linking to detail page -->
<a href="{% url_for('main.cve_detail', cve_id=cve.get('ID')) %}" style="color:#63a4ff;
font-weight:700; font-size:1.1em; text-decoration:none;">
{{ cve.get('ID', 'N/A') }}
</a>
<!-- Color-coded severity badge -->
<span style="background: {% if cve.get('Severity')=='CRITICAL' %}#f47174{% elif
cve.get('Severity')=='HIGH' %}#fca311{% elif cve.get('Severity')=='MEDIUM' %}#3498db{%
else %}#42d392{% endif %}; color:white; padding:4px 12px; border-radius:16px;
font-size:0.83em; font-weight:600;">
{{ cve.get('Severity', 'UNKNOWN') }}
</span>
<!-- Optional CWE classification -->
{% if cve.get('CWE') %}
<span style="margin-left:10px; background:#212d46; color:#63a4ff; padding:1px 9px;
border-radius:11px; font-size:0.95em;">{{ cve.get('CWE') }}</span>
{% endif %}
</div>
<!-- Truncated description with ellipsis -->
<div style="color:#e2e8f0; line-height:1.44; margin-top:7px; font-size:0.97em;">
{{ cve.get('Description', 'No description available')[:170] }}{% if
cve.get('Description', '')|length > 170 %}...{% endif %}
</div>
</div>
<!-- Publication date aligned to right -->
<div style="min-width:105px; text-align:right; color:#a9adc1;">
<div>{{ cve.get('Published')[:10] }}</div>
</div>
</div>
</div>
{% endfor %}
{% else %}
<!-- Empty state when no CVEs found for today -->
<div style="text-align:center; color:#94a3b8; padding:26px;font-size:1.08em;">
<div style="font-size:1.7rem;margin-bottom:8px;">?</div>
No new CVEs detected today.
</div>
{% endif %}
</div>
</div>
</div>

<!-- Timeline of Major CVEs -->
<h2 style="color:#cdd7ff; margin-top:34px;">Timeline of Major CVEs</h2>
<!-- Historical context table -->
<div style="background:#232b45;border-radius:10px;padding:24px 18px 20px 18px;

```

```

margin-bottom:18px;">
<!-- Structured table of famous CVEs -->





```

```

<!-- Interactive Highlight/Tooltip JS -->
<script>
// Tooltip data object containing educational information
const cveSampleTooltips = {
'id': { title: "CVE ID", text: "A unique vulnerability identifier. Used to track and
search. Format: CVE-YYYY-NNNNN." },
'published': { title: "Published Date", text: "Date the vulnerability was publicly
announced. Crucial for understanding risk exposure windows." },
'severity': { title: "Severity", text: "Shows risk level (Critical, High, Medium,
Low)?often based on the CVSS score. Prioritize higher severities." },
'cwe': { title: "CWE", text: "Specifies the underlying type of coding/design flaw. Links
to common patterns for prevention (e.g. Deserialization, XSS)." },
'desc': { title: "Description", text: "A summary of what the vulnerability is and how it
can be exploited. Always read to see if it applies to your environment." },
'refs': { title: "References", text: "Links to authoritative advisories, patches, vendor
updates, and analysis. The best source of detailed remediation steps." }
};

// Immediately invoked function expression to avoid global namespace pollution
(function() {
// Get DOM references for tooltip system
const section = document.getElementById('cve-sample-section');
const tooltip = document.getElementById('cve-sample-tooltip');
let active = null; // Track currently active tooltip

// Function to display tooltip with positioning logic
function showTip(label, event) {
const field = label.dataset.field; // Get field type from data attribute
const data = cveSampleTooltips[field]; // Retrieve tooltip content
if (!data) return; // Exit if no data found

// Set tooltip content with title and description
tooltip.innerHTML = "<b style='color:#63a4ff;'>" + data.title + "</b><br>" + data.text;
tooltip.style.display = "block";

// Calculate optimal positioning
const rect = label.getBoundingClientRect();
tooltip.style.position = 'fixed';
let left = rect.left;
let top = rect.bottom + 8; // Position below the trigger element

// Prevent tooltip from going off-screen
if((left + tooltip.offsetWidth + 10) > window.innerWidth){
left = window.innerWidth - tooltip.offsetWidth - 22;
}

// Apply calculated position
tooltip.style.left = left + "px";
tooltip.style.top = top + "px";
active = label; // Set as currently active
}

// Function to hide tooltip and reset state
function hideTip(ev) {
tooltip.style.display = "none";
active = null;
}

// Set up event listeners for all interactive elements
const labels = section.querySelectorAll('.field-label');
labels.forEach(function(label) {
// Mouse events for desktop interaction
label.addEventListener('mouseover', function(e) { showTip(label, e); });
label.addEventListener('mouseleave', hideTip);
// Keyboard events for accessibility
label.addEventListener('focus', function(e){ showTip(label, e); });
label.addEventListener('blur', hideTip);
// Make elements focusable for keyboard navigation
label.setAttribute('tabindex', '0');
});

// Global event handlers for cleanup
document.addEventListener('mousedown', function(e){
// Hide tooltip when clicking elsewhere
if(active && !active.contains(e.target)) hideTip();
});

```



```

// Hide on scroll to prevent positioning issues
window.addEventListener('scroll', hideTip, {passive:true});
// Hide on resize to prevent positioning issues
window.addEventListener('resize', hideTip);
// Hide on escape key for accessibility
document.addEventListener('keydown', function(e){
if(e.key === 'Escape') hideTip();
});
})();

// Today's CVEs collapse function - standalone for onclick handler
function toggleTodaysCVEs() {
// Get DOM references
const content = document.getElementById('todaysCvesContent');
const icon = document.getElementById('todaysCvesIcon');

// Toggle visibility and update icon
if (content.style.display === 'none') {
content.style.display = 'block';
icon.textContent = '?'; // Down arrow for expanded state
} else {
content.style.display = 'none';
icon.textContent = '?'; // Right arrow for collapsed state
}
}
</script>

<!-- Navigation back to main dashboard -->
<div style="text-align:center; margin:2.5em 0 1em 0;">
<!-- Back button using Flask routing for dynamic URL generation -->
<a href="{{ url_for('main.index') }}" style="color:#63a4ff; padding:0.8em 2em;
border-radius:6px; font-weight:600; text-decoration:none;">
? Back to Dashboard
</a>
</div>
{% endblock %}

```

templates\learn\what-is-cwe.html

```
{% extends "base.html" %}

{% block head %}
<style>
/* Custom scrollbar styling for dark theme */
/* Creates consistent dark-themed scrollbars across webkit browsers */
.cwe-list-scroll {
max-height: 550px; /* Prevents excessive vertical space usage */
overflow-y: auto; /* Enables vertical scrolling when content exceeds height */
list-style: none; /* Removes default bullet points */
padding: 0; /* Removes default list padding */
margin: 0; /* Removes default list margins */
}

/* Webkit browsers (Chrome, Safari, Edge) scrollbar customization */
.cwe-list-scroll::-webkit-scrollbar {
width: 8px; /* Thin scrollbar for modern appearance */
}

.cwe-list-scroll::-webkit-scrollbar-track {
background: #1a2236; /* Dark background matching site theme */
border-radius: 4px; /* Rounded edges for polished look */
}

.cwe-list-scroll::-webkit-scrollbar-thumb {
background: #63a4ff; /* Blue accent color matching site branding */
border-radius: 4px; /* Consistent rounded appearance */
border: 1px solid #263159; /* Subtle border for definition */
}

.cwe-list-scroll::-webkit-scrollbar-thumb:hover {
background: #5292e6; /* Slightly darker blue on hover */
}

.cwe-list-scroll::-webkit-scrollbar-thumb:active {
background: #4a80cc; /* Even darker when actively dragging */
}

/* Firefox scrollbar styling fallback */
.cwe-list-scroll {
scrollbar-width: thin; /* Thin scrollbar option */
scrollbar-color: #63a4ff #1a2236; /* Thumb and track colors */
}

/* CWE list item styling for interactive elements */
.cwe-list-item {
padding: 8px 24px; /* Comfortable click target size */
cursor: pointer; /* Indicates clickable element */
font-weight: 500; /* Medium weight for readability */
color: #e3e7f2; /* Light gray text for contrast */
user-select: none; /* Prevents accidental text selection */
border-left: 4px solid transparent; /* Space for selection indicator */
transition: background 0.13s, border-left 0.13s, color 0.13s; /* Smooth hover effects */
}

.cwe-list-item:hover {
background: #212d46; /* Darker background on hover */
color: #ffffff; /* Pure white text for maximum contrast */
}

.cwe-list-item.selected {
background: #212d46; /* Same background as hover state */
border-left: 4px solid #63a4ff; /* Blue left border for selection indicator */
color: #63a4ff; /* Blue text to match selection theme */
}

.cwe-code {
color: #94a3b8; /* Muted color for CWE codes */
font-size: 0.9em; /* Slightly smaller than main text */
}

/* Enhanced button styling with hover effects */
```

```

.toggle-button {
margin: 16px 0 0 24px; /* Consistent spacing with list items */
background: #263159; /* Dark blue background */
color: #63a4ff; /* Blue text color */
padding: 8px 18px; /* Comfortable button size */
border-radius: 8px; /* Rounded corners */
border: none; /* Remove default button styling */
cursor: pointer; /* Indicates interactive element */
font-weight: 600; /* Bold text for emphasis */
font-size: 0.97em; /* Slightly smaller than body text */
transition: background 0.2s, transform 0.1s; /* Smooth hover animations */
}

.toggle-button:hover {
background: #2d3d6b; /* Lighter blue on hover */
transform: translateY(-1px); /* Subtle lift effect */
}

.toggle-button:active {
transform: translateY(0); /* Returns to normal position when clicked */
}
</style>
{% endblock %}

{% block content %}
<!-- Main container with responsive max-width -->
<div style="max-width:1400px;margin:0 auto 32px auto;padding-top:20px;">
<!-- Page title with consistent blue theming -->
<h1 style="color:#63a4ff;">What is a CWE?</h1>
<!-- Comprehensive introduction explaining CWE concept and importance -->
<p style="color:#e2e8f0;font-size:1.13em;">
<strong>CWE</strong> stands for <em><span style="color:#63a4ff">Common Weakness
Enumeration</span></em>.<br> In
technology and computer science, we learn by identifying patterns?not just memorizing a
list of past mistakes. The <b>CWE catalog</b> is a map of the <span
style="color:#fca311;">most common
error patterns</span> that lead to vulnerabilities in software.<br>
<br>
<b>If you spot the pattern, you can prevent the problem:</b>
<!-- Educational bullet points explaining CWE value proposition -->
<ul style="color:#e2e8f0;margin-left:20px;">
<li>Just like a math book teaches "types of mistakes" (e.g., sign error, division by
zero), CWE teaches you <span style="color:#42d392;">recurring security
problems</span>.</li>
<li><b>CWE entries</b> are like chapters in a textbook?each describes a different kind
of software weakness.</li>
<li>You'll see real-world examples (<b>CVEs</b>), but you'll learn the general idea with
CWE.</li>
</ul>
</p>

<!-- Educational section explaining CWE structure -->
<h2 style="color:#cdd7ff;">CWE Structure: How Weakness Types Are Organized</h2>
<ul style="color:#e2e8f0;margin-left:20px;">
<li><b>ID format:</b> <code>CWE<&lt;Number&gt; ? &lt;Name&gt;</code> (e.g., <code>CWE-79
? Cross-Site Scripting</code>)</li>
<li><b>Name:</b> A concise label for the type of mistake (e.g., "Buffer Overflow")</li>
<li><b>Description:</b> Explains what usually goes wrong?and why</li>
<li><b>Example:</b> Shows a typical code or configuration that triggers this
weakness</li>
<li><b>Mitigations:</b> Teaches you practical ways to avoid making this mistake when
writing software</li>
<li><b>Hierarchy:</b> Shows if a weakness is broad ("Injection") or a sub-type ("SQL
Injection")</li>
</ul>

<!-- Interactive chart section for data visualization -->
<h2 style="color:#cdd7ff;margin-top:26px;">CWEs by Severity</h2>
<div style="background:#1a2236;border-radius:12px;padding:28px 18px 25px 18px;
margin-bottom:18px;">
<!-- Context information about data source -->
<div style="color:#94a3b8; font-size:0.98em; margin-bottom:9px;">
<span>Showing data from <b>recent analysis</b>. Click any CWE below for live MITRE
data.</span>
</div>

<!-- Filter controls for chart customization -->

```

```

<div style="display:flex;align-items:center; justify-content:space-between;
margin-bottom:18px;">
<label style="color:#e2e8f0;font-size:1.07em; font-weight:600;">
Show:
<!-- Dropdown to control number of displayed CWEs -->
<select id="cweSeverityFilter" style="background:#232b45;color:#63a4ff;padding:7px
14px;border-radius:6px;border:none;margin-left:7px;">
<option value="10" selected>Top 10 CWEs</option>
<option value="5">Top 5 CWEs</option>
</select>
</label>
</div>

<!-- Chart container with responsive wrapper -->
<div style="max-width:1400px;width:100%;overflow-x:auto;">
<canvas id="cwesBySeverityChart" width="700" height="235"></canvas>
</div>
</div>

<!-- JavaScript for chart functionality -->
<script>
// Wait for DOM to be fully loaded before initializing
document.addEventListener('DOMContentLoaded', function() {
// Initialize CWE severity data with fallback values
window.cweSeverityData = window.cweSeverityData || {
labels: ['Cross-Site Scripting', 'SQL Injection', 'Input Validation', 'Path Traversal',
'Buffer Overflow'],
indices: ['CWE-79', 'CWE-89', 'CWE-20', 'CWE-22', 'CWE-119'],
data: {
CRITICAL: [15, 12, 8, 5, 7],
HIGH: [25, 20, 15, 10, 12],
MEDIUM: [30, 25, 20, 15, 18],
LOW: [10, 8, 12, 8, 6]
}
};

// Navigation function to filter vulnerabilities by CWE
function goToVulnsForCWE(id) {
const cwe_key = window.cweSeverityData.indices[id];
if (cwe_key) {
// Redirect to vulnerabilities page with CWE filter
window.location.href = "/vulnerabilities?q=" + encodeURIComponent(cwe_key);
}
}

// Main chart rendering function
function renderChart(topN) {
const data = window.cweSeverityData.data || {};

// Slice data arrays to show only requested number of items
const labels = window.cweSeverityData.labels ? window.cweSeverityData.labels.slice(0,
topN) : [];

// Debug logging for development
console.log("Rendering chart with topN:", topN);
console.log("Labels:", labels);
console.log("Data:", data);

// Chart.js configuration object
const chartData = {
labels: labels,
datasets: [
// Each severity level as separate dataset for stacking
{label:'Critical', data: data.CRITICAL ? data.CRITICAL.slice(0, topN) : [],
backgroundColor:'#f55855'},
{label:'High', data: data.HIGH ? data.HIGH.slice(0, topN) : [],
backgroundColor:'#f8a541'},
{label:'Medium', data: data.MEDIUM ? data.MEDIUM.slice(0, topN) : [],
backgroundColor:'#3b8ded'},
{label:'Low', data: data.LOW ? data.LOW.slice(0, topN) : [], backgroundColor:'#42d392'}
]
};

// Destroy existing chart to prevent memory leaks
if (window.cweSeverityChart) {
window.cweSeverityChart.destroy();
}
}

```

```

// Create new Chart.js instance
window.cwesSeverityChart = new
Chart(document.getElementById('cwesBySeverityChart').getContext('2d'), {
type: 'bar', // Stacked bar chart type
data: chartData,
options:{
responsive:true, // Adapts to container size
maintainAspectRatio:false, // Allows height customization
plugins:{
legend:{display:true, position:"bottom"}, // Legend below chart
title:{display:true, text:`CWES by Severity ? Top ${topN}`} // Dynamic title
},
scales:{
// X and Y axis configuration with stacking enabled
x:{stacked:true, ticks:{color:'#cdd7ff', font:{size:11}}},
y:{stacked:true, beginAtZero:true, ticks:{color:'#cdd7ff'}}
},
// Click handler for navigation to vulnerability details
onClick: (evt, activeEls) => {
if (activeEls && activeEls.length > 0) {
goToVulnsForCWE(activeEls[0].index);
}
}
});

// Initialize with default Top 10 view
renderChart(10);

// Event listener for filter dropdown changes
document.getElementById('cweSeverityFilter').addEventListener('change', function(){
renderChart(parseInt(this.value,10)); // Parse selected value and re-render
});
});
</script>

<!-- CWE Explorer Interface Section -->
<h2 style="color:#cdd7ff;margin-top:34px;">CWE List & Explorer</h2>
<!-- Instructions for using the explorer -->
<div style="color:#94a3b8; font-size:0.95em; margin-bottom:12px;">
<strong>How it works:</strong> Click any CWE below to fetch live, detailed information
from the official MITRE database. The first click may take a few seconds to load.
</div>

<!-- Two-column grid layout for explorer interface -->
<div style="display:grid; grid-template-columns: 300px 1fr; gap: 36px;">
<!-- Left sidebar with CWE list -->
<aside style="background:#1a2236; border-radius:14px; padding:18px 0 18px 0; height:
fit-content;">
<!-- Sidebar header with mode indicator -->
<div style="padding-left:24px;font-weight:700;color:#63a4ff;margin-bottom:10px;">
CWE List <span id="cweListModeTip"
style="color:#94a3b8;font-size:0.93em;font-weight:400;">(Your Key Set)</span>
</div>

<!-- Curated key CWES list (default view) -->
<ul id="cweListColumn" class="cwe-list-scroll">
{% for cwe in key_cwes %}
<!-- Generate list items with data attributes for JavaScript -->
<li data-cwe="{{ cwe }}" class="cwe-list-item">
{{ key_cwe_titles[cwe] }} <span class="cwe-code">({{ cwe }})</span>
</li>
{% endfor %}
</ul>

<!-- Full CWE dictionary list (hidden by default) -->
<ul id="cweListFull" class="cwe-list-scroll" style="display:none;">
{% for code, entry in cwe_dict.items() %}
<!-- Generate complete list from CWE dictionary -->
<li data-cwe="{{ code }}" class="cwe-list-item">
{{ entry.name }} <span class="cwe-code">({{ code }})</span>
</li>
{% endfor %}
</ul>

```

```

<!-- Toggle button to switch between list views -->
<button id="toggleShowAllCwe" class="toggle-button">
Show All CWEs
</button>
</aside>

<!-- Right panel for detailed CWE information -->
<section style="background:#1a2236; border-radius:14px; padding:28px
26px;min-height:390px;">
<div id="cweDetailPanel">
<!-- Conditional rendering based on data availability -->
{% if cwe_dict|length > 0 %}
<!-- Default empty state with instructions -->
<div style="text-align: center; padding: 40px; color: #a9adc1;">
<div style="font-size: 2.5rem; margin-bottom: 16px;">?</div>
<div style="font-size: 1.1rem; margin-bottom: 8px;">Select a CWE to explore</div>
<div style="margin-top: 12px; font-size: 0.9em;">Click any CWE from the list to see
detailed information, mitigations, examples, and relationships.</div>
</div>
{% else %}
<!-- Error state when no CWE data available -->
<p style="color:#f47174;">No CWE data found in catalog or fallback.</p>
{% endif %}
</div>
</section>
</div>

<!-- Pass server-side data to JavaScript global variables -->
<script>
// CWE severity data for chart rendering
window.cweSeverityData = {{ cwe_severity | tojson }};
// Complete CWE dictionary for reference
window.learnCweDict = {{ cwe_dict|tojson }};
// Curated list of important CWEs
window.key_cwes = {{ key_cwes|tojson }};
// Display names for key CWEs
window.key_cwe_titles = {{ key_cwe_titles|tojson }};

// Debug logging for development and troubleshooting
console.log("CWE Severity Data:", window.cweSeverityData);
</script>

<!-- External JavaScript module for CWE interaction handling -->
<script src="{{ url_for('static', filename='js/pages/learn.js') }}"></script>
</div>

<!-- Navigation back to main dashboard -->
<div style="text-align:center; margin:2.5em 0 1em 0;">
<a href="{{ url_for('main.index') }}" style="color:#63a4ff; padding:0.8em 2em;
border-radius:6px; font-weight:600; text-decoration:none;">
? Back to Dashboard
</a>
</div>
{% endblock %}

```

templates\learn\what-is-nvd-mitre.html

```
{% extends "base.html" %}
{% block content %}
<!-- Main container with consistent max-width and spacing -->
<div style="max-width:1400px;margin:0 auto 32px auto;padding-top:20px;">
<!-- Page title with consistent blue theming -->
<h1 style="color:#63a4ff;">What is NVD & MITRE?</h1>
<!-- Introductory paragraph explaining the purpose and importance -->
<p style="color:#e2e8f0;font-size:1.12em;line-height:1.6;">
<strong>NVD</strong> (National Vulnerability Database) and <strong>MITRE</strong> are
the two foundational organizations
that maintain the vulnerability intelligence systems used worldwide. Understanding their
roles helps you navigate
the cybersecurity landscape more effectively.
</p>

<!-- NVD Section with government/official orange theming -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with emoji icon and organizational branding -->
<h2 style="color:#fca311;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">??</span> National Vulnerability Database (NVD)
</h2>
<!-- Basic organizational information in structured format -->
<div style="color:#e2e8f0;margin-bottom:20px;">
<strong>Who:</strong> Managed by the National Institute of Standards and Technology
(NIST), part of the U.S. Department of Commerce<br>
<strong>What:</strong> The U.S. government's repository of standards-based vulnerability
management data<br>
<strong>Why it matters:</strong> Provides comprehensive vulnerability information with
standardized severity scores (CVSS)
</div>

<!-- Services and offerings section with distinct background -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<h3 style="color:#42d392;margin:0 0 12px 0;">What NVD Provides:</h3>
<!-- Bulleted list of key services -->
<ul style="color:#a9adc1;margin-left:20px;">
<li><strong>CVSS Scores:</strong> Standardized severity ratings for vulnerabilities</li>
<li><strong>CPE Names:</strong> Standardized naming for IT products and platforms</li>
<li><strong>CWE Mappings:</strong> Links vulnerabilities to weakness categories</li>
<li><strong>Reference Links:</strong> Technical details, patches, and advisories</li>
<li><strong>API Access:</strong> Machine-readable data feeds for automation</li>
</ul>
</div>

<!-- Real-world application context with accent background -->
<div style="background:#263159;border-radius:8px;padding:16px;">
<strong style="color:#fca311;">Real-World Usage:</strong><br>
<span style="color:#e2e8f0;">Security teams use NVD data to prioritize patching,
vulnerability scanners reference NVD for severity scores, and compliance frameworks
often require NVD-based vulnerability management.</span>
</div>
</section>

<!-- MITRE Section with research/innovation green theming -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with research-focused icon -->
<h2 style="color:#42d392;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">??</span> MITRE Corporation
</h2>
<!-- Organizational overview in consistent format -->
<div style="color:#e2e8f0;margin-bottom:20px;">
<strong>Who:</strong> Non-profit organization that operates federally funded research
and development centers<br>
<strong>What:</strong> Creates and maintains the CVE and CWE programs<br>
<strong>Why it matters:</strong> Provides the universal naming conventions that make
vulnerability communication possible
</div>

<!-- Key programs section with visual organization -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<h3 style="color:#63a4ff;margin:0 0 12px 0;">MITRE's Key Programs:</h3>
<!-- Grid layout for program descriptions -->
```

```

<div style="display:grid;gap:16px;">
<!-- Each program with distinct color-coded left border -->
<div style="border-left:4px solid #63a4ff;padding-left:16px;">
<strong style="color:#63a4ff;">CVE Program</strong><br>
<span style="color:#a9adc1;">Assigns unique identifiers to publicly known
vulnerabilities (CVE-YYYY-NNNNN format)</span>
</div>
<div style="border-left:4px solid #3b8ded;padding-left:16px;">
<strong style="color:#3b8ded;">CWE Program</strong><br>
<span style="color:#a9adc1;">Catalogs common types of software and hardware
weaknesses</span>
</div>
<div style="border-left:4px solid #f8a541;padding-left:16px;">
<strong style="color:#f8a541;">ATT&CK Framework</strong><br>
<span style="color:#a9adc1;">Knowledge base of adversary tactics and techniques based on
real-world observations</span>
</div>
</div>
</div>

<!-- Global impact statement with accent styling -->
<div style="background:#263159;border-radius:8px;padding:16px;">
<strong style="color:#42d392;">Global Impact:</strong><br>
<span style="color:#e2e8f0;">MITRE's standards are used by security vendors,
researchers, and organizations worldwide to ensure consistent vulnerability
identification and communication.</span>
</div>
</section>

<!-- Collaboration section explaining organizational relationship -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with partnership theme -->
<h2 style="color:#63a4ff;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">?</span> How NVD & MITRE Work Together
</h2>

<!-- Two-column comparison layout using CSS Grid -->
<div style="display:grid;grid-template-columns:1fr 1fr;gap:24px;margin-bottom:24px;">
<!-- MITRE's responsibilities column -->
<div style="background:#232b45;border-radius:8px;padding:20px;">
<h3 style="color:#fca311;margin:0 0 12px 0;">MITRE's Role</h3>
<ul style="color:#a9adc1;margin-left:16px;">
<li>Creates CVE identifiers</li>
<li>Maintains CWE catalog</li>
<li>Coordinates with researchers</li>
<li>Ensures quality standards</li>
</ul>
</div>
<!-- NVD's responsibilities column -->
<div style="background:#232b45;border-radius:8px;padding:20px;">
<h3 style="color:#42d392;margin:0 0 12px 0;">NVD's Role</h3>
<ul style="color:#a9adc1;margin-left:16px;">
<li>Adds CVSS scores to CVEs</li>
<li>Provides detailed analysis</li>
<li>Links to technical references</li>
<li>Offers API access for automation</li>
</ul>
</div>
</div>

<!-- Process flow description with centered emphasis -->
<div style="background:#263159;border-radius:8px;padding:16px;text-align:center;">
<strong style="color:#63a4ff;">The Process:</strong><br>
<span style="color:#e2e8f0;">MITRE assigns CVE ? Security researchers provide details ?
NVD analyzes and scores ? Public database updated ? Security tools worldwide sync the
data</span>
</div>
</section>

<!-- Career-focused section for professional development -->
<section style="background:#232b45;border-radius:14px;padding:28px;">
<!-- Education-themed header -->
<h2 style="color:#63a4ff;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">?</span> Why This Matters for Your Career
</h2>

<!-- Role-specific information grid -->

```



```
<div style="display:grid;gap:20px;">
<!-- Security professionals section -->
<div style="border-left:4px solid #42d392;padding-left:20px;">
<h3 style="color:#42d392;margin:0 0 8px 0;">For Security Professionals</h3>
<span style="color:#e2e8f0;">Understanding NVD and MITRE helps you communicate
effectively with global security teams, choose the right tools, and implement
industry-standard vulnerability management processes.</span>
</div>

<!-- Developers section with distinct color coding -->
<div style="border-left:4px solid #f8a541;padding-left:20px;">
<h3 style="color:#f8a541;margin:0 0 8px 0;">For Developers</h3>
<span style="color:#e2e8f0;">Knowledge of CVE/CWE helps you understand security
patterns, write more secure code, and participate in responsible disclosure when you
find vulnerabilities.</span>
</div>

<!-- Researchers section with academic focus -->
<div style="border-left:4px solid #3b8ded;padding-left:20px;">
<h3 style="color:#3b8ded;margin:0 0 8px 0;">For Researchers</h3>
<span style="color:#e2e8f0;">These frameworks provide the foundation for security
research, academic papers, and contribute to the global knowledge base of
cybersecurity.</span>
</div>
</div>
</section>
</div>
{% endblock %}
```

templates\pages\cve_detail.html

```
{% extends "base.html" %}

{% block content %}
<div style="max-width: 1400px; margin: 2em auto; background: #1a2236; border-radius: 16px; box-shadow: 0 8px 32px rgba(0,0,0,0.3); padding: 2.5em 3em; color: #e2e8f0;">

<div style="display: flex; align-items: center; justify-content: space-between; margin-bottom: 2em; padding-bottom: 1.5em; border-bottom: 2px solid #263159;">
<div style="flex: 1;">
<h1 style="margin: 0; font-size: 2.2em; color: #C9B896; letter-spacing: 0.5px;">
{{ cve.get('ID', '-') }}
</h1>
<div style="display: flex; gap: 2em; margin-top: 1em; color: #94a3b8; font-size: 1em;">
<div><strong>Published:</strong>&nbsp;{{ cve.get('Published', '-')[:10] }}</div>
<div><strong>Last Modified:</strong>&nbsp;{{ cve.get('lastModified', '-')[:10] if
cve.get('lastModified') else '-' }}</div>
{% set metrics = cve.get('metrics', {}) %}
{% set cvss = metrics.get('cvssMetricV31', metrics.get('cvssMetricV30',
metrics.get('cvssMetricV2', []))) %}
{% if cvss|length > 0 %}
{% set primary_metric = cvss[0] %}
{% set cvssData = primary_metric.get('cvssData', {}) %}
{% set header_score = cvssData.get('baseScore', 0) %}
<div><strong>CVSS:</strong>&nbsp;{{ header_score }}</div>
{% elif cve.get('CVSS_Score') %}
<div><strong>CVSS:</strong>&nbsp;{{ cve.get('CVSS_Score', '-') }}</div>
{% endif %}
</div>
</div>

{% if cve.get('Severity') %}
<div style="text-align: center;">
<div style="background: {% if cve.get('Severity') == 'CRITICAL' %}#f41717{% elif
cve.get('Severity') == 'HIGH' %}#fc2311{% elif cve.get('Severity') == 'MEDIUM'
%}#3498db{% elif cve.get('Severity') == 'LOW' %}#42d392{% else %}#888{% endif %}; color:
white; padding: 0.8em 1.5em; border-radius: 12px; font-weight: bold; font-size: 1.2em;
box-shadow: 0 4px 12px rgba(0,0,0,0.3);">
{{ cve.get('Severity', 'UNKNOWN') }}
</div>
<div style="margin-top: 8px; color: #94a3b8; font-size: 0.9em;">Severity Level</div>
</div>
{% endif %}
</div>

<div style="margin-bottom: 2.5em;">
<h2 style="margin: 0 0 1em 0; font-size: 1.4em; color: #cdd7ff;">Description</h2>
<div style="background: linear-gradient(135deg, #212d46 0%, #1e2640 100%); padding:
1.5em; border-radius: 12px; line-height: 1.7; border-left: 4px solid #C9B896;">
{{ cve.get('Description', 'No description available') }}
</div>
</div>

<div style="margin-bottom: 2.5em;">
<h2 style="margin: 0 0 1em 0; font-size: 1.4em; color: #cdd7ff;">Severity & Risk
Assessment</h2>
{% set metrics = cve.get('metrics', {}) %}
{% set cvss = metrics.get('cvssMetricV31', metrics.get('cvssMetricV30',
metrics.get('cvssMetricV2', []))) %}
{% if cvss|length > 0 %}
{% set primary_metric = cvss[0] %}
{% set cvssData = primary_metric.get('cvssData', {}) %}
{% set score = cvssData.get('baseScore', 0) %}
{% set severity = cvssData.get('baseSeverity', 'UNKNOWN') %}

<div style="background: linear-gradient(135deg, #232b45 0%, #1e2640 100%);
border-radius: 12px; padding: 1.5em; border: 1px solid rgba(201, 184, 150, 0.2);">
<div style="display: flex; justify-content: space-between; align-items: center;
margin-bottom: 1.5em;">
<div>
<div style="font-size: 2.5em; font-weight: bold; color: {% if severity == 'CRITICAL'
%}#f47174{% elif severity == 'HIGH' %}#fca311{% elif severity == 'MEDIUM' %}#3b8ded{%
elif severity == 'LOW' %}#42d392{% else %}#a9adc1{% endif %};">
```

```

{{ score }} <span style="font-size: 0.6em;">({{ severity }})</span>
</div>
</div>
<div style="flex: 1; max-width: 400px; margin-left: 2em;">
<div style="background: #263159; height: 20px; border-radius: 10px; overflow: hidden;
position: relative;">
<div style="background: linear-gradient(90deg, {% if severity == 'CRITICAL' %}#f47174{%
elif severity == 'HIGH' %}#fca311{% elif severity == 'MEDIUM' %}#3b8ded{% elif severity
== 'LOW' %}#42d392{% else %}#a9adc1{% endif %}, {% if severity == 'CRITICAL' %}#f47174{%
elif severity == 'HIGH' %}#fca311{% elif severity == 'MEDIUM' %}#3b8ded{% elif severity
== 'LOW' %}#42d392{% else %}#a9adc1{% endif %}); width: {{ (score/10*100)|round }}%;
height: 100%; border-radius: 10px;"></div>
<div style="position: absolute; top: 0; left: 0; right: 0; bottom: 0; display: flex;
align-items: center; justify-content: center; color: white; font-size: 0.8em;
font-weight: bold;">{{ score }}/10</div>
</div>
<div style="display: flex; justify-content: space-between; margin-top: 8px; font-size:
0.8em; color: #94a3b8;">
<span>0.0</span><span>Low</span><span>Medium</span><span>High</span><span>Critical</span>
<span>10.0</span>
</div>
</div>
</div>
<div>
<div style="font-size: 0.9em; color: #94a3b8; margin-bottom: 4px;">Vector String</div>
<div style="font-family: monospace; background: #263159; padding: 8px; border-radius:
6px; word-break: break-all; font-size: 0.9em;">{{ cvssData.get('vectorString', '-')
}}</div>
</div>
</div>
{% else %}
<div style="background: #212d46; padding: 1.5em; border-radius: 12px; text-align:
center; color: #94a3b8; border: 2px dashed #394a6b;">
<div style="font-size: 2em; margin-bottom: 8px;">?</div>
<div>No CVSS metrics available for this vulnerability</div>
</div>
{% endif %}
</div>

<div style="margin-bottom: 2.5em;">
<h2 style="margin: 0 0 1em 0; font-size: 1.4em; color: #cdd7ff;">Weakness Classification
(CWE)</h2>
{% if cve.get('CWE') %}
{% set cwe_code = cve.get('CWE') %}
<div style="background: linear-gradient(135deg, #232b45 0%, #1e2640 100%); padding:
1.5em; border-radius: 12px; border: 1px solid rgba(201, 184, 150, 0.2);">
<div style="display: flex; align-items: center; justify-content: space-between;
margin-bottom: 1em;">
<div>
<div style="font-size: 1.3em; font-weight: bold; color: #C9B896; margin-bottom: 4px;">{{
cwe_code }}</div>
<div style="color: #a9adc1; font-size: 1em;">
{% if cwe_code == 'CWE-79' %}Cross-Site Scripting (XSS)
{% elif cwe_code == 'CWE-89' %}SQL Injection
{% elif cwe_code == 'CWE-20' %}Improper Input Validation
{% elif cwe_code == 'CWE-22' %}Path Traversal
{% elif cwe_code == 'CWE-119' %}Buffer Overflow
{% elif cwe_code == 'CWE-200' %}Information Exposure
{% elif cwe_code == 'CWE-287' %}Improper Authentication
{% elif cwe_code == 'CWE-78' %}OS Command Injection
{% elif cwe_code == 'CWE-266' %}Incorrect Privilege Assignment
{% else %}Common Weakness Enumeration{% endif %}
</div>
</div>
<div style="display: flex; gap: 12px;">
{% if cwe_code.startswith('CWE-') %}
<a href="https://cwe.mitre.org/data/definitions/{{ cwe_code[4:] }}.html" target="_blank"
style="background: #263159; color: #C9B896; padding: 8px 16px; border-radius: 8px;
text-decoration: none; font-weight: 500;">View MITRE Details<span style="margin-left:
6px;">?</span></a>
<a href="{{ url_for('learn.learn_topic', topic='what-is-cwe') }}" style="background:
#42d392; color: white; padding: 8px 16px; border-radius: 8px; text-decoration: none;
font-weight: 500;">Learn About CWEs</a>
{% endif %}
</div>
</div>
<div style="background: #1a2236; padding: 12px; border-radius: 8px; border-left: 4px

```

```

solid #C9B896;">
<div style="color: #94a3b8; font-size: 0.9em; margin-bottom: 4px;">What this
means:</div>
<div style="color: #e2e8f0; line-height: 1.5;">
{% if cwe_code == 'CWE-79' %}This vulnerability allows attackers to inject malicious
scripts into web pages viewed by other users, potentially stealing sensitive information
or performing actions on their behalf.
{% elif cwe_code == 'CWE-89' %}This vulnerability allows attackers to manipulate
database queries, potentially accessing, modifying, or deleting sensitive data.
{% elif cwe_code == 'CWE-20' %}This vulnerability occurs when software fails to properly
validate input data, potentially allowing attackers to provide malicious input that
causes unexpected behavior.
{% elif cwe_code == 'CWE-266' %}This vulnerability involves incorrect assignment of
privileges, potentially allowing users to access resources or perform actions beyond
their intended authorization level.
{% else %}This represents a common pattern of software weakness that can lead to
security vulnerabilities. Click the links above to learn more about this specific
weakness type.{% endif %}
</div>
</div>
</div>
{% else %}
<div style="background: #212d46; padding: 1.5em; border-radius: 12px; text-align:
center; color: #94a3b8; border: 2px dashed #394a6b;">
<div style="font-size: 2em; margin-bottom: 8px;">?</div>
<div>No CWE classification available for this vulnerability</div>
</div>
{% endif %}
</div>

{% if cve.get('Products') and cve.get('Products')|length > 0 %}
<div style="margin-bottom: 2.5em;">
<h2 style="margin: 0 0 1em 0; font-size: 1.4em; color: #cdd7ff;">Affected Products</h2>
<div style="background: linear-gradient(135deg, #232b45 0%, #1e2640 100%); padding:
1.5em; border-radius: 12px; border: 1px solid rgba(201, 184, 150, 0.2);">
<div style="display: grid; gap: 8px;">
{% for product in cve.get('Products', []) %}
<div style="background: #263159; padding: 12px; border-radius: 8px; border-left: 4px
solid #f8a541;">
<span style="color: #f8a541; margin-right: 8px;">?</span>{{ product }}
</div>
{% endfor %}
</div>
</div>
</div>
{% endif %}

<div style="margin-bottom: 2.5em;">
<h2 style="margin: 0 0 1em 0; font-size: 1.4em; color: #cdd7ff;">References &
Additional Information</h2>
{% if cve.get('References') and cve.get('References')|length > 0 %}
<div style="background: linear-gradient(135deg, #232b45 0%, #1e2640 100%); padding:
1.5em; border-radius: 12px; border: 1px solid rgba(201, 184, 150, 0.2);">
<div style="display: grid; gap: 12px;">
{% for ref in cve.get('References', []) %}
<div style="background: #263159; padding: 14px; border-radius: 8px; display: flex;
align-items: center;" onmouseover="this.style.background='#2d3d6b'"
onmouseout="this.style.background='#263159'">
<span style="color: #C9B896; margin-right: 12px; font-size: 1.2em;">
{% if 'nvd.nist.gov' in ref %}?{% elif 'cve.mitre.org' in ref %}?{% elif 'github.com' in
ref %}?{% elif 'advisory' in ref.lower() or 'security' in ref.lower() %}?{% else %}?{%
endif %}
</span>
<a href="{{ ref }}" target="_blank" style="color: #C9B896; text-decoration: none; flex:
1; word-break: break-all; line-height: 1.4;">{{ ref }}</a>
<span style="margin-left: 8px; color: #94a3b8; font-size: 0.9em;">?</span>
</div>
{% endfor %}
</div>
</div>
{% else %}
<div style="background: #212d46; padding: 1.5em; border-radius: 12px; text-align:
center; color: #94a3b8; border: 2px dashed #394a6b;">
<div style="font-size: 2em; margin-bottom: 8px;">?</div>
<div>No additional references available</div>
</div>
{% endif %}

```

</div>

```
<div style="text-align: center; margin: 3em 0 1em 0; padding-top: 2em; border-top: 1px
solid #394a6b;">
<a href="{{ url_for('main.index') }}" style="background: linear-gradient(135deg, #9370DB
0%, #7B68EE 100%); color: white; padding: 12px 24px; border-radius: 8px; font-weight:
600; text-decoration: none; display: inline-flex; align-items: center;"
onmouseover="this.style.transform='translateY(-2px)'"
onmouseout="this.style.transform='translateY(0)'">
? Back to Dashboard
</a>
</div>
```

```
</div>
{% endblock %}
```

templates\pages\dashboard.html

```
{% extends "layouts/dashboard_layout.html" %}

{% block content %}
<div style="display: flex; flex-direction: column; gap: 36px">
<div style="display: flex; justify-content: space-between; align-items: center;">
<div>
<h1 style="margin: 0; color: #F5E8D8; letter-spacing: 1px; font-size: 1.8rem;">
Educational CVE Analysis Tool
</h1>
<div style="color: #a9adc1; margin-top: 4px; font-size: 1.07rem;">
Dashboard & Analytics
</div>
</div>
{% include 'components/filters/search_filters.html' %}
</div>

{% if note_text %}
<div style="font-size: 0.85em; color: #a9adc1; margin-bottom: 12px; font-style: italic;">
{{ note_text }}
</div>
{% endif %}

<div class="dashboard-row-1">
<div class="trend-sevi-blocks">
<div style="font-weight: 700; color: #F5E8D8; margin-bottom: 8px; font-size: 1.02rem; letter-spacing: 0.15px; text-align: left;">
CVE Trends (Last 30 days)
</div>
<canvas id="dailyTrendChart" height="70"></canvas>
</div>
{% include 'components/severity_cards.html' %}
</div>

<div class="dashboard-row-2">
{% include 'components/charts/pie_chart.html' %}
<section style="background: #1a2236; border-radius: 14px; padding: 30px 30px 10px 30px; flex: 2; display: flex; flex-direction: column; min-width: 350px; min-height: 250px; height: 300px;">
<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px;">
<div style="font-weight: 700; color: #F5E8D8; font-size: 1.02rem; letter-spacing: 0.15px;">
Vulnerabilities Over Time
</div>
<select id="timelineYearsFilter" style="background: #263159; color: #63a4ff; padding: 6px 12px; border-radius: 6px; border: none; cursor: pointer;">
<option value="1" {% if timeline_years == 1 %}selected{% endif %}>Last 1 Year</option>
<option value="2" {% if timeline_years == 2 %}selected{% endif %}>Last 2 Years</option>
<option value="3" {% if timeline_years == 3 %}selected{% endif %}>Last 3 Years</option>
<option value="5" {% if timeline_years == 5 %}selected{% endif %}>Last 5 Years</option>
</select>
</div>
<div style="position: relative; height: 250px;">
<canvas id="timelineChart"></canvas>
</div>
</section>
</div>

<div style="display: flex; gap: 25px; align-items: stretch;">
{% include 'components/charts/radar_chart.html' %}
</div>

<script>
window.timelineData = {{ timeline_data_years | tojson }};
window.timelineDataDaily = {{ timeline_data_days | tojson }};
window.severityStats = {{ severity_stats | tojson }};
window.cweRadar = {{ cwe_radar | tojson }};
window.cweRadarAll = {{ cwe_radar_all | tojson }};
window.cweRadarWeighted = {{ cwe_radar_weighted | tojson }};
window.cweRadarDescriptions = {{ cwe_radar_descriptions | tojson }};
{% if cwe_radar_all and cwe_radar_all.cwe_30_day_counts %}
window.cwe30DayCounts = {{ cwe_radar_all.cwe_30_day_counts | tojson }};
```

```
{% else %}  
window.cwe30DayCounts = {};  
{% endif %}  
window.cweMitigations = {};  
window.currentTimelineYears = {{ timeline_years }};  
</script>  
<script src="{{ url_for('static', filename='js/pages/dashboard.js') }}"></script>  
</div>  
{% endblock %}
```

templates\pages\vulnerabilities.html

```
{% extends "base.html" %}

{% block content %}
<div style="display: flex; flex-direction: column; gap: 32px">
<div style="display: flex; justify-content: space-between; align-items: center;">
<div>
<h1 style="margin: 0; color: #F5E8D8; letter-spacing: 1px; font-size: 1.8rem;">
Vulnerabilities Database
</h1>
<div style="color: #a9adc1; margin-top: 4px; font-size: 1.07rem;">
Complete CVE Listing & Search
</div>
</div>
{% include 'components/filters/search_filters.html' %}
</div>

{% if note_text %}
<div style="font-size: 0.85em; color: #a9adc1; margin-bottom: 12px; font-style: italic;">
{{ note_text }}
</div>
{% endif %}

<section style="background: #1a2236; padding: 28px; border-radius: 16px;">
<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 20px; cursor: pointer;" onclick="toggleCriticalHighSection()">
<div>
<h2 style="margin: 0; font-size: 1.6rem; color: #F5E8D8; display: flex; align-items: center;">
<span style="margin-right: 10px;">?</span> Latest Critical & High Vulnerabilities
<span id="collapseIcon" style="margin-left: 15px; font-size: 1.2rem; color: #a9adc1;">?</span>
</h2>
<div style="color: #94a3b8; margin-top: 6px; font-size: 0.95em;">
Most recent high-impact security vulnerabilities
</div>
</div>
</div>

<div id="criticalHighContent" style="display: block;">
<div style="display: grid; gap: 16px;">
{% set latest_critical_high = [] %}
{% for cve in latest_cves[:10] %}
{% set severity = cve.get('Severity', '') %}
{% set severity_str = severity if severity is string else (severity[0] if severity and severity|length > 0 else '') %}
{% if severity_str.upper() in ['CRITICAL', 'HIGH'] %}
{% set _ = latest_critical_high.append(cve) %}
{% endif %}
{% endfor %}

{% if latest_critical_high %}
{% for cve in latest_critical_high[:5] %}
{% set severity = cve.get('Severity', '') %}
{% set severity_str = severity if severity is string else (severity[0] if severity and severity|length > 0 else '') %}
<div style="background: #263159; border-radius: 12px; padding: 20px; border-left: 4px solid {% if severity_str == 'CRITICAL' %}#f47174{% else %}#fca311{% endif %};">
<div style="display: flex; justify-content: space-between; align-items: flex-start; margin-bottom: 12px;">
<div style="flex: 1;">
<div style="display: flex; align-items: center; gap: 12px; margin-bottom: 8px;">
<a href="{% url_for('main.cve_detail', cve_id=cve.get('ID'), year=year_filter, month=month_filter, page=current_page, severity=severity_filter) %}" style="color: #F5E8D8; font-weight: 700; font-size: 1.1em; text-decoration: none;">
{{ cve.get('ID', 'N/A') }}
</a>
<span style="background: {% if severity_str == 'CRITICAL' %}#f47174{% elif severity_str == 'HIGH' %}#fca311{% elif severity_str == 'MEDIUM' %}#3498db{% else %}#42d392{% endif %}; color: white; padding: 4px 10px; border-radius: 20px; font-size: 0.8em; font-weight: bold;">
{{ severity_str or 'UNKNOWN' }}
```



```

</span>
</div>
<div style="color: #e2e8f0; line-height: 1.5; font-size: 0.95em;">
  {{ cve.get('Description', 'No description available')[:200] }}
  {% if cve.get('Description', '')|length > 200 %}...{% endif %}
</div>
</div>
<div style="margin-left: 20px; text-align: right; color: #94a3b8; font-size: 0.85em;">
  {% if cve.get('Published') %}
  <div>{{ cve.get('Published')[:10] }}</div>
  {% endif %}
  {% if cve.get('CWE') %}
  <div style="margin-top: 4px;">{{ cve.get('CWE') }}</div>
  {% endif %}
</div>
</div>
</div>
{% endfor %}
{% else %}
<div style="text-align: center; color: #94a3b8; padding: 20px;">
<div style="font-size: 1.5rem; margin-bottom: 8px;">?</div>
<div>No critical or high severity vulnerabilities found in current dataset</div>
</div>
{% endif %}
</div>
</div>
</section>

<div style="display: flex; justify-content: space-between; align-items: center;
background: #1a2236; padding: 16px 24px; border-radius: 12px;">
<div style="color: #e2e8f0; font-weight: 600;">
<span style="color: #F5E8D8;">{{ total_results }}</span> vulnerabilities found
  {% if search_query or severity_filter or year_filter or month_filter %}
  <span style="color: #94a3b8; font-weight: normal; margin-left: 8px;">
    (filtered results)
  </span>
  {% endif %}
</div>
  {% if total_pages > 1 %}
  <div style="color: #94a3b8;">
    Page {{ current_page }} of {{ total_pages }}
  </div>
  {% endif %}
</div>

<section style="background: #1a2236; border-radius: 16px; overflow: hidden; box-shadow:
0 4px 20px rgba(0,0,0,0.1);">
<div style="padding: 24px 28px; border-bottom: 1px solid #2a364f;">
<h2 style="margin: 0; font-size: 1.3rem; color: #e2e8f0; display: flex; align-items:
center;">
  <span style="margin-right: 10px;">?</span> All Vulnerabilities
</h2>
</div>

  {% if latest_cves %}
  <div style="overflow-x: auto;">
  <table style="width: 100%; border-collapse: collapse;">
  <thead>
  <tr style="background: #212d46;">
  <th style="padding: 16px 20px; text-align: center; color: #94a3b8; font-weight: 600;
font-size: 1.1em; border-bottom: 1px solid #2a364f;">CVE ID</th>
  <th style="padding: 16px 20px; text-align: center; color: #94a3b8; font-weight: 600;
font-size: 1.1em; border-bottom: 1px solid #2a364f;">Description</th>
  <th style="padding: 16px 20px; text-align: center; color: #94a3b8; font-weight: 600;
font-size: 1.1em; border-bottom: 1px solid #2a364f;">Severity</th>
  <th style="padding: 16px 20px; text-align: center; color: #94a3b8; font-weight: 600;
font-size: 1.1em; border-bottom: 1px solid #2a364f;">Published</th>
  <th style="padding: 16px 20px; text-align: center; color: #94a3b8; font-weight: 600;
font-size: 1.1em; border-bottom: 1px solid #2a364f;">CWE</th>
  </tr>
  </thead>
  <tbody>
  {% for cve in latest_cves %}
  {% set severity = cve.get('Severity', '') %}
  {% set severity_str = severity if severity is string else (severity[0] if severity and
severity|length > 0 else '') %}
  <tr style="background: #1a2236; border-bottom: 1px solid #2a364f; transition: background

```

```

0.2s;" onmouseover="this.style.background='#212d46'"
onmouseout="this.style.background='#1a2236'">
<td style="padding: 16px 20px;text-align: center; font-size: 1.18rem;">
<a href="{{ url_for('main.cve_detail', cve_id=cve.get('ID'), year=year_filter,
month=month_filter, page=current_page, severity=severity_filter) }}" style="color:
#F5E8D8; font-weight: 600; text-decoration: none; font-family: monospace;">
{{ cve.get('ID', 'N/A') }}
</a>
</td>
<td style="padding: 16px 20px; color: #e2e8f0; line-height: 1.4; max-width: 400px;">
{{ cve.get('Description', 'No description available')[:120] }}
{% if cve.get('Description', '')|length > 120 %}...{% endif %}
</td>
<td style="padding: 16px 20px; text-align: center;">
<span style="background: {% if severity_str == 'CRITICAL' %}#f47174{% elif severity_str
== 'HIGH' %}#fca311{% elif severity_str == 'MEDIUM' %}#3498db{% elif severity_str ==
'LOW' %}#42d392{% else %}#6b7280{% endif %}; color: white; padding: 6px 12px;
border-radius: 20px; font-size: 0.8em; font-weight: bold;">
{{ severity_str or 'UNKNOWN' }}
</span>
</td>
<td style="padding: 16px 20px; text-align: center; color: #94a3b8; font-size: 1.18em;
font-family: monospace;">
{% if cve.get('Published') %}
{{ cve.get('Published')[:10] }}
{% else %}
-
{% endif %}
</td>
<td style="padding: 16px 20px; text-align: center; color: #94a3b8; font-size: 1.18em;
font-family: monospace;">
{% if cve.get('CWE') %}
{{ cve.get('CWE') }}
{% else %}
-
{% endif %}
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}
<div style="padding: 40px; text-align: center; color: #94a3b8;">
<div style="font-size: 3rem; margin-bottom: 16px;">?</div>
<div style="font-size: 1.1rem; margin-bottom: 8px;">No vulnerabilities found</div>
<div style="font-size: 0.9rem;">Try adjusting your filters or search terms</div>
</div>
{% endif %}
</section>

{% if total_pages > 1 %}
<div style="display: flex; justify-content: center; align-items: center; gap: 8px;
margin-top: 20px;">
{% if current_page > 1 %}
<a href="{{ url_for('main.vulnerabilities', page=current_page-1, q=search_query,
severity=severity_filter, year=year_filter, month=month_filter) }}" style="background:
#263159; color: #F5E8D8; padding: 10px 16px; border-radius: 8px; text-decoration: none;
font-weight: 600;">
? Previous
</a>
{% endif %}

{% for page_num in page_numbers %}
{% if page_num == current_page %}
<span style="background: #D4BA7F; color: white; padding: 10px 14px; border-radius: 8px;
font-weight: 700;">
{{ page_num }}
</span>
{% else %}
<a href="{{ url_for('main.vulnerabilities', page=page_num, q=search_query,
severity=severity_filter, year=year_filter, month=month_filter) }}" style="background:
#263159; color: #a9adc1; padding: 10px 14px; border-radius: 8px; text-decoration: none;
font-weight: 600;">
{{ page_num }}
</a>
{% endif %}

```

```

{% endfor %}

{% if current_page < total_pages %}
<a href="{{ url_for('main.vulnerabilities', page=current_page+1, q=search_query,
severity=severity_filter, year=year_filter, month=month_filter) }}" style="background:
#263159; color: #F5E8D8; padding: 10px 16px; border-radius: 8px; text-decoration: none;
font-weight: 600;">
Next ?
</a>
{% endif %}
</div>
{% endif %}
</div>

<script>
function toggleCriticalHighSection() {
const content = document.getElementById('criticalHighContent');
const icon = document.getElementById('collapseIcon');
if (content.style.display === 'none') {
content.style.display = 'block';
icon.textContent = '?';
} else {
content.style.display = 'none';
icon.textContent = '?';
}
}
</script>

<script src="{{ url_for('static', filename='js/pages/vulnerabilities.js') }}"></script>

<div style="text-align:center; margin:2.5em 0 1em 0;">
<a href="{{ url_for('main.index') }}" style="color:#F5E8D8; padding:0.8em 2em;
border-radius:6px; font-weight:600; text-decoration:none;">
? Back to Dashboard
</a>
</div>
{% endblock %}

```

templates\references\apis-feeds.html

```
<!-- APIs & Data Feeds Section -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with technical icon and green accent color -->
<h2 style="color:#42d392;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">?</span> APIs & Data Feeds
</h2>

<!-- Section introduction explaining technical focus -->
<div style="color:#e2e8f0;margin-bottom:20px;">
Technical interfaces and data formats used to programmatically access vulnerability
information.
</div>

<!-- NVD API 2.0 Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<!-- API title with consistent blue branding -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">NVD API 2.0</h3>
<!-- API description for context -->
<div style="color:#e2e8f0;margin-bottom:16px;">
RESTful API providing JSON responses for CVE data, CVSS metrics, and vulnerability
details.
</div><!-- APIs & Data Feeds Section -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with technical icon and green accent color -->
<h2 style="color:#42d392;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">?</span> APIs & Data Feeds
</h2>

<!-- Section introduction explaining technical focus -->
<div style="color:#e2e8f0;margin-bottom:20px;">
Technical interfaces and data formats used to programmatically access vulnerability
information.
</div>

<!-- NVD API 2.0 Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<!-- API title with consistent blue branding -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">NVD API 2.0</h3>
<!-- API description for context -->
<div style="color:#e2e8f0;margin-bottom:16px;">
RESTful API providing JSON responses for CVE data, CVSS metrics, and vulnerability
details.
</div>
<!-- Code example with dark background and syntax highlighting -->
<div style="background:#263159;padding:12px;border-radius:6px;margin-bottom:16px;">
<code style="color:#42d392;font-family:monospace;">
GET https://services.nvd.nist.gov/rest/json/cves/2.0
</code>
</div>
<!-- Responsive grid for technical specifications -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(200px,1fr));gap:12px;">
<!-- Rate limiting information with specific numeric constraints -->
<div>
<strong style="color:#fca311;">Rate Limits:</strong><br>
<span style="color:#a9adc1;">5 requests/30 sec (no key)<br>50 requests/30 sec (with
key)</span>
</div>
<!-- Response format specification -->
<div>
<strong style="color:#fca311;">Response Format:</strong><br>
<span style="color:#a9adc1;">JSON with CVE objects</span>
</div>
<!-- Key API parameters for developers -->
<div>
<strong style="color:#fca311;">Key Parameters:</strong><br>
<span style="color:#a9adc1;">cveId, pubStartDate, lastModStartDate</span>
</div>
</div>

<!-- Data Formats & Standards Documentation Block -->
```

```
<div style="background:#232b45;border-radius:8px;padding:20px;">
<!-- Standards section title -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Data Formats & Standards</h3>
<!-- Grid layout for different data format specifications -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(250px,1fr));gap:16px;">
<!-- CVE JSON format specification -->
<div>
<strong style="color:#42d392;">CVE JSON 5.0</strong><br>
<span style="color:#a9adc1;">Standard format for CVE vulnerability data</span>
</div>
<!-- CVSS scoring methodology reference -->
<div>
<strong style="color:#42d392;">CVSS v3.1</strong><br>
<span style="color:#a9adc1;">Vulnerability scoring methodology</span>
</div>
<!-- CWE XML format specification -->
<div>
<strong style="color:#42d392;">CWE XML</strong><br>
<span style="color:#a9adc1;">Machine-readable weakness catalog</span>
</div>
</div>
</section>
<!-- Code example with dark background and syntax highlighting -->
<div style="background:#263159;padding:12px;border-radius:6px;margin-bottom:16px;">
<code style="color:#42d392;font-family:monospace;">
GET https://services.nvd.nist.gov/rest/json/cves/2.0
</code>
</div>
<!-- Responsive grid for technical specifications -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(200px,1fr));gap:12px;">
<!-- Rate limiting information with specific numeric constraints -->
<div>
<strong style="color:#fca311;">Rate Limits:</strong><br>
<span style="color:#a9adc1;">5 requests/30 sec (no key)<br>50 requests/30 sec (with
key)</span>
</div>
<!-- Response format specification -->
<div>
<strong style="color:#fca311;">Response Format:</strong><br>
<span style="color:#a9adc1;">JSON with CVE objects</span>
</div>
<!-- Key API parameters for developers -->
<div>
<strong style="color:#fca311;">Key Parameters:</strong><br>
<span style="color:#a9adc1;">cveId, pubStartDate, lastModStartDate</span>
</div>
</div>

<!-- Data Formats & Standards Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;">
<!-- Standards section title -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Data Formats & Standards</h3>
<!-- Grid layout for different data format specifications -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(250px,1fr));gap:16px;">
<!-- CVE JSON format specification -->
<div>
<strong style="color:#42d392;">CVE JSON 5.0</strong><br>
<span style="color:#a9adc1;">Standard format for CVE vulnerability data</span>
</div>
<!-- CVSS scoring methodology reference -->
<div>
<strong style="color:#42d392;">CVSS v3.1</strong><br>
<span style="color:#a9adc1;">Vulnerability scoring methodology</span>
</div>
<!-- CWE XML format specification -->
<div>
<strong style="color:#42d392;">CWE XML</strong><br>
<span style="color:#a9adc1;">Machine-readable weakness catalog</span>
</div>
</div>
</div>
</section>
```

templates\references\data-sources.html

```
<!-- Primary Data Sources Educational Section -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with government/institutional icon and orange accent -->
<h2 style="color:#fca311;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">??</span> Primary Data Sources
</h2>

<!-- Section introduction explaining the purpose and context -->
<div style="color:#e2e8f0;margin-bottom:20px;">
The foundational databases and organizations that provide the core vulnerability data
used in VulnEdu.
</div>

<!-- National Vulnerability Database (NVD) Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<!-- NVD title with blue accent color for consistency -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">National Vulnerability Database (NVD)</h3>
<!-- Structured Who/What/Why information for organizational context -->
<div style="color:#e2e8f0;margin-bottom:16px;">
<strong>Who:</strong> Managed by NIST, part of the U.S. Department of Commerce<br>
<strong>What:</strong> The U.S. government's repository of standards-based vulnerability
management data<br>
<strong>Why it matters:</strong> Provides comprehensive vulnerability information with
standardized severity scores (CVSS)
</div>
<!-- Detailed services breakdown with visual distinction -->
<div style="background:#263159;border-radius:6px;padding:16px;margin-bottom:16px;">
<h4 style="color:#42d392;margin:0 0 8px 0;">What NVD Provides:</h4>
<!-- Bulleted list of specific services and capabilities -->
<ul style="color:#a9adc1;margin-left:16px;">
<li><strong>CVSS Scores:</strong> Standardized severity ratings for vulnerabilities</li>
<li><strong>CPE Names:</strong> Standardized naming for IT products and platforms</li>
<li><strong>CWE Mappings:</strong> Links vulnerabilities to weakness categories</li>
<li><strong>Reference Links:</strong> Technical details, patches, and advisories</li>
<li><strong>API Access:</strong> Machine-readable data feeds for automation</li>
</ul>
</div>
<!-- Flexbox layout for external resource links -->
<div style="display:flex;gap:16px;flex-wrap:wrap;">
<a href="https://nvd.nist.gov/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Official Website</a>
<a href="https://nvd.nist.gov/developers" target="_blank"
style="color:#63a4ff;text-decoration:underline;">API Documentation</a>
<a href="https://nvd.nist.gov/developers/vulnerabilities" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Vulnerability API</a>
</div>
</div>

<!-- MITRE CVE Program Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<!-- CVE title with green accent to distinguish from NVD -->
<h3 style="color:#42d392;margin:0 0 16px 0;">MITRE CVE Program</h3>
<!-- CVE program description with focus on standardization role -->
<div style="color:#e2e8f0;margin-bottom:16px;">
<strong>What:</strong> The authoritative source for CVE identifiers and the CVE naming
standard<br>
<strong>Why:</strong> CVE IDs provide universal references for vulnerabilities across
all security tools<br>
<strong>How we use it:</strong> CVE ID validation, cross-referencing, and understanding
vulnerability disclosure processes
</div>
<!-- CVE-specific resource links -->
<div style="display:flex;gap:16px;flex-wrap:wrap;">
<a href="https://www.cve.org/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CVE Main Site</a>
<a href="https://cve.mitre.org/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CVE Details</a>
</div>
</div>

<!-- MITRE Common Weakness Enumeration (CWE) Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;">
```

```
<!-- CWE title with blue accent to distinguish from other MITRE programs -->
<h3 style="color:#3b8ded;margin:0 0 16px 0;">MITRE Common Weakness Enumeration
(CWE)</h3>
<!-- CWE program description emphasizing classification and education -->
<div style="color:#e2e8f0;margin-bottom:16px;">
<strong>What:</strong> A comprehensive catalog of software and hardware weakness
types<br>
<strong>Why:</strong> Provides classification framework for understanding root causes of
vulnerabilities<br>
<strong>How we use it:</strong> CWE mapping, weakness analysis, and educational content
about vulnerability patterns
</div>
<!-- CWE-specific resource links including educational materials -->
<div style="display:flex;gap:16px;flex-wrap:wrap;">
<a href="https://cwe.mitre.org/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CWE Main Site</a>
<a href="https://cwe.mitre.org/data/index.html" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CWE Data Downloads</a>
<a href="https://cwe.mitre.org/top25/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CWE Top 25</a>
</div>
</div>
</section>
```

templates\references\dev_tools.html

```
<!-- Development Tools & Frameworks Documentation Section -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with tools icon and red accent for technical focus -->
<h2 style="color:#f47174;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">??</span> Development Tools & Frameworks
</h2>

<!-- Introduction explaining the purpose and context of tools documentation -->
<div style="color:#e2e8f0;margin-bottom:20px;">
These are the core tools and libraries that power <strong>VulnEdu</strong>.
They allow us to fetch CVE data, process it, and present it visually for education.
</div>

<!-- Web Development Stack Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<!-- Stack section title with blue accent for consistency -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Web Development Stack</h3>
<!-- Responsive grid layout for development tools -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(230px,1fr));gap:20px;">
<!-- Flask web framework entry -->
<div>
<strong style="color:#42d392;">Flask</strong><br>
<span style="color:#a9adc1;">Python web framework</span><br>
<a href="https://flask.palletsprojects.com/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Documentation</a>
</div>
<!-- Chart.js visualization library entry -->
<div>
<strong style="color:#42d392;">Chart.js</strong><br>
<span style="color:#a9adc1;">JavaScript charting library</span><br>
<a href="https://www.chartjs.org/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Documentation</a>
</div>
<!-- Pandas data analysis library entry -->
<div>
<strong style="color:#42d392;">Pandas</strong><br>
<span style="color:#a9adc1;">Python data analysis</span><br>
<a href="https://pandas.pydata.org/docs/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Documentation</a>
</div>
<!-- Requests HTTP library entry -->
<div>
<strong style="color:#42d392;">Requests</strong><br>
<span style="color:#a9adc1;">Python HTTP library</span><br>
<a href="https://requests.readthedocs.io/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Documentation</a>
</div>
</div>
</div>

<!-- Security Analysis Tools Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;">
<!-- Security tools section title -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Security Analysis Tools</h3>
<!-- Grid layout for security analysis tools -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(230px,1fr));gap:20px;">
<!-- OpenVAS vulnerability scanner entry -->
<div>
<strong style="color:#f47174;">OpenVAS</strong><br>
<span style="color:#a9adc1;">Open-source vulnerability scanner</span><br>
<a href="https://www.openvas.org/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Project Site</a>
</div>
<!-- Nessus professional scanner entry -->
<div>
<strong style="color:#f47174;">Nessus</strong><br>
<span style="color:#a9adc1;">Professional vulnerability scanner</span><br>
<a href="https://www.tenable.com/products/nessus" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Product Page</a>
</div>
</div>
</div>
```



```
<!-- OWASP ZAP web application scanner entry -->
<div>
<strong style="color:#f47174;">OWASP ZAP</strong><br>
<span style="color:#a9adc1;">Web application security scanner</span><br>
<a href="https://zapproxy.org/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Project Site</a>
</div>
</div>
</div>
</section>
```

templates\references\references.html

```
{% extends "base.html" %}

{% block content %}
<!-- Main container with consistent max-width and spacing -->
<div style="max-width:1400px;margin:0 auto 32px auto;padding-top:20px;">
<!-- Page Header -->
<!-- Primary title with brand color and consistent typography -->
<h1 style="color:#63a4ff;">References & Resources</h1>
<!-- Comprehensive page description explaining purpose and value -->
<p style="color:#e2e8f0;font-size:1.1em;line-height:1.6;margin-bottom:32px;">
This page contains all the authoritative sources, APIs, documentation,
and resources that power VulnEdu. Understanding these resources helps you dive deeper
into cybersecurity
vulnerability research and build your own security tools.
</p>

<!-- Quick Navigation - Scroll-to-section links for single-page experience -->
<div style="background:#1a2236;border-radius:12px;padding:20px;margin-bottom:32px;">
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Quick Navigation</h3>
<!-- Responsive grid for navigation buttons -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(200px,1fr));gap:12px;">
<!-- Anchor links with consistent styling and emoji icons -->
<a href="#data-sources" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">?? Primary Data Sources</a>
<a href="#apis-feeds" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">? APIs & Data Feeds</a>
<a href="#tech-docs" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">?? Technical
Documentation</a>
<a href="#dev-tools" style="color:#63a4ff;text-decoration:none;padding:8px
12px;background:#263159;border-radius:6px;font-weight:500;">?? Development Tools</a>
</div>
</div>

<!-- Resource Categories Overview -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<h2 style="color:#63a4ff;margin:0 0 20px 0;">Resource Categories</h2>
<!-- Grid layout for category descriptions -->
<div style="display:grid;gap:20px;">
<!-- Data Sources category with orange accent -->
<div style="border-left:4px solid #f8a541;padding-left:20px;">
<h3 style="color:#fca311;margin:0 0 8px 0;">Data Sources</h3>
<div style="color:#e2e8f0;margin-bottom:12px;">
The foundational databases (NVD, MITRE) that provide core vulnerability data.
</div>
</div>
<!-- APIs & Feeds category with green accent -->
<div style="border-left:4px solid #42d392;padding-left:20px;">
<h3 style="color:#42d392;margin:0 0 8px 0;">APIs & Feeds</h3>
<div style="color:#e2e8f0;margin-bottom:12px;">
Technical interfaces for programmatic access to vulnerability information.
</div>
</div>
<!-- Standards & Documentation category with blue accent -->
<div style="border-left:4px solid #3b8ded;padding-left:20px;">
<h3 style="color:#3b8ded;margin:0 0 8px 0;">Standards & Documentation</h3>
<div style="color:#e2e8f0;margin-bottom:12px;">
CVSS scoring methodology, security standards, and technical specifications.
</div>
</div>
<!-- Tools & Frameworks category with red accent -->
<div style="border-left:4px solid #f47174;padding-left:20px;">
<h3 style="color:#f47174;margin:0 0 8px 0;">Tools & Frameworks</h3>
<div style="color:#e2e8f0;margin-bottom:12px;">
Development tools, security scanners, and frameworks used in the project.
</div>
</div>
</div>
</section>

<!-- Modular Content Sections - Template Includes for Maintainability -->
```

```

<!-- Data Sources section with anchor ID for navigation -->
<div id="data-sources">
{% include 'references/data-sources.html' %}
</div>

<!-- APIs & Data Feeds section -->
<div id="apis-feeds">
{% include 'references/apis-feeds.html' %}
</div>

<!-- Technical Documentation section -->
<div id="tech-docs">
{% include 'references/tech-docs.html' %}
</div>

<!-- Development Tools section -->
<div id="dev-tools">
{% include 'references/dev_tools.html' %}
</div>

<!-- Attribution & Acknowledgments Section -->
<section style="background:#1a2236;border-radius:14px;padding:28px;">
<h2 style="color:#63a4ff;margin:0 0 20px 0;">Attribution & Acknowledgments</h2>
<div style="color:#e2e8f0;line-height:1.6;">
<!-- Introduction to attribution importance -->
<p>VulnEdu is built using publicly available data from government and nonprofit organizations dedicated to improving cybersecurity. We acknowledge:</p>
<!-- List of acknowledged organizations and contributions -->
<ul style="margin-left:20px;color:#a9adc1;">
<li><strong>NIST</strong> for maintaining the National Vulnerability Database</li>
<li><strong>MITRE Corporation</strong> for the CVE and CWE programs</li>
<li><strong>FIRST.org</strong> for the CVSS scoring methodology</li>
<li><strong>OWASP</strong> for educational resources and best practices</li>
<li>The global <strong>cybersecurity research community</strong> for continuous improvements</li>
</ul>
</div>
</section>
</div>

<!-- Navigation back to dashboard -->
<div style="text-align:center; margin:2.5em 0 1em 0;">
<a href="{{ url_for('main.index') }}" style="color:#63a4ff; padding:0.8em 2em; border-radius:6px; font-weight:600; text-decoration:none;">
? Back to Dashboard
</a>
</div>

<!-- JavaScript Enhancement for Smooth Scrolling Navigation -->
<script>
// Smooth scrolling implementation for better user experience
document.addEventListener('DOMContentLoaded', function() {
// Select all anchor links that point to sections on the same page
const links = document.querySelectorAll('a[href^="#"]');
for (const link of links) {
// Add click event listener to each anchor link
link.addEventListener('click', function(e) {
// Prevent default anchor link behavior
e.preventDefault();
// Extract target element ID from href attribute
const targetId = this.getAttribute('href').substring(1);
// Find the target element by ID
const targetElement = document.getElementById(targetId);
if (targetElement) {
// Smooth scroll to target element with modern API
targetElement.scrollIntoView({
behavior: 'smooth', // Smooth scrolling animation
block: 'start' // Align to top of viewport
});
}
});
}
});
</script>
{% endblock %}

```

templates\references\tech-docs.html

```
<!-- Technical Documentation & Standards Section -->
<section style="background:#1a2236;border-radius:14px;padding:28px;margin-bottom:32px;">
<!-- Section header with gear icon and blue accent for technical content -->
<h2 style="color:#3b8ded;margin:0 0 20px 0;display:flex;align-items:center;">
<span style="margin-right:12px;">??</span> Technical Documentation & Standards
</h2>

<!-- Section introduction explaining scope and importance -->
<div style="color:#e2e8f0;margin-bottom:20px;">
Industry standards, methodologies, and technical specifications that form the foundation
of vulnerability analysis.
</div>

<!-- Common Vulnerability Scoring System (CVSS) Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;margin-bottom:20px;">
<!-- CVSS title with consistent blue branding -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Common Vulnerability Scoring System
(CVSS)</h3>
<!-- CVSS description establishing authority and purpose -->
<div style="color:#e2e8f0;margin-bottom:16px;">
Industry standard for assessing the severity of computer system security
vulnerabilities.
</div>
<!-- Flexbox layout for CVSS-related resources -->
<div style="display:flex;gap:16px;flex-wrap:wrap;">
<!-- Official CVSS website link -->
<a href="https://www.first.org/cvss/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CVSS Official Site</a>
<!-- Current specification document for technical implementation -->
<a href="https://www.first.org/cvss/v3.1/specification-document" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CVSS v3.1 Specification</a>
<!-- Interactive calculator for practical scoring experience -->
<a href="https://www.first.org/cvss/calculator/3.1" target="_blank"
style="color:#63a4ff;text-decoration:underline;">CVSS Calculator</a>
</div>
</div>

<!-- Security Standards & Guidelines Documentation Block -->
<div style="background:#232b45;border-radius:8px;padding:20px;">
<!-- Standards section title -->
<h3 style="color:#63a4ff;margin:0 0 16px 0;">Security Standards & Guidelines</h3>
<!-- Responsive grid for compliance framework organization -->
<div
style="display:grid;grid-template-columns:repeat(auto-fit,minmax(300px,1fr));gap:16px;">
<!-- ISO 27001/27002 international standards entry -->
<div>
<strong style="color:#42d392;">ISO 27001/27002</strong><br>
<span style="color:#a9adc1;">Information security management standards</span><br>
<a href="https://www.iso.org/isoiec-27001-information-security.html" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Learn More</a>
</div>
<!-- NIST federal security controls framework entry -->
<div>
<strong style="color:#42d392;">NIST SP 800-53</strong><br>
<span style="color:#a9adc1;">Security controls for federal information
systems</span><br>
<a href="https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final"
target="_blank" style="color:#63a4ff;text-decoration:underline;">Publication</a>
</div>
<!-- PCI DSS payment industry compliance standard entry -->
<div>
<strong style="color:#42d392;">PCI DSS</strong><br>
<span style="color:#a9adc1;">Payment card industry data security standard</span><br>
<a href="https://www.pcisecuritystandards.org/" target="_blank"
style="color:#63a4ff;text-decoration:underline;">Official Site</a>
</div>
</div>
</div>
</section>
```

utils\filters.py

```
from typing import List, Dict, Any, Optional
from datetime import datetime, timezone, timedelta

class FilterService:
    """Service for filtering and pagination operations"""

    def filter_by_severity(self, cves: List[Dict], severity_filter: str) -> List[Dict]:
        """Filter CVEs by severity"""
        # Return original list if no filter specified
        if not severity_filter:
            return cves

        # Convert to uppercase for case-insensitive comparison
        severity_upper = severity_filter.upper()

        # Handle special "UNKNOWN" case - CVEs without standard severity ratings
        if severity_upper == 'UNKNOWN':
            return [
                cve for cve in cves
            ]
        # Check if severity is not one of the standard levels
        if severity_upper not in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
            # Handle standard severity levels
            elif severity_upper in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
                return [
                    cve for cve in cves
                ]
            # Match the exact severity level (case-insensitive)
            if cve.get('Severity', '').upper() == severity_upper:
                # Return unfiltered list if filter doesn't match any known severity
                return cves

    def filter_by_search(self, cves: List[Dict], search_query: str) -> List[Dict]:
        """Filter CVEs by search query"""
        # Return original list if no search query provided
        if not search_query:
            return cves

        # Convert to lowercase and remove whitespace for consistent matching
        q_lower = search_query.lower().strip()

        # Special handling for CWE (Common Weakness Enumeration) searches
        # CWE format: "cwe-" followed by digits (e.g., "cwe-79")
        if q_lower.startswith("cwe-") and q_lower[4:].isdigit():
            return [
                cve for cve in cves
            ]
        # Exact match for CWE field
        if (cve.get("CWE") or "").lower() == q_lower:
            # General text search in ID and Description
            filtered_cves = []
            for cve in cves:
                # Get fields and convert to lowercase for case-insensitive search
                cve_id = cve.get('ID', '').lower()
                description = cve.get('Description', '').lower()
                cwe = cve.get('CWE', '').lower()

                # Check if search query appears in any of the searchable fields
                if (q_lower in cve_id or
                    q_lower in description or
                    q_lower in cwe):
                    filtered_cves.append(cve)

            return filtered_cves

    def generate_page_numbers(self, current_page: int, total_pages: int) -> List[int]:
        """Generate page numbers for pagination"""
        # If 7 or fewer pages, show all page numbers
        if total_pages <= 7:
            return list(range(1, total_pages + 1))
```

```

# If user is on early pages (1-4), show first 7 pages
if current_page <= 4:
    return list(range(1, 8))
# If user is on final pages, show last 7 pages
elif current_page >= total_pages - 3:
    return list(range(total_pages - 6, total_pages + 1))
# For middle pages, show 7-page window centered on current page
else:
    return list(range(current_page - 3, current_page + 4))

def generate_note_text(self, year: Optional[int] = None,
month: Optional[int] = None,
day: Optional[int] = None) -> str:
    """Generate note text for data range"""
    # Most specific: year, month, and day provided
    if year and month and day:
        return f"Showing data from {year}-{month:02d}-{day:02d}"
    # Medium specific: year and month provided
    elif year and month:
        return f"Showing data from {year}-{month:02d}"
    # Least specific: only year provided
    elif year:
        return f"Showing data from {year}"
    # Default case: show 30-day rolling window
    else:
        # Use UTC timezone for consistency across deployments
        end_date = datetime.now(timezone.utc).date()
        # Calculate start date as 29 days ago (30 days total including today)
        start_date = end_date - timedelta(days=29)
        # Format dates in ISO format (YYYY-MM-DD)
        return f"Showing data from {start_date.strftime('%Y-%m-%d')} to {end_date.strftime('%Y-%m-%d')}"

```

utils\validators.py

```
import re
from typing import Optional, Dict, Any
from datetime import datetime

class ValidationService:
    """Service for input validation"""

    @staticmethod
    def validate_cve_id(cve_id: str) -> bool:
        """Validate CVE ID format"""
        if not cve_id:
            # Empty input is always invalid
            return False

        # CVE format should be CVE-YYYY-NNNN or CVE-YYYY-NNNNNNNN
        pattern = r'^CVE-\d{4}-\d{4,7}$'
        # Use uppercase to handle both cve- and CVE-
        return bool(re.match(pattern, cve_id.upper()))

    @staticmethod
    def validate_cwe_code(cwe_code: str) -> bool:
        """Validate CWE code format"""
        if not cwe_code:
            # Empty input is invalid
            return False

        # Pattern: CWE-N, where N is one or more digits
        pattern = r'^CWE-\d+$'
        # Use uppercase for consistency
        return bool(re.match(pattern, cwe_code.upper()))

    @staticmethod
    def validate_severity(severity: str) -> bool:
        """Validate severity level"""
        if not severity:
            # Empty input is invalid
            return False

        # Acceptable severity levels
        valid_severities = ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']
        # Return True if uppercased severity in the list
        return severity.upper() in valid_severities

    @staticmethod
    def validate_year(year: int) -> bool:
        """Validate year input"""
        if not year:
            # None/0 is invalid
            return False

        # Get current year
        current_year = datetime.now().year
        # Valid range: 1999 to current year
        return 1999 <= year <= current_year

    @staticmethod
    def validate_month(month: int) -> bool:
        """Validate month input"""
        if not month:
            # None/0 is invalid
            return False

        # Month must be between 1 and 12
        return 1 <= month <= 12

    @staticmethod
    def validate_day(day: int) -> bool:
        """Validate day input"""
        if not day:
            # None/0 is invalid
            return False
```

```
# Day must be between 1 and 31
return 1 <= day <= 31
```

```
@staticmethod
def sanitize_search_query(query: str) -> str:
    """Sanitize search query input"""
    if not query:
        # No input returns empty string
        return ""
```

```
# Remove unwanted characters, allowing only letters, digits, spaces, hyphens, and
periods
sanitized = re.sub(r'^\w\s\-.]', '', query)
# Trim whitespace and limit to 100 characters
return sanitized.strip()[:100]
```

```
@staticmethod
def validate_pagination(page: int, per_page: int) -> Dict[str, int]:
    """Validate and normalize pagination parameters"""
    # Ensure page minimum is 1
    page = max(1, page or 1)

    # per_page bounded between 1 and 100, default 20 if 0/None
    per_page = max(1, min(100, per_page or 20))

    # Return as dict for consistency and extensibility
    return {'page': page, 'per_page': per_page}
```